Contents

Preface **3**
Publication status 3
How this book is structured 3
The Quantum Angle 6
Worked Example 11
The Quantum Fourier Transform 29
Reality Check 37
Chef's Notes 37

Deep-Dive 3: The VQE Pipeline **39**
In This Chapter 39
Why electrons aren't qubits 39
Second quantisation: the language of electrons 40
Fermion-to-qubit encodings 41
Measuring the energy 42
The ansatz: parameterised chemistry 43
What you should take away 45

Unit 4: Machine Learning **47**
The Hook 47
The Bottleneck 47
The Quantum Angle 48
Worked Example 49
Building the kernel matrix and training the SVM 53
When does this actually help? 53
What you should take away 53

Unit 5: Finance **55**
The Hook 55
The Bottleneck 55
The Quantum Angle 56
From Grover to amplitude estimation 60
What you should take away 61

Unit 6: Supply Chains **63**
The Hook 63
The Bottleneck 63
The Quantum Angle 64
Reality Check 73
Chef's Notes 74

Deep-Dive 7: QPE and Trotterisation **76**
In This Chapter 76
QPE: from phases to eigenvalues 76
Trotterisation: implementing e^{-iHt} 78
Worked Example 82
Reality Check 83
Chef's Notes 83

Deep-Dive 8: Quantum Embedding Methods **85**
In This Chapter 85
The active space idea 85
DMET: Density Matrix Embedding Theory 86
The complete pipeline 87
What you should take away 87

The Elephant in the Room: Quantum Error Correction **89**
Why quantum computers make mistakes 89
The basic idea: redundancy without looking 89
Surface codes: the leading approach 90
LDPC codes: doing more with less 90
Error mitigation: the NISQ bridge 90
Where we are and where we're going 91

Deep-Dive 9: Inside Quantum Error Correction **92**
In This Chapter 92
The problem, concretely 92
The bit-flip code: your first quantum error-correcting code 92
The phase-flip code 94
The Shor code: correcting everything 95
The stabiliser formalism: the algebra behind it all 97
The surface code: why it wins 98
Why T-gates are expensive 99
What you should take away 99
Beyond stabiliser codes: the frontier 100

The Quantum Bottleneck **102**
What's real, what's next, what's unknown 102
The one thing to remember 102
Where to go from here 103
Annotated Bibliography 103

Preface

Most quantum computing books start with a qubit. Then two qubits. Then gates. Then circuits. Somewhere around chapter eight, if you're still reading, you might encounter a real problem.

This book does the opposite.

We start with problems. Real ones, with dollar figures attached. UPS saving $50 million a year by shaving one mile off delivery routes. Banks pricing derivatives with Monte Carlo simulations that take hours. Pharmaceutical companies spending $2 billion and twelve years to bring a single drug to market. Climate scientists searching for catalysts that could pull carbon from the atmosphere.

For each problem, we ask: what makes it hard? Not "hard" in the vague sense, but structurally hard. What is the specific computational bottleneck that defeats classical computers? And then: does quantum mechanics offer anything at precisely that bottleneck?

Sometimes the answer is yes. Sometimes the answer is "maybe, if hardware improves by two orders of magnitude." Sometimes the answer is "we honestly don't know yet." We give you all three, clearly labelled.

Publication status

This manuscript is being prepared for publication. Unless explicitly stated otherwise, the text, figures, and layout in this repository are unpublished material shared for review, technical validation, and submission only. All rights are reserved by the author.

How this book is structured

The book is organised into eight **units**, each built around one industry problem. Every unit has two parts:

- The **application chapter** tells the story. It opens with the industry context, identifies the computational bottleneck, shows how a quantum algorithm addresses it, walks through a toy example, and closes with an honest assessment of where things stand today. You need no prior quantum knowledge to read these chapters.
- The **deep-dive chapter** teaches the algorithm at the circuit level. It introduces quantum concepts (qubits, gates, interference, entanglement) organically, as the algorithm needs them. These chapters are more technical, but they build everything from scratch.

"'{figure} ../figures/book-zoom-rhythm.png :name: fig-book-zoom-rhythm :alt: The book zooms from industry stakes to bottlenecks, the quantum angle, a worked example, and a reality check.

This is the book's recurring teaching move: start wide with stakes, zoom into the bottleneck, introduce only the quantum machinery the chapter has earned, show a runnable toy example, then zoom back out for an honest reality check.

```
## Two ways to read this book

**The fast path.** Read only the application chapters (Units 1 through 8). You'll unders
world impact.

**The full path.** Read the application chapters *and* the deep-
dives. By the end, you'll understand QAOA, Shor's algorithm, VQE, quantum kernels, ampli

Either path works. The deep-dives never assume you've read other deep-
dives out of order; each one states its prerequisites explicitly.

After the eight units, a standalone chapter on **quantum error correction** explains why
and what it will take to get there. The book closes with a short **conclusion** that ties

## The notebooks

Every unit has a companion Jupyter notebook that runs a version of the algorithm on a rea

Where the notebook code looks more explicit than a production engineer might write, that
processing visible so you can inspect the mechanism end to end. A polished library would

## What we don't promise

We don't promise that quantum computers will revolutionise your industry next year. Some
tolerant quantum computers that don't exist yet. Every chapter includes a **Reality Chec

We use "could" and "in principle" where others use "will" and "soon." We cite specific re

## Who this book is for

You're curious about quantum computing, but you want to know *what it's for* before learn

If you're looking for a comprehensive quantum computing textbook, read Nielsen and Chuan

Let's start with a delivery truck.

# Unit 1: Logistics — *Saving $50 Million One Mile at a Time*


## The Hook

Every morning, 130,000 UPS drivers leave their depots. Each driver visits between 100 an

In 2012, UPS deployed a system called ORION: On-Road Integrated Optimization and Navigat

One mile doesn't sound like much. But multiply by 130,000 drivers, 250 working days per y

Now consider what a *two*-mile improvement would be worth. Or five.
```

```
The problem is that finding the optimal route isn't just difficult; it's one of the harde

This is the **Travelling Salesman Problem** (TSP), and it has tormented mathematicians,
hard.

In practice, nobody solves TSP exactly at scale. ORION doesn't find the best route; it fi
from simulated annealing to specialised graph algorithms — that have been refined over de

They work remarkably well. But they all share a fundamental limitation: they explore the

What if you could explore the entire landscape at once?


## The Bottleneck

Let's make the problem concrete. Strip away the geography and the delivery trucks, and wh

You have a graph; a set of cities (nodes) connected by roads (edges), each road with a cos

For our purposes, let's start with something even simpler: **MaxCut**. Take a graph and c

MaxCut sounds like it has nothing to do with delivery trucks. But TSP, MaxCut, and virtua
each decision is one qubit, and the cost function translates directly into the language o

Here's why MaxCut is hard. Each node is coloured one of two colours, so an $n$-
node graph has $2^n$ possible two-colourings. For each colouring, you can count the cut e
force algorithm takes $O(m \cdot 2^n)$ time. For $n = 50$, that's roughly $10^{15}$ opera

The problem is the search space. Every node is a binary decision: red or blue, 0 or 1. The
every node that shares an edge with $i$. These interactions create a rugged *cost landsca
valleys that look good from nearby but aren't the deepest.

```{figure} ../figures/cost-landscape.png
:name: fig-cost-landscape
:alt: A rugged cost landscape with many local optima and one global optimum.

Local search can improve a route assignment one move at a time and still never cross the b
```

Classical heuristics navigate this landscape by making local moves. Flip one node's colour. If the cut count improves, keep it; otherwise, try another flip. Simulated annealing adds randomness; occasionally accept a worse solution to escape local optima, gradually reducing the randomness as you "cool down."

These methods are powerful, but they have a fundamental weakness: they explore the landscape sequentially. At any moment, they're sitting at a single point in the space of 2^n possible colourings. They can only see the immediate neighbourhood. A better solution might exist on the other side of a high barrier, and no amount of local flipping will find it.

This is where the story turns quantum.
```

The Quantum Angle

What if, instead of sitting at one colouring and looking around, you could be at all 2^n colourings simultaneously, and then use interference to amplify the ones that cut more edges?

That's the core idea behind the **Quantum Approximate Optimization Algorithm** (QAOA), introduced by Farhi, Goldstone, and Gutmann in 2014. It's not a magic bullet; it won't solve NP-hard problems in polynomial time (nobody believes that's possible). But it offers a fundamentally different way to explore the solution landscape, one that exploits quantum mechanics in a structural way that classical algorithms cannot mimic.

Before we dive into QAOA itself, let's step back and sketch how quantum algorithms work in general. Nearly every quantum algorithm follows the same three-act structure:

1. **Encode** the problem into a quantum system. Binary decisions become qubits. Costs become operators. The search space becomes the space of quantum states.
2. **Manipulate** the quantum state using carefully chosen operations. This is where the quantum magic happens: interference suppresses bad solutions and amplifies good ones, without ever looking at them one by one.
3. **Measure** the result. Measurement collapses the quantum state to a single classical answer. If the manipulation was done well, that answer is likely to be a good one.

That's the whole recipe. The art is in step 2, and it's different for every algorithm. For QAOA, the manipulation is a rhythmic alternation between "imprint the cost" and "mix the solutions." For Shor's algorithm (Unit 2), it's a Fourier transform. For VQE (Unit 3), it's a chemistry-inspired trial state. But the three-act structure is always the same.

Let's see how it plays out for MaxCut.

Qubits as binary decisions

The first step is encoding. Each node in the graph gets one qubit. A classical bit is either 0 or 1. A qubit is the quantum version: it has two definite states, which physicists write as $|0\rangle$ and $|1\rangle$. The vertical bar and angle bracket are just packaging — the notation is called a "ket," and it's how physicists label quantum states. You can read $|0\rangle$ as "the state labelled zero" and $|1\rangle$ as "the state labelled one." When you measure a qubit, you always get one of these two outcomes: 0 or 1, just like a classical bit. The difference is what happens before you measure — but we'll get to that in a moment.

For MaxCut, $|0\rangle$ means "red" and $|1\rangle$ means "blue." An n-node graph requires n qubits. A specific colouring — say, nodes 1 and 3 are blue, node 2 is red — corresponds to the quantum state $|101\rangle$ (just the individual qubit labels strung together).

So far, this is just notation. The quantum part starts when we use superposition.

> **Superposition.** A qubit doesn't have to be $|0\rangle$ or $|1\rangle$. It can be in a state like $\alpha|0\rangle + \beta|1\rangle$, where α and β are numbers (complex, if you care, but the intuition works with real numbers) satisfying $|\alpha|^2 + |\beta|^2 = 1$. The values $|\alpha|^2$ and $|\beta|^2$ are the probabilities of getting 0 or 1 when you measure. Before measurement, both possibilities coexist. This isn't a metaphor; it's how the physics works. When you put n qubits into superposition, the system represents all 2^n possible states at once. You'll hear people say "the quantum computer tries all answers simultaneously." That's misleading. What really happens is subtler: the 2^n possibilities can interfere with each other, and clever algorithms arrange for the good answers to reinforce and the bad ones to cancel. Deep-Dive 1 makes this precise.

With n qubits in superposition, we have all 2^n colourings present in a single quantum state. We can't look at them all; measurement gives us just one. But between preparation and measurement, we can manipulate the amplitudes to make good colourings more likely to appear. That's the game.

Turning costs into energy

Now we need to tell the quantum computer what "good" means. We need to translate the MaxCut objective; "cut as many edges as possible"; into something a quantum system can work with.

Physicists have a word for "the function that assigns a number to every state of a system": they call it a **Hamiltonian**. In physics, the Hamiltonian describes the total energy, and the system naturally evolves toward its lowest-energy state. We're going to borrow this idea: assign an "energy" to every colouring, where lower energy means more edges cut. Then "find the best colouring" becomes "find the lowest-energy state."

This reframing is not just convenient; it's the key insight that makes quantum optimisation work. Quantum mechanics provides natural machinery for exploring energy landscapes — the Hamiltonian governs how a quantum system evolves, and the tools of quantum computing let us steer that evolution toward low-energy states. By encoding our problem as a Hamiltonian, we're harnessing this machinery for optimisation.

Here's how we build the Hamiltonian for MaxCut. For each edge (i, j), we need an expression that equals 1 when the edge is cut and 0 when it isn't. The trick uses a quantum operator called Z (the Pauli-Z operator, which gives $+1$ for $|0\rangle$ and -1 for $|1\rangle$):

$$\frac{1 - Z_i Z_j}{2}$$

Does it work? Check all four cases:

q_i	q_j	Same colour?	Z_i	Z_j	Z_iZ_j	$\frac{1}{2}(1 - Z_iZ_j)$
$\|0\rangle$ (red)	$\|0\rangle$ (red)	yes	$+1$	$+1$	$+1$	0 — uncut
$\|0\rangle$ (red)	$\|1\rangle$ (blue)	no	$+1$	-1	-1	1 — **cut**
$\|1\rangle$ (blue)	$\|0\rangle$ (red)	no	-1	$+1$	-1	1 — **cut**
$\|1\rangle$ (blue)	$\|1\rangle$ (blue)	yes	-1	-1	$+1$	0 — uncut

The formula returns 1 exactly when the edge is cut and 0 when it isn't. That's all we need.

The total cost Hamiltonian sums this over every edge:

$$C = \sum_{(i,j)\in E} \frac{1 - Z_iZ_j}{2}$$

The state with the highest value of C is the colouring that cuts the most edges. Equivalently, the **ground state** of $-C$ (the lowest-energy state) is the optimal solution.

This pattern; "encode the objective as a Hamiltonian, then find its ground state"; is the central trick of quantum optimisation, and it recurs throughout this book. In Unit 3, the Hamiltonian encodes molecular energy. In Unit 6, it encodes scheduling constraints. In Unit 7, it encodes the physics of a material. The problems are different, but the strategy is the same: translate "find the best X" into "find the lowest energy."

Reading a quantum circuit

Before we look at the QAOA circuit, a quick orientation on how quantum programs are drawn.

A quantum circuit is read left to right, like sheet music. Each horizontal line (called a **wire**) represents one qubit. Boxes or symbols on the wires represent **operations** (also called gates). A gate on one wire acts on that qubit alone; a gate connecting two wires (drawn with a vertical line between them) acts on both qubits together. At the end, a measurement symbol (M or a meter icon) reads out the qubit's value as a classical 0 or 1.

Here's what that looks like in practice. The circuit below implements **quantum teleportation** — moving the state of one qubit to another using entanglement and two classical bits. You don't need to understand it yet; just notice the structure:

"'{figure} ../figures/teleportation-circuit.png :name: fig-teleportation-circuit :alt: Quantum teleportation circuit with three qubit wires, entangling gates, measurements, and classically controlled corrections.

This is a reading guide for every circuit in the book: wires are qubits, gates act left to right, and measurements can feed classical information forward to later operations.

```
Three wires, three qubits. Gates ($H$, CNOT) sit on the wires at specific moments in time
just like a musical score tells musicians what to play and when. This is the visual langua

> **Unitary.** You'll see quantum operations called "unitary." This is a technical term t
Dive 1 goes deeper into the mathematics.

A few gates we'll meet repeatedly:

- $H$ (Hadamard): puts a qubit into superposition. Turns $|0\rangle$ into an equal mix of
- $Z$ (Pauli-Z): you've already met this; it gives $+1$ for $|0\rangle$ and $-
1$ for $|1\rangle$. Related gates $R_Z(\theta)$ rotate by a tunable angle.
- $X$ (Pauli-X): flips a qubit; $|0\rangle \to |1\rangle$ and vice versa. Related gate $R
- CNOT: a two-qubit gate that flips the second qubit if the first is $|1\rangle$. This is

With that vocabulary, let's look at the QAOA circuit.

### The QAOA circuit

Here is the entire QAOA algorithm for our three-node MaxCut problem:

```{figure} ../figures/qaoa-box-circuit.png
:name: fig-qaoa-box-circuit
:alt: QAOA circuit with Hadamard initialisation, repeated Cost and Mix rounds, and final

QAOA repeats one simple rhythm: imprint the cost as phases, mix neighbouring bit strings
good solution.
```

Read left to right. Initialise, then alternate between two boxes — Cost and Mix — for p rounds, each round with its own pair of angles (γ_k, β_k). Then measure. That's the entire algorithm. The angles are different in every round: round 1 uses γ_1, β_1; round 2 uses γ_2, β_2; and so on. There are $2p$ parameters in total, and choosing them well is the whole game. We'll get to that. First, let's open each box.

Box 1: Initialisation. A Hadamard gate H on every qubit. This creates an equal superposition of all 2^n colourings, each with the same amplitude $1/\sqrt{2^n}$. Think of it as a blank canvas: every possible solution is present, and none is preferred. If you measured right now, you'd get a uniformly random colouring. Not useful yet.

Box 2: The cost phase $(e^{-i\gamma_k C})$. This is where our Hamiltonian earns its keep. For each edge (i, j), the circuit applies a rotation whose angle is proportional to γ_k. The effect: every colouring in the superposition picks up a phase that depends on its cut count. Colourings that cut more edges get one phase; colourings that cut fewer get a different phase.

But here's the catch: phase changes alone are invisible to measurement. The probabilities haven't changed at all. If you measured right now, you'd still get a uniformly random colouring. The cost phase has written information in invisible ink: it tagged good solutions differently from bad ones, but the tags are hidden in the phases, and measurement only sees probabilities.

So what good are the phases? This is where interference enters.
```

> **Interference.** Quantum amplitudes aren't just numbers — they have both a magnitude and a direction (technically, a complex phase). Think of them as arrows. When two arrows point the same way, they reinforce: their magnitudes add. When they point in opposite directions, they cancel: their magnitudes subtract. This is interference, and it's the mechanism that makes quantum computing fundamentally different from classical computing.
>
> Superposition gives you the blank canvas — all 2^n solutions present at once. But superposition alone is useless: you can't look at all the solutions, and a random one is no better than a coin flip. Interference is what sculpts the canvas. It converts the phase information (which solution is good, which is bad) into probability information (which solution you're likely to measure). Every quantum algorithm is, at its core, an interference machine — and this is the pattern for every chapter in this book: encode the problem, let interference amplify the right answer, measure. The details differ — Shor's algorithm uses the Quantum Fourier Transform to create constructive interference at the period. Grover's uses the diffusion operator. QAOA uses the mixer. But the trick is always the same: arrange for good answers to reinforce and bad answers to cancel.

Box 3: The mixer ($e^{-i\beta_k B}$). An $R_X(2\beta_k)$ rotation on every qubit. This "stirs" the superposition, causing amplitudes from different colourings to overlap and interfere. Because the cost phase tagged good colourings with different phases from bad ones, the mixer's stirring produces constructive interference for high-cut colourings and destructive interference for low-cut ones. After the mixer, the probabilities are no longer uniform: good solutions are more likely to appear when you measure.

Then repeat. One round of cost-then-mix shifts the probabilities a little. Two rounds shift them more. Each round uses fresh angles (γ_k, β_k), so the algorithm can tag and stir with different intensities at each step — like a photographer adjusting focus and exposure between shots. In the limit $p \to \infty$ with optimally chosen parameters, QAOA can in principle reach the exact optimum. In practice, you use a small p and accept an approximate answer — hence "Approximate" in the name.

But who chooses the $2p$ angles? You could try random values, but that's wasteful. Instead, QAOA uses a **variational loop** — a quantum-classical hybrid:

1. Pick initial values of $\gamma_1, \beta_1, \ldots, \gamma_p, \beta_p$
2. Run the quantum circuit, measure the output
3. Compute the expected cost (run many times, take the average)
4. Feed this cost to a classical optimiser (COBYLA, Nelder-Mead, gradient descent)
5. The optimiser suggests better angles
6. Repeat until convergence

The quantum computer explores the solution space. The classical computer tunes the exploration strategy. Neither could do the other's job efficiently.

For the gate-level decomposition of each box — how the cost phase becomes a sequence of CNOTs and R_Z rotations, and what R_X does geometrically — see Deep-Dive 1.

Worked Example

Let's solve MaxCut on a triangle; three nodes, three edges. The optimal cut is 2 (colour one node differently from the other two).

The graph

"'{figure} ../figures/maxcut-triangle-graph.png :name: fig-maxcut-triangle-graph :alt: Triangle graph with nodes 0, 1, 2 and edges between every pair.

The triangle is small enough to solve exactly yet still large enough to show a genuine MaxCut optimum of 2.

```
Three nodes, three edges: (0,1), (0,2), (1,2). Each node gets one qubit.

### The cost Hamiltonian

$$C = \frac{1 - Z_0 Z_1}{2} + \frac{1 - Z_0 Z_2}{2} + \frac{1 - Z_1 Z_2}{2}$$

### What QAOA produces

Using pre-optimised parameters ($\gamma \approx \pi/4$, $\beta \approx \pi/8$), a single

| Outcome | Cut value | Probability |
|---------|-----------|-------------|
| 000 | 0 | ~3% |
| 001 | 2 | ~16% |
| 010 | 2 | ~16% |
| 011 | 2 | ~16% |
| 100 | 2 | ~16% |
| 101 | 2 | ~16% |
| 110 | 2 | ~16% |
| 111 | 0 | ~3% |

The colourings with cut value 2 (the optimum for the triangle) are amplified. The "all sa

→ *The next chapter builds this circuit gate by gate, runs it, and shows you the code.*

### Back to the trucks

We started with UPS drivers and 20-stop routes. We solved MaxCut on a triangle. How do you

The same machinery applies. TSP and its industrial cousin, the **Vehicle Routing Problem
which city does driver $k$ visit at time step $t$? The cost Hamiltonian becomes a sum of c
you need $O(n^2)$ qubits instead of $n$, and the constraint penalties add terms to the Ha
but the algorithm is identical: initialise in superposition, alternate cost and mixer ph
```

That's also why we'll see this same pattern in every unit of this book. Drug molecules, fi
they all become Hamiltonians, and the quantum algorithm searches for the ground state. Th

Reality Check

Let's be honest about where QAOA stands; and where it's heading.

At low depth, QAOA underperforms classical methods. For depth-1 QAOA on MaxCut, the guaranteed approximation ratio is 0.6924 for 3-regular graphs. The best classical polynomial-time algorithm (Goemans-Williamson) achieves 0.878. If you can only run one or two rounds, you're better off clas

At high depth, QAOA is competitive. Performance improves monotonically with depth $p
satisfaction problems. For these specific problem families, QAOA at sufficient depth is

The hardware isn't there yet. Today's quantum computers can run QAOA at $p = 1$ or $p
tolerant quantum computers with $\sim 10^4$ logical qubits.

Barren plateaus — the training problem. A **barren plateau** is a region of paramete
the gradient vanishes, leaving the optimiser with no direction to improve. For generic ra

What's real today: The algorithm's performance is well-characterised. The bottlenec

Chef's Notes

- **MaxCut is a gateway drug.** We used it here because it maps directly to qubits ($Z_i Z
XORSAT are the problems where we can compute QAOA's exact performance; for other problems

- **The Max-k-XORSAT family.** MaxCut is the $k = 2$ case of a broader family: Max-k-XORSAT, where each constraint involves k variables connected by an XOR. As k incr
body Z-interactions instead of 2-body. The same tree tensor network machinery evaluate
Hadamard factorisation) that makes $k \geq 3$ tractable.

- **The cost Hamiltonian pattern recurs everywhere.** The idea of encoding an objective f

- **QAOA vs. quantum annealing.** QAOA explores the cost landscape via alternating unita
based). Quantum annealing explores it via adiabatic evolution (slowly changing the Hamil

- **The variational quantum-classical loop appears again in VQE** (Unit 3). If you unders
motivated vs. graph-motivated).

- **Further reading:**
  - Farhi, Goldstone, Gutmann (2014). *A Quantum Approximate Optimization Algorithm.* [
  - Basso, Farhi, Marwaha, Villalonga, Zhou (2021). *The Quantum Approximate Optimizati
Girth Regular Graphs and the Sherrington-Kirkpatrick Model.* [arXiv:2110.14206](https:/
  - Farhi, Gutmann, Ranard, Villalonga (2025). *Lower bounding the MaxCut of high girth
regular graphs using the QAOA.* [arXiv:2503.12789](https://arxiv.org/abs/2503.12789)

```
   - Jordan, Shutty, Wootters, Zalcman, et al. (2025). *Optimization by decoded quantum i
025-09527-5) ([arXiv:2408.08292](https://arxiv.org/abs/2408.08292))
   - Azariah and Jordan (2026). *Exact QAOA on Max-$k$-XORSAT via generic tree folding.*

# Deep-Dive 1: Building QAOA from Gates

_This deep dive teaches the QAOA algorithm at the circuit level —
how every piece becomes concrete operations on qubits. It pairs with Unit 1 (Logistics),

## In This Chapter

- **What you'll learn:** How to encode a MaxCut problem as a quantum circuit, what each ga
- **What you need:** No prior quantum knowledge. This is the first deep dive; we introduc
- **Runnable version:** The companion notebook [`01-logistics.ipynb`](../notebooks/01-
logistics.ipynb) runs the same circuit on a cloud Quokka.




### Where we are and what we need

We have a cost function; the MaxCut objective; and we've learned that QAOA encodes it as a
i\gamma C}$ and the mixer $e^{-i\beta B}$), and measures. What we haven't explained is ho

Let's fix that. We'll build the entire circuit for MaxCut on a triangle (3 nodes, 3 edges

### Binary decisions become qubits

The first step is representation. In MaxCut, each node is coloured red or blue; a binary c

| Node | Qubit | $\lvert 0\rangle$ meaning | $\lvert 1\rangle$ meaning |
|:---|:---|:---|:---|
| 0 | `q[0]` | Red | Blue |
| 1 | `q[1]` | Red | Blue |
| 2 | `q[2]` | Red | Blue |

A specific colouring; say, node 0 is blue, nodes 1 and 2 are red; corresponds to the quant
qubit computational basis states: $\lvert 000\rangle, \lvert 001\rangle, \ldots, \lvert

So far, nothing quantum. A classical computer could do the same: 3 bits, 8 combinations.

### Creating the starting state

QAOA begins by creating an *equal superposition* of all possible colourings. We want:

$$\lvert s \rangle = \frac{1}{\sqrt{8}}(\lvert 000\rangle + \lvert 001\rangle + \lvert 0

Every colouring has the same amplitude ($1/\sqrt{8}$), so every colouring has the same pr

How do you create this state? With the **Hadamard gate** ($H$), applied to each qubit ind
```

```
$$H\lvert 0\rangle = \frac{1}{\sqrt{2}}(\lvert 0\rangle + \lvert 1\rangle)$$

Apply $H$ to qubit 0, then qubit 1, then qubit 2. Because the qubits are independent at th

$$H\lvert 0\rangle \otimes H\lvert 0\rangle \otimes H\lvert 0\rangle = \frac{1}{\sqrt{8

In circuit form:

```{figure} ../figures/qaoa-hadamard-init.png
:name: fig-qaoa-hadamard-init
:alt: Hadamard gates applied to all three qubits.

A layer of Hadamards puts all eight triangle colourings into superposition before the op
```

Three gates. That's all it takes to create a uniform superposition of 8 colourings. For n qubits, it's n Hadamard gates and 2^n terms in superposition. The exponential scaling is free.

> **Common Mistake #1:** Students sometimes say "the quantum computer is trying all 8 colourings at once." This is misleading. All 8 colourings are present in the superposition, but you can only read out one of them (the one you measure). The power of QAOA is not parallel evaluation; it's interference: manipulating the amplitudes so that good colourings become more likely to be measured.

The problem unitary: turning edge costs into phases

Now we need to "imprint" the cost function onto the quantum state. We want colourings that cut many edges to acquire different phases from colourings that cut few edges. Later, the mixer will cause these phases to interfere, amplifying the good solutions.

The cost Hamiltonian for our triangle is:

$$C = \frac{1 - Z_0 Z_1}{2} + \frac{1 - Z_0 Z_2}{2} + \frac{1 - Z_1 Z_2}{2}$$

Each term contributes 1 when the edge is cut (qubits differ) and 0 when it's not (qubits agree). The total is the number of cut edges.

The problem unitary is $e^{-i\gamma C}$. What does this do? For each computational basis state $|x\rangle$:

$$e^{-i\gamma C}|x\rangle = e^{-i\gamma \cdot \mathrm{cut}(x)}|x\rangle$$

It multiplies each colouring by a phase that depends on how many edges it cuts. The colouring itself doesn't change; only its phase. Colourings with cut value 2 (the optimum for the triangle) get phase $e^{-2i\gamma}$. Colourings with cut value 0 get phase $e^0 = 1$. Different phases, same probabilities (for now).
```

But how do we implement $e^{-i\gamma C}$ as gates? Since the three edge terms in C all commute with each other (they're all diagonal in the computational basis), we can apply them separately:

$$e^{-i\gamma C} = e^{-i\gamma(1-Z_0Z_1)/2} \cdot e^{-i\gamma(1-Z_0Z_2)/2} \cdot e^{-i\gamma(1-Z_1Z_2)/2}$$

Each factor handles one edge. Let's focus on the first edge, $(0, 1)$.

The ZZ gate: one edge at a time

We need to implement $e^{-i\gamma(1-Z_0Z_1)/2}$. The constant part $e^{-i\gamma/2}$ is just a global phase (it multiplies every state equally), so we can ignore it; global phases are unobservable. What remains is $e^{i\gamma Z_0Z_1/2}$.

Now, Z_0Z_1 is diagonal in the computational basis. Its eigenvalues are:

q_0	q_1	Z_0	Z_1	Z_0Z_1	Meaning
0	0	+1	+1	+1	Same colour; edge not cut
0	1	+1	−1	−1	Different colours; edge cut
1	0	−1	+1	−1	Different colours; edge cut
1	1	−1	−1	+1	Same colour; edge not cut

So $e^{i\gamma Z_0Z_1/2}$ applies phase $e^{i\gamma/2}$ when the qubits agree and $e^{-i\gamma/2}$ when they disagree.

How do we build this from standard gates? The trick is the **CNOT sandwich**:

"'{figure} ../figures/zz-gate-cnot-sandwich.png :name: fig-zz-gate-cnot-sandwich :alt: ZZ gate implemented as a CNOT sandwich with an Rz rotation between the two CNOTs.

The CNOT sandwich computes edge parity, rotates on that parity, then uncomputes it, which is why one graph edge becomes one ZZ cost term.

```
Here's what happens step by step:

1. **First CNOT** (`cx q[0], q[1]`): XORs qubit 1 with qubit 0. Now qubit 1 holds the *par

2. **$R_Z(\gamma)$ on qubit 1** (`rz(γ) q[1]`): applies phase $e^{-
i\gamma/2}$ if qubit 1 is $\lvert 1\rangle$ (qubits were different = edge cut), and $e^{i

3. **Second CNOT** (`cx q[0], q[1]`): undoes the XOR, restoring qubit 1 to its original v

Net effect: the computational basis states are unchanged, but each one picks up a phase t

For edge $(0, 1)$, this is the same CNOT sandwich shown above. Repeat for edges $(0, 2)$ a

> **Why does this work?** The CNOT "extracts" the parity into a single qubit, the $R_Z$ "a

### The mixer: where phases become probabilities
```

After the problem unitary, the quantum state has different phases for different colourin
the phases are invisible to measurement. So what was the point?

The point is that phases are *potential energy for interference*. The mixer is the step t

Let's see this concretely. Before the mixer, our state looks like:

$$\frac{1}{\sqrt{8}}\bigl(e^{i\phi_0}\lvert 000\rangle + e^{i\phi_1}\lvert 001\rangle

where each ϕ_x depends on the cut value of colouring x. Every amplitude has the sa

Now the mixer acts. It applies $R_X(2\beta)$ to each qubit, which mixes $\lvert 0\rangle
those that differ by a single bit flip. Each contribution arrives carrying the phase it p

Here's the crucial point: when contributions arrive with *similar* phases, their arrows

Let's see this on our triangle. Take the good colouring $\lvert 001\rangle$ (cut = 2). It
the colourings one bit-flip away — are:

```{figure} ../figures/neighbours-good.png
:name: fig-neighbours-good
:alt: A good colouring with cut value 2 and its three one-bit-flip neighbours.

Good colourings are surrounded by neighbours with similar cut values, so the mixer tends

Two out of three neighbours also have cut value 2 — the same as the original. They picked up similar phases from the cost step, so when their contributions arrive during mixing, the arrows point roughly the same way. Constructive interference: $|001\rangle$'s amplitude grows.

Now take the poor colouring $|000\rangle$ (cut = 0). Its neighbours:

"'{figure} ../figures/neighbours-poor.png :name: fig-neighbours-poor :alt: A poor colouring with cut value 0 and its three one-bit-flip neighbours.

Poor colourings receive amplitude from neighbours with very different phases, so interference suppresses them.

Every neighbour has cut value 2 — all *different* from the original's cut value of 0. They

The result: after the mixer, the probabilities are no longer uniform. Good colourings hav

This is the heart of QAOA. The cost phase *writes* information in the phases. The mixer *
phases without mixing are invisible, and mixing without phases is just random stirring.

The mixer Hamiltonian is $B = X_0 + X_1 + X_2$, and the mixer unitary is $e^{-i\beta B}$. Since X operators on different qubits commute, this factorises:

$$e^{-i\beta B} = e^{-i\beta X_0} \cdot e^{-i\beta X_1} \cdot e^{-i\beta X_2}$$

Each factor is just an $R_X$ rotation: $e^{-i\beta X} = R_X(2\beta)$:
```

```{figure} ../figures/qaoa-mixer.png
:name: fig-qaoa-mixer
:alt: Mixer layer with Rx rotations applied to each qubit.

The mixer is just local $R_X$ rotations, but applied after the cost phase it turns hidden
```

What does $R_X(\theta)$ do? It rotates the qubit's state around the X axis of the **Bloch sphere** — a geometric picture where a single qubit's state is a point on a sphere, with $|0\rangle$ at the north pole, $|1\rangle$ at the south pole, and superpositions on the surface between. In the computational basis, R_X mixes $|0\rangle$ and $|1\rangle$:

$$R_X(\theta)|0\rangle = \cos(\theta/2)|0\rangle - i\sin(\theta/2)|1\rangle$$

At $\beta = 0$, the mixer does nothing ($R_X(0) = I$). At $\beta = \pi/2$, it flips every qubit ($R_X(\pi) = -iX$). In between, it partially mixes — allowing amplitude to flow between colourings that differ by one bit flip. This is the quantum analogue of the "flip one node" move in classical local search, but done coherently so that the phases from the cost step drive the interference we described above.

The full circuit, assembled

Putting it all together for the triangle, at depth $p = 1$:

"'{figure} ../figures/qaoa-full-circuit.png :name: fig-qaoa-full-circuit :alt: Complete QAOA circuit for MaxCut on a triangle, with initial Hadamards, three ZZ cost unitaries, a mixer layer, and measurement.

At depth $p = 1$, the triangle example is already the full algorithm: superposition, one ZZ block per edge, a mixer layer, then measurement.

```
Gate count: 3 Hadamards + 6 CNOTs + 3 $R_Z$ + 3 $R_X$ + 3 measurements = **18 gates**, of w

### Running it

With pre-optimised parameters $\gamma \approx \pi/4$, $\beta \approx \pi/8$, the circui

The companion notebook runs this circuit end-to-end — constructing the Hamiltonian, buil
force and random sampling.

→ **See [notebook `01-logistics.ipynb`](../notebooks/01-logistics.ipynb) for the runna

### Choosing $\gamma$ and $\beta$: the classical outer loop

The quantum circuit takes two numbers: $\gamma$ and $\beta$; and produces measurement sa

This is a classical optimisation problem. You run the circuit many times (say, 1,000 shot

The optimisation landscape; average cut value as a function of $(\gamma, \beta)$; is smoo

> **Common Mistake #2:** "QAOA finds the optimal solution by exploration." Not quite. A s

### Going deeper: depth $p > 1$
```

Everything above is depth-1 QAOA. At depth p, you repeat the problem unitary and mixer

$$\lvert \vec{\gamma}, \vec{\beta} \rangle = \prod_{k=1}^{p} e^{-i\beta_k B} e^{-i\gamma_k C} \lvert + \rangle^n$$

More depth means more expressive circuits; the algorithm can create more complex interfe

In the limit $p \to \infty$, QAOA can produce *any* quantum state and find the exact optir

What you should take away

1. **Every classical binary decision becomes a qubit.** The encoding is direct: 0/1 → $\l
2. **The cost function becomes a phase operator.** The CNOT sandwich trick turns pairwise
3. **The mixer creates interference.** R_X rotations allow amplitude to flow between c
4. **The classical optimiser tunes the circuit.** γ controls how strongly the cos
5. **The circuit is small.** A depth-1 QAOA for n nodes and m edges uses $2m$ CNOTs. T
to-moderate graphs.

Unit 2: Cryptography — *The Trapdoor Quantum Computers Can Kick Open*

The Hook

Every time you type a credit card number into a website, a quiet miracle happens. Your bro

This is **public-key cryptography**, and it protects virtually every financial transacti

Multiplying two large prime numbers is easy. Factoring their product is hard.

Take two 1,000-digit primes p and q. Multiplying them takes microseconds. The produc
digit number. Now hand someone N and ask them to find p and q. The best classical al

This asymmetry; easy to combine, hard to separate; is a **trapdoor**. RSA encryption, inv

For nearly 50 years, this trapdoor has held. Classical computers have gotten faster, and
exponential*; they scale as roughly $e^{n^{1/3}}$ where n is the number of digits. For
digit numbers, that's far beyond any conceivable classical computer.

In 1994, Peter Shor shattered this assumption.

Shor showed that a quantum computer can factor N in time *polynomial* in the number of d
exponential. Not "somewhat faster." Polynomial. For cryptographically relevant sizes, t
time attack once a large fault-tolerant quantum computer exists.

Shor's algorithm doesn't just threaten RSA. His 1994 paper actually contains *two* quant
given $g^x \bmod p = y$, find x — which breaks Diffie-Hellman (the original public-
key protocol) and **elliptic curve cryptography** (ECC), a modern alternative to RSA bas
256 curve needs about **2,330 logical qubits**. That's fewer than the roughly **6,000 log
2048, though the closer classical-security comparison is RSA-3072, not RSA-

2048. (We'll define logical vs. physical qubits in the Reality Check.) Both algorithms us
finding.

Let's understand why; and how.

The Bottleneck

Why is factoring hard? It's not enough to say "the search space is big." The search space

The trapdoor in detail

RSA works like this:

1. Alice picks two large primes p and q and computes $N = pq$
2. She publishes N (her public key) and keeps p, q secret (her private key)
3. Bob encrypts a message using N; a fast operation
4. Only Alice, who knows p and q, can decrypt it; because decryption requires a key de
1)(q-1)$ — the count of integers less than N that share no factor with N

The security assumption: given N, nobody can find p and q efficiently. If they coul
1)(q-1)$, derive the decryption key, and read everything.

Why classical algorithms struggle

The naive approach; try dividing N by every number up to $\sqrt{N}$; takes $O(\sqrt{N}
digit N, that's 10^{1000} operations. Hopeless.

The best classical algorithm, the General Number Field Sieve (GNFS), is far cleverer. It

$$\exp\!\left(1.923 \, (\ln N)^{1/3} \, (\ln \ln N)^{2/3}\right)$$

where n is the number of bits in N (equivalently, $n = \log_2 N$). This is *sub-
exponential* — faster than brute force, but still far slower than polynomial. For RSA-
2048 (a 617-digit number), the estimated classical effort is around 2^{112} operations

The fundamental issue: factoring has no known *polynomial-time* classical algorithm. The
complete — meaning it hasn't been proven to be among the "hardest of the hard" problems),

Factoring reduces to period-finding

Here's the insight that opens the door to quantum computation. Factoring N reduces to f

Pick a random number $a < N$. Define the function:

$$f(x) = a^x \bmod N$$

The notation $\bmod N$ means "take the remainder after dividing by N." So $7^2 \bmod 15

This function is periodic. There exists some integer r (called the **order** of a moc

If you can find r, you can (with high probability) factor N. The algebraic trick:

$$a^r - 1 \equiv 0 \pmod{N}$$
$$(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N}$$

If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then the **greatest common divisor**
1, N)$ — the largest number that divides both arguments — gives a non-
trivial factor of N.

So: **if you can find periods efficiently, you can factor efficiently.** Classically, fi

The Quantum Angle

Shor's algorithm finds the period r of $f(x) = a^x \bmod N$ using three quantum ingredi

1. **Superposition**; query f on all inputs simultaneously
2. **The Quantum Fourier Transform**; extract the period from the phase pattern
3. **Interference**; amplify the right answer, suppress the wrong ones

Superposition: querying all inputs at once

In Unit 1, we saw that qubits can be in superposition. Here we use it for something specif

Start with two quantum **registers** — groups of qubits that together represent a single

Apply Hadamard gates to every qubit in the input register. The notation $H^{\otimes n}$ i

$$|0\rangle^n \xrightarrow{H^{\otimes n}} \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n -
1} |x\rangle$$

This creates an *equal superposition* of all 2^n possible inputs. Now apply the unitary

$$\frac{1}{\sqrt{2^n}} \sum_{x} |x\rangle|0\rangle \xrightarrow{U_f} \frac{1}{\sqrt{2

With one application of U_f, we've "evaluated" f on all 2^n inputs. The catch: if w

The Quantum Fourier Transform: extracting periodicity

The function $f(x) = a^x \bmod N$ is periodic with period r. This means the amplitudes

We need to extract r without knowing which y we'll observe. The tool for this is the *

The classical DFT converts a signal from the time domain to the frequency domain. A perio

Applied to the input register, the QFT maps:

$$|x\rangle \xrightarrow{\text{QFT}} \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-
1} e^{2\pi i xk / 2^n} |k\rangle$$

```
The key property: if the input amplitudes are periodic with period $r$, then after the QF
processing (the continued fractions algorithm) recover $r$ exactly.

### Phase kickback: why the QFT "sees" the period

Why does the QFT work here? The deep reason is **phase kickback**; the mechanism by which

When $f$ is computed in superposition, the output register becomes entangled with the inp

$$\frac{1}{\sqrt{M}} \sum_{j=0}^{M-1} |x_0 + jr\rangle$$

where $x_0$ is the smallest such $x$ and $M \approx 2^n/r$ is the number of terms (one for

We don't actually need to measure the output register. The entanglement does the work for
finding work.

### Interference: why the wrong answers cancel

The QFT isn't magic; it's interference. Each output state $|k\rangle$ receives contribut
spaced inputs align; they add constructively. When $k$ is not a multiple of $2^n / r$, the

This is the same physics that creates light and dark bands when light passes through narr
slit experiment with $2^n$ slits, and the period $r$ determines which bands are bright.

```{figure} ../figures/fourier-peaks.png
:name: fig-fourier-peaks
:alt: A periodic support pattern in the input register and the corresponding peaks after

Period-finding works because a periodic pattern in the input register becomes sharp freq
```

Putting it together

Here is Shor's algorithm as a circuit, drawn at the same box level we used for QAOA in Unit 1:

"'{figure} ../figures/shor-box-circuit.png :name: fig-shor-box-circuit :alt: Shor's algorithm circuit with Hadamards on the input register, an oracle spanning both registers, an inverse QFT, and measurement on the input register.

Shor's circuit follows the same three-act pattern as QAOA: prepare superposition, compute the structured oracle, then use a Fourier transform to make the period measurable.

```
Two registers. The top four wires are the input register; the bottom four are the output

Shor's algorithm:

1. Pick a random $a < N$, check $\gcd(a, N) = 1$
2. Prepare a superposition of all inputs: $\frac{1}{\sqrt{2^n}} \sum_x |x\rangle|0\rang
3. Compute $f(x) = a^x \bmod N$ in superposition
4. Apply the QFT to the input register
5. Measure the input register → get $k \approx j \cdot 2^n / r$
```
```

6. Use the **continued fractions algorithm** — a classical method that finds the simples
to extract r
7. Compute $\gcd(a^{r/2} \pm 1, N)$ → factors of N

Steps 1, 6, and 7 are classical. Steps 2–5 are quantum. With standard schoolbook arithmet

Worked Example

Let's factor $N = 15$ using Shor's algorithm.

Step 1: Pick a

Choose $a = 7$. Check: $\gcd(7, 15) = 1$. Good.

Step 2–3: Compute $7^x \bmod 15$ in superposition

The values cycle:

x	$7^x \bmod 15$
0	1
1	7
2	4
3	13
4	1
5	7
6	4
7	13

The period is $r = 4$.

Step 4–5: QFT and measurement

Using 4 qubits for the input register ($2^4 = 16$), the QFT concentrates amplitude on mul

Measured k	$k / 2^n = k / 16$	Continued fractions → r
0	0	No information
4	1/4	$r = 4$ $\checkmark$
8	1/2	$r = 2$ (try again)
12	3/4	$r = 4$ $\checkmark$

Two of the four outcomes directly return $r = 4$. The outcome $k = 8$ gives denominator 2
we'd need to try again or check). The outcome $k = 0$ gives no information.

Step 6–7: Extract factors

With $r = 4$:

$$7^{4/2} = 7^2 = 49$$

```
$$\gcd(49 - 1, 15) = \gcd(48, 15) = 3$$
$$\gcd(49 + 1, 15) = \gcd(50, 15) = 5$$

And indeed: $15 = 3 \times 5$. $\blacksquare$

This is a toy example; 4 qubits, a number you can factor in your head. But the algorithm s
2048 (a 617-digit, 2,048-bit number), leading abstract-circuit estimates use roughly 6,(
scale operations before fault-tolerance overhead. The same algorithm, larger registers,

→ *The next chapter builds the period-finding circuit from scratch, and the companion no
toy version honestly.*

### Back to your browser

We started with credit card numbers and the quiet miracle of public-
key cryptography. We factored $15 = 3 \times 5$ on a toy quantum computer. What connects t

Scale. The same algorithm that finds the period of $7^x \bmod 15$ can find the period of $
2048, $N$ is a 617-digit number, the abstract circuit needs about 6,000 logical qubits ar
scale operations, and a fault-tolerant machine must then implement those logical qubits

That's why the world's cryptographic infrastructure is being rebuilt. Not because a quan
none has — but because the algorithm is ready and waiting. When the hardware catches up, i


## Reality Check

Shor's algorithm is the most famous quantum algorithm, and its implications are enormous

**What's been factored quantumly.** The largest number factored by a genuine implementat
López et al., 2012, using photonic qubits). A separate experiment factored 15 using NMR (
specific circuits that don't generalise; they exploit known structure of the answer, whi

In short: Shor's algorithm has never been run, at scale, on a number whose factors were un

**Resource estimates for RSA-2048.** To make these estimates concrete, two terms matter:
corrected qubit built from many physical qubits — hundreds or thousands, depending on the
correction scheme. Today's machines have physical qubits; Shor's algorithm at scale need
tolerant** quantum computer is one that uses error correction to run arbitrarily long co

Gidney and Ekerå (2021) estimated that factoring a 2,048-bit RSA key would require approx
3}$) and roughly **8 hours** of computation.

That estimate has dropped dramatically under different coding assumptions. The **Pinnacl
2048 factoring with fewer than **100,000 physical qubits** under specific hardware assum
3}$, code cycle time $1\,\mu\text{s}$, reaction time $10\,\mu\text{s}$, and a one-
month runtime in its low-qubit example. The point is not that RSA-
2048 is about to fall tomorrow. The point is that architecture and error-
correction choices can change the resource picture by orders of magnitude.
```

On the elliptic-curve side, new circuit designs (2026) estimate fault-
tolerant attacks on prime-field ECC in minutes to hours using millions of physical qubits
time tradeoff. For ECC-224 — closer in classical security to RSA-
2048 than P-256 is — the paper quotes **34-96 minutes** using roughly **6.9-
19.1 million physical qubits**.

The post-quantum migration. The threat is taken seriously. NIST finalised its first
quantum cryptography standards in 2024: CRYSTALS-Kyber (key encapsulation) and CRYSTALS
Dilithium (digital signatures), both based on lattice problems believed to be hard for qu
quantum cryptography. Google, Cloudflare, and Apple have already deployed post-
quantum key exchange in their products.

The migration timeline is measured in decades, not years. Cryptographic infrastructure i
tolerant quantum computer becomes available. For secrets that must remain confidential f

What's real today: Shor's algorithm works. The mathematics is settled. The only ques

Chef's Notes

- **Superposition, interference, and phase kickback.** These three concepts; introduced

- **The QFT appears again in QPE** (Unit 7). Quantum Phase Estimation is essentially "Sho
finding subroutine applied to a unitary operator instead of modular exponentiation." If
tolerant quantum computing.

- **Shor's algorithm via amplitude estimation.** There's a structural connection to Unit
finding can be viewed as a special case of quantum phase estimation, which is itself rela

- **Why we don't start with Shor.** Most quantum computing textbooks open with Shor's alg

- **Further reading:**
  - Shor (1994). *Algorithms for Quantum Computation: Discrete Logarithms and Factoring
ph/9508027](https://arxiv.org/abs/quant-ph/9508027)
  - Gidney and Ekerå (2021). *How to factor 2048 bit RSA integers in 8 hours using 20 mil
  - Webster, Berent, Chandra, Hockings et al. (2026). *The Pinnacle Architecture: Reduc
2048 to 100,000 physical qubits using quantum LDPC codes.* [arXiv:2602.11457](https://a
  - Roetteler, Naehrig, Svore, Lauter (2017). *Quantum Resource Estimates for Computing
  - Kim, Jang, Wang, Baksi, Song et al. (2026). *New Quantum Circuits for ECDLP: Breakin
  - NIST (2024). *Post-Quantum Cryptography Standardization.* [csrc.nist.gov/projects/
quantum-cryptography](https://csrc.nist.gov/projects/post-quantum-
cryptography)
  - Vandersypen et al. (2001). *Experimental realization of Shor's quantum factoring al
887](https://doi.org/10.1038/414883a) ([arXiv:quant-ph/0112176](https://arxiv.org/abs
ph/0112176))
  - Nielsen and Chuang (2000). *Quantum Computation and Quantum Information.* Chapter 5

Deep-Dive 2: Inside Shor's Algorithm

_This deep dive pairs with Unit 2 (Cryptography), which explained why factoring breaks RS
finding solves factoring. Here we build the period-finding circuit from scratch._

```
## In This Chapter

- **What you'll learn:** How to extract the period of a function using a quantum circuit;
- **What you need:** From Deep-Dive 1 (Building QAOA), you know what qubits are, how Hada
- **Runnable version:** The companion notebook [`02-cryptography.ipynb`](../notebooks/
cryptography.ipynb) keeps the classical factoring reduction exact and runs a compiled pe
finding toy for $N = 15$ on a cloud Quokka.


## The problem we need to solve

Unit 2 showed that factoring $N$ reduces to finding the period $r$ of the function $f(x) =

1. Evaluates $f$ on all possible inputs simultaneously (superposition)
2. Extracts the period $r$ from the results (without knowing which specific outputs appea

In Deep-Dive 1, we built QAOA from gates. The cost Hamiltonian imprinted phases on the qu
different colourings got different phases — and then the mixer caused interference that a

Let's build it up, starting from things you already know.


## From CNOT to function evaluation

### A function you already know

In Deep-Dive 1, we used CNOT to compute the *parity* of two qubits as part of the ZZ gate.

Here's the full truth table for CNOT, acting on an input qubit and an output qubit:

| Input $|x\rangle$ | Output before | Output after | What happened |
|:---:|:---:|:---:|:---|
| $|0\rangle$ | $|0\rangle$ | $|0\rangle$ | $0 \oplus 0 = 0$ |
| $|0\rangle$ | $|1\rangle$ | $|1\rangle$ | $1 \oplus 0 = 1$ |
| $|1\rangle$ | $|0\rangle$ | $|1\rangle$ | $0 \oplus 1 = 1$ |
| $|1\rangle$ | $|1\rangle$ | $|0\rangle$ | $1 \oplus 1 = 0$ |

The input qubit is never changed. The output qubit gets XOR'd with the input: $|b\rangle
the same as flipping the output when the input is 1, which is exactly what CNOT does.

When the output starts at $|0\rangle$, the XOR simply copies: $|0 \oplus x\rangle = |x\ra

$$\text{CNOT}|x\rangle|0\rangle = |x\rangle|x\rangle = |x\rangle|f(x)\rangle$$

The input qubit is unchanged. The output qubit holds $f(x)$.

This is the pattern for *all* quantum function evaluation: you have an input register hol

$$U_f|x\rangle|0\rangle = |x\rangle|0 \oplus f(x)\rangle = |x\rangle|f(x)\rangle$$
```

Why XOR and not just "write $f(x)$"? Because quantum operations must be *reversible* —
you need to be able to undo them. Overwriting a register isn't reversible (the old value i
like the one that computes $a^x \bmod N$ — are built from networks of CNOTs and Toffoli ga
into-the-output pattern.

For the identity function, U_f is a single CNOT. For $f(x) = a^x \bmod N$, U_f is a mu

What happens in superposition

Now put the input in superposition. Apply Hadamard to the input qubit before the CNOT:

$$H|0\rangle \otimes |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|0\rangle$$

Then apply U_f (the CNOT):

$$\frac{1}{\sqrt{2}}(|0\rangle|f(0)\rangle + |1\rangle|f(1)\rangle) = \frac{1}{\sqrt{2

We've evaluated f on both inputs with one operation. But if we measure, we get one rando
both answers are in the superposition — but measurement destroys it.

This is the fundamental challenge: superposition lets you evaluate f on all inputs at o

Phase kickback: the trick behind everything

The problem with reading the output

When we evaluate f in superposition, the result sits in the output register as a comput
a "classical" value that we could read out. But reading it gives us one random answer. We

In QAOA, we faced a similar problem. The cost Hamiltonian didn't write "this colouring cu
invisible to measurement but available for interference. Can we do the same thing with fu

Yes. Here's how.

The $|{-}\rangle$ trick

First, two useful shorthands. Applying a Hadamard to $|0\rangle$ gives $|{+}\rangle = \f
equal superposition with a plus sign. Applying X then H to $|0\rangle$ gives $|{-
}\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ — equal superposition with a minus
}\rangle$ is one sign. That sign is about to do all the work.

Instead of initialising the output register to $|0\rangle$, initialise it to $|{-
}\rangle$. Now watch what happens when we apply U_f (for a single-
bit function f):

$$U_f|x\rangle|{-}\rangle = (-1)^{f(x)}|x\rangle|{-}\rangle$$

The output register doesn't change — it comes back as $|{-}\rangle$ every time. But the *
1)^{f(x)}$: a plus sign when $f(x) = 0$, a minus sign when $f(x) = 1$. The function value

Why does this happen?

Let's trace it. We established that U_f XORs $f(x)$ into the output: $U_f|x\rangle|b\r
}\rangle$ and apply U_f to each term:

$$U_f|x\rangle|{-}\rangle = U_f|x\rangle \cdot \frac{1}{\sqrt{2}}(|0\rangle -
|1\rangle)$$

$$= \frac{1}{\sqrt{2}}(|x\rangle|0 \oplus f(x)\rangle - |x\rangle|1 \oplus f(x)\rangle

Here $\oplus$ means XOR (addition modulo 2): $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus

If $f(x) = 0$: XOR with 0 changes nothing. We get $\frac{1}{\sqrt{2}}(|x\rangle|0\ra
|x\rangle|1\rangle) = |x\rangle|{-}\rangle$. Phase $= +1$.

If $f(x) = 1$: XOR with 1 flips both bits. We get $\frac{1}{\sqrt{2}}(|x\rangle|1\ra
|x\rangle|0\rangle) = -|x\rangle|{-}\rangle$. Phase $= -1$.

The output register returns to $|{-}\rangle$ in both cases — it's a catalyst, not a conta

A caveat for later: this clean "catalyst" picture works perfectly when f outputs a sing
bit outputs (like $a^x \bmod N$ in Shor's algorithm), the output register does *not* retu
it remains entangled with the input register. But the QFT on the input register still ext
finding circuit below.

> **Why this matters:** Phases are invisible if you measure immediately: $|+1|^2 = |-
1|^2 = 1$. But phases *interfere*. If we apply the right transformation after the phase k
the cost phase was invisible until the mixer turned phase differences into probability di

Phase kickback for general functions

Phase kickback isn't limited to single-bit functions. For the period-
finding problem, $f(x) = a^x \bmod N$ produces multi-bit outputs. The mechanism is more g
1)^{f(x)}$, the phase encodes the *eigenvalue* of the oracle operator. (An eigenvalue is
finding circuit below.

The pattern generalises — several quantum algorithms (from simple ones like Deutsch-
Jozsa to powerful ones like Shor's) all use phase kickback as their engine. Master it onc

Oracles: packaging functions as gates

What an oracle is

We've been calling the function-evaluating circuit U_f. In quantum computing, this is
a black box that computes a function f reversibly. You don't need to know how it works i

In Deep-Dive 1 (QAOA), we didn't use oracles — the cost function was built directly as a s
finding, the function $f(x) = a^x \bmod N$ is packaged as an oracle U_f. The quantum alg

The Deutsch-Jozsa pattern

```
To see phase kickback in its simplest complete form, consider this problem: you're given

The circuit:

```{figure} ../figures/deutsch-jozsa-circuit.png
:name: fig-deutsch-jozsa-circuit
:alt: Deutsch-Jozsa circuit with a superposition step, oracle, and final Hadamard before

This is phase kickback in its cleanest form: the oracle writes information into a phase,
```

1. Prepare $|0\rangle|-\rangle$ (the second qubit — called an **ancilla**, a helper qubit used as scratch space — is set to $|-\rangle$ via X then H)
2. Apply H to the input qubit → superposition $|+\rangle|-\rangle$
3. Apply the oracle → phase kickback: $\frac{1}{\sqrt{2}}((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle)|-\rangle$
4. Apply H to the input qubit → interference
5. Measure

Step 4 is where interference happens, so let's slow down and watch it. After the oracle, the input qubit is in:

$$\frac{1}{\sqrt{2}}((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle)$$

Now recall what H does to $|0\rangle$ and $|1\rangle$:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \qquad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Applying H to our state, each term produces a contribution to $|0\rangle$ and to $|1\rangle$:

If f is constant ($f(0) = f(1)$, so both phases are the same): the $|0\rangle$ contributions add up (both point the same way), while the $|1\rangle$ contributions cancel (one $+$, one $-$). Result: $|0\rangle$ with certainty. Constructive interference on $|0\rangle$, destructive on $|1\rangle$.

If f is balanced ($f(0) \neq f(1)$, so the phases are opposite): now the $|0\rangle$ contributions cancel and the $|1\rangle$ contributions reinforce. Result: $|1\rangle$ with certainty.

This is interference in its purest form: two amplitudes arriving at the same state, either reinforcing or cancelling depending on their relative phase. The Hadamard is the "beam splitter" that makes them overlap so the phases can interact. Without it, the phases would sit on $|0\rangle$ and $|1\rangle$ separately, invisible to measurement. The Hadamard forces them to interfere — and the outcome tells you whether the phases were the same or different.

One query. The phase kickback converted the function's output into a phase, and the Hadamard converted the phase difference into a measurable bit. This is the template for everything that follows — and the QFT generalises this from "same or different?" to "what's the period?"
```

From one bit to n bits

The Deutsch-Jozsa algorithm generalises: for $f : \{0,1\}^n \to \{0,1\}$ (a function that takes an n-bit string and returns a single bit), promised to be constant or balanced, one quantum query suffices (vs. $2^{n-1} + 1$ classically). The circuit is the same: $H^{\otimes n}$, oracle, $H^{\otimes n}$, measure; and the mechanism is the same: phase kickback + Hadamard interference.

Bernstein-Vazirani extends this: the oracle encodes a hidden string s, and the same circuit recovers all n bits of s in one query. Simon's algorithm goes further: for a function with a hidden period s (where $f(x) = f(x \oplus s)$, with $\oplus$ meaning bitwise XOR), $O(n)$ queries suffice quantumly vs. $O(2^{n/2})$ classically. Each of these is a stepping stone toward Shor's algorithm — we won't detail them here, but they all use phase kickback and Fourier-type interference, applied to increasingly structured problems.

The Quantum Fourier Transform

Why we need it

Shor's algorithm needs to extract the period of a function; not a single bit of information (constant vs. balanced) but a number r that could be anywhere from 1 to N.

The Hadamard transform, which we used in Deutsch-Jozsa, is actually a 1-qubit Fourier transform. To extract a period from a multi-qubit state, we need the full **Quantum Fourier Transform**; the generalisation of the Hadamard to n qubits.

What the QFT does

The classical Discrete Fourier Transform converts a sequence of numbers from the "time domain" to the "frequency domain." A periodic signal with period r has energy concentrated at frequency $1/r$.

The QFT does the same thing to quantum amplitudes. The factor $e^{2\pi i\theta}$ is a complex number of magnitude 1 that encodes an angle θ — it's the mathematical language for phases:

$$\text{QFT}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i x k/2^n} |k\rangle$$

If the input state is periodic; amplitudes concentrated on values equally spaced by r; then the output state is concentrated on multiples of $2^n/r$. Measuring the output gives a multiple of $2^n/r$, from which we can extract r.

How the QFT is built

The remarkable thing about the QFT is that it factorises. The output state can be written as a tensor product of single-qubit states:

$$\mathrm{QFT}|x\rangle = \bigotimes_{\ell=1}^{n} \frac{1}{\sqrt{2}} \left(|0\rangle + e^{2\pi i x/2^\ell}|1\rangle\right)$$

Each qubit in the output depends on the input through a phase $e^{2\pi i x/2^\ell}$. This factorisation means the QFT can be built from:

- **Hadamard gates** (creating the $\frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)$ superpositions)
- **Controlled phase gates** R_k: a **controlled gate** applies an operation to one qubit only when another qubit is $|1\rangle$ — CNOT is a controlled-X; here R_k is a controlled rotation that adds a phase of $e^{2\pi i/2^k}$

For 3 qubits, the circuit is:

"'{figure} ../figures/qft-3qubit-circuit.png :name: fig-qft-3qubit-circuit :alt: Three-qubit quantum Fourier transform circuit built from Hadamards and controlled phase gates.

The QFT factorises into Hadamards and controlled phase rotations, which is why it can be built with only $O(n^2)$ gates instead of a full exponential matrix.

```
where $R_k$ applies a controlled phase of $e^{2\pi i / 2^k}$. The QFT naturally produces

```{figure} ../figures/qft-3qubit-with-swap.png
:name: fig-qft-3qubit-with-swap
:alt: Three-qubit quantum Fourier transform circuit with a final swap layer for bit reve

The final swap layer reverses the QFT's natural bit order so the output register reads in
significant-bit to least-significant-bit order.
```

Gate count: n Hadamards + $n(n-1)/2$ controlled rotations = $O(n^2)$ gates. Compare with the classical Fast Fourier Transform (FFT): $O(n \cdot 2^n)$ operations. The QFT is exponentially faster; but you can't read out the full Fourier transform (measurement collapses it to one value).

> **Common Mistake #1:** "The QFT gives an exponential speedup for Fourier transforms." Not exactly. The classical FFT transforms a vector of 2^n numbers and lets you read all of them. The QFT transforms 2^n amplitudes but only lets you sample one outcome. The speedup comes from the combination of QFT with specific problem structure (like periodicity), not from the QFT alone.

Why the QFT "sees" the period: phase arrows

The QFT works because of interference — the same mechanism we just watched in Deutsch-Jozsa, scaled up. In Deutsch-Jozsa, we had two amplitudes arriving at $|0\rangle$, either reinforcing or cancelling. In Shor's, we have many amplitudes arriving at each output $|k\rangle$, and the question is: do their phases line up?

Each output state $|k\rangle$ receives contributions from all the periodic input states, each carrying a phase $e^{2\pi i x k/2^n}$. Think of each contribution as an arrow on the unit circle, pointing in the direction given by its phase. The total amplitude at $|k\rangle$ is the sum of all these arrows.
```

For our $N = 15$ example (period $r = 4$, register size $2^n = 16$), the four periodic inputs $x = 0, 4, 8, 12$ each contribute an arrow:

"'{figure} ../figures/phase-arrows-qft.png :name: fig-phase-arrows-qft :alt: Phase arrows showing constructive alignment at one frequency and cancellation at another.

When the phases line up, the arrows add; when they spread around the circle, they cancel. That is the whole reason the QFT can see a period.

```
**Left ($k = 4$):** The four arrows point at $0°$, $0°$, $0°$, $0°$. Why? Because the phas
every arrow lands back at $0°$. They all add up: constructive interference. High probabil

**Right ($k = 3$):** The phase is $e^{2\pi i \cdot x \cdot 3/16}$. For $x = 0$: $0°$. For s
zero probability.

The pattern: when $k$ is a multiple of $2^n/r = 4$, the periodicity of the inputs guarante
Jozsa's "same phase = reinforce, different phase = cancel" principle, but now applied acr
frequency generalisation of the Hadamard beam splitter.


## Assembling Shor's algorithm

Now we have all the pieces. Let's put them together for factoring $N = 15$ with $a = 7$.

### Step 1: Superposition

Apply Hadamard to 4 input qubits:

$$|0000\rangle \xrightarrow{H^{\otimes 4}} \frac{1}{4}\sum_{x=0}^{15} |x\rangle$$

We've created a superposition of 16 input values. This is identical to what we did in Deep
Dive 1 for QAOA; the Hadamard creates uniform superposition.

### Step 2: Oracle (modular exponentiation)

Compute $f(x) = 7^x \bmod 15$ in an output register. The state becomes:

$$\frac{1}{4}\sum_{x=0}^{15} |x\rangle|7^x \bmod 15\rangle$$

The output register creates entanglement between input values that produce the same outp

### Step 3: Inverse QFT

Apply the **inverse QFT** to the input register. Since every quantum gate is reversible,

### Step 4: Measure

We get one of $\{0, 4, 8, 12\}$ with equal probability.

### Step 5: Classical post-processing

From the measured value $k$ and the known register size $2^n = 16$:
```

```
$$k/16 \approx j/r$$

The **continued fractions algorithm** extracts $r$ from this rational approximation. Fo

### Step 6: Factor

$a^{r/2} = 7^2 = 49$. Compute $\gcd(49-1, 15) = \gcd(48, 15) = 3$ and $\gcd(49+1, 15) = \g

$15 = 3 \times 5$. Done.

### The cost

For an $n$-bit number $N$: $O(n)$ qubits for the registers, $O(n^2)$ gates for the QFT, a

The companion notebook does **not** implement general modular exponentiation end-
to-end. It keeps the classical reduction exact, then runs a compiled period-
finding toy for one eigenphase branch of the $r = 4$ problem, followed by the same contin
fractions and $\\gcd$ post-processing.

→ **See [notebook `02-cryptography.ipynb`](../notebooks/02-cryptography.ipynb) for the

> **Common Mistake #2:** "Shor's algorithm finds factors by trying all possible factors


## The modular exponentiation circuit

The QFT is elegant and compact ($O(n^2)$ gates). The modular exponentiation; computing $

### Repeated squaring

$a^x \bmod N$ is computed using the **binary expansion** of $x$. Any integer can be writt
1} 2^{n-1} + \cdots + x_1 \cdot 2 + x_0$, where each $x_k$ is 0 or 1. This lets us decompos

$$a^x = a^{x_0 \cdot 1} \cdot a^{x_1 \cdot 2} \cdot a^{x_2 \cdot 4} \cdots a^{x_{n-
1} \cdot 2^{n-1}} \pmod{N}$$

We pre-compute $a^1, a^2, a^4, a^8, \ldots \pmod{N}$ by squaring at each step (hence "rep

Each factor is a **controlled multiplication**: if input bit $x_k = 1$, multiply the outp

### Modular arithmetic on a quantum computer

Multiplication modulo $N$ is built from:

1. **Quantum adders** (efficient reversible circuits for addition modulo $N$)
2. **Modular reduction** (subtract $N$ if the result exceeds $N$, using a comparison and
3. **Uncomputation**: quantum operations must be reversible, so any scratch work must be
called *garbage entanglement* — and this ruins interference because measuring the answer
```

Each controlled multiplication costs $O(n^2)$ gates. There are $2n$ such multiplications
a Toffoli flips a target qubit only when *two* control qubits are both $|1\rangle$, makin

For $N = 15$ (our toy example), the modular exponentiation is small enough to compile by h
2048, it's $\sim 10^{10}$ gates; feasible on a fault-tolerant machine, far too deep for t
Noisy Intermediate-Scale Quantum).

What you should take away

1. **Phase kickback** converts function values into phases. For single-bit functions, the output register acts as a catalyst. For multi-bit functions (like modular exponentiation), the output register stays entangled with th

2. **The QFT** converts periodicity in amplitudes to peaks in frequency. It's built from

3. **Oracles** abstract away function computation. You don't need to know how f is impl

4. **The cost is dominated by the oracle.** The QFT is cheap ($O(n^2)$). The modular expo

5. **Shor's algorithm is not trying all factors.** It's using interference to extract a n
theoretic property (the period) that happens to reveal the factors. The quantum computer

Unit 3: Drug Discovery — *$2 Billion and 12 Years Per Drug*

The Hook

In 2020, the pharmaceutical industry spent approximately $83 billion on research and dev

Why is it so expensive? The cost isn't mostly in the lab work itself. It's in the *failure

The bottleneck is **molecular simulation**. To predict whether a candidate drug will bin
protein complex — the arrangement of electrons that determines binding affinity, reactio
the fundamental law governing quantum systems, which relates a system's energy to the qua
and the only way to predict the molecule's behaviour accurately is to solve that equation

For small molecules — up to about 50 electrons — classical computers can do this well enou
which approximates the many-electron problem by working with the electron density rather
handles molecules of this size in minutes. For a drug candidate docked to a protein bindi
hundreds or thousands of electrons — DFT is too inaccurate and exact methods are unafford

The exponential wall is not a metaphor. Each electron in a molecule occupies an **orbital
a quantum state describing where the electron is likely to be found and what energy it has
orbital** adds the electron's spin (up or down), doubling the count of available states.

The full quantum state of N electrons occupying M orbitals lives in a space of dimens
the number of ways to choose which N orbitals are occupied. This space of all possible c
say, 30 electrons in 60 orbitals — that's $\binom{60}{30} \approx 1.2 \times 10^{17}$ dim

The Bottleneck

Let's understand what makes molecular simulation hard; and why it's a fundamentally diff

The electronic structure problem

Given a molecule — its atoms and their positions — compute its **ground-state energy**: the lowest-energy arrangement of its electrons.

The ground-state energy determines everything chemists care about: whether a reaction is

In the language of quantum mechanics, the ground-state energy is the smallest eigenvalue all the forces between electrons and atomic nuclei — and the Schrödinger equation $H|\Psi the multi-electron generalisation of the qubit states $|\psi\rangle$ you already know.

Why electrons aren't qubits

There's a crucial complication. Electrons are **fermions** — particles that obey the Pau if electrons could pile into the same state, every atom would behave the same.)

This is different from qubits. If you swap two qubits on a quantum computer, nothing spec no minus sign appears. A quantum computer built from qubits doesn't natively enforce fer to-qubit encoding** — a mathematical map that translates between the language of electro

The classical approximation hierarchy

Classical computational chemistry has developed a remarkable hierarchy of approximation

Method	Accuracy	Scaling	What it does
Hartree-Fock (HF)	~99% of total energy	$O(N^4)$	Each electron moves in the averag
DFT	Good for weak correlation	$O(N^3)$	Works with electron density instead of the
MP2	Perturbative correction	$O(N^5)$	Adds a first-order correction for electron-electron interaction
CCSD(T)	"Gold standard"	$O(N^7)$	Systematically accounts for how pairs (and tripl
Full CI	Exact (within basis)	$O(e^N)$	Tries every possible electron arrangement — brute force

The pattern: more accuracy costs exponentially more computation.

The simplest method, Hartree-Fock, treats each electron as if it moves through the averag a **mean-field** approximation. This ignores the fact that electrons actively dodge each field approximation is called **electron correlation**. Hartree-Fock misses it entirely. Each step up the ladder captures more correlation, but at drasti

The gap between CCSD(T) (the best practical classical method, scaling as N^7) and Full where the mean-field picture breaks down entirely, such as transition metal complexes (m orbitals) and reaction transition states — CCSD(T) fails and Full CI is unaffordable. The

The Quantum Angle

A quantum computer can represent the quantum state of N electrons directly, using qubi
classical algorithm for finding the ground-state energy of a molecular Hamiltonian.

From electrons to qubits

To put a molecular problem on a quantum computer, we need to translate the language of el
electron interactions become a sum of Pauli operators — just like the cost Hamiltonian in

The translation isn't trivial because of the fermion sign rule. Different **encodings**
Wigner, Bravyi-Kitaev, and others) handle the minus signs differently, making different
Dive 3 walks through the Jordan-Wigner encoding in detail; for the full story, see our co
book).

After encoding, the molecular Hamiltonian becomes a weighted sum of Pauli operators —
terms like $Z_0 Z_1$, $X_0 X_1$, and so on — with real-valued coefficients computed from

VQE: the variational quantum eigensolver

VQE uses the **variational principle**: for any trial state $|\psi(\theta)\rangle$, the
state energy. (The notation $\langle \psi | H | \psi \rangle$ means "the expected value o
the quantum analogue of a weighted average.)

$$E_0 \leq \langle \psi(\theta) | H | \psi(\theta) \rangle$$

The algorithm:

1. **Prepare a trial state** on the quantum computer using a parameterised circuit (the *
German for "initial guess," meaning the family of trial states the circuit can reach): $|
2. **Measure the energy** by decomposing H into a sum of Pauli operators and measuring
3. **Optimise classically**: feed the measured energy to a classical optimiser, which su
4. **Repeat** until the energy converges.

This is the same variational loop as QAOA (Unit 1); quantum circuit produces an estimate,
inspired circuits instead of alternating problem/mixer layers).

The ansatz: encoding chemical intuition

The quality of VQE depends heavily on the ansatz; the family of quantum states the parame

UCCSD (Unitary Coupled Cluster): inspired by the classical CCSD method. It applies s

Hardware-efficient ansätze: generic circuits (layers of rotations and entangling ga
regions of parameter space where the energy landscape is exponentially flat, leaving the

Measurement overhead

There's a catch. The molecular Hamiltonian, after encoding, becomes a sum of many Pauli t
Wigner, there are about 15 terms. For a drug-sized molecule, there could be millions. Eac

```
## Worked Example

Let's keep the runnable example honest: take **H₂** at a single bond length, reduce it to
qubit Hamiltonian with precomputed coefficients, and run the VQE loop on that fixed geome

The full minimal-basis H₂ problem starts with 4 spin-orbitals and 4 qubits before symmetr
qubit Hamiltonian at $R = 0.735$ Å, then shows the measurement and optimisation loop clea

### The pipeline

1. **Start from a reduced Hamiltonian** for H₂ at one geometry
2. **Prepare the Hartree-Fock reference** and a one-parameter entangling ansatz
3. **Measure the Pauli terms directly** in the $Z$, $XX$, and $YY$ bases
4. **Run the variational loop**; sweep $\theta$ and estimate the energy at each point
5. **Benchmark against exact diagonalisation** of the same reduced Hamiltonian

This is not yet a potential-energy-surface calculation. To build the full H₂ curve, you w

```{figure} ../figures/h2-potential-energy.png
:name: fig-h2-potential-energy
:alt: Ground-state energy of H2 as a function of bond length.

A chemistry calculation is not one number but a curve: as bond length changes, the ground
state energy traces a potential-energy surface whose minimum marks the stable molecular
```

Even at this reduced scale, the structure is the same as the larger chemistry story: a Hamiltonian decomposed into Pauli terms, a trial state prepared on the quantum device, repeated measurements, and a classical search for the lowest energy. The notebook's direct XX and YY measurements matter because they expose the real measurement overhead rather than hiding it in an analytic shortcut.

→ The next chapter builds the full molecule-to-qubit pipeline behind this reduced demo — from fermionic Hamiltonian to Pauli measurements to variational optimisation.

Back to the pharmacy

We computed the energy of H_2 — the simplest molecule there is. How does this help find a $2.6 billion drug?

The pipeline is identical for any molecule. Only two things change: the number of qubits (one per orbital) and the number of Pauli terms in the Hamiltonian (which scales as $O(M^4)$ for M orbitals). H_2 needs 4 qubits and ~15 terms. Caffeine would need ~100 qubits. A drug-sized active site might require several hundred. The algorithm doesn't change — you still prepare a trial state, measure Pauli terms, optimise classically, repeat.

The practical strategy, which we'll develop fully in Unit 8, is to use a classical computer for the "easy" electrons and a quantum computer only for the strongly correlated ones in the active site. This hybrid approach is how quantum computing will likely enter the pharmaceutical pipeline — not replacing classical chemistry, but solving the hardest sub-problems that classical methods can't.
```

Reality Check

What's been demonstrated. VQE has been run on real quantum hardware for small molecules: H_2 (2 qubits, various groups since 2014), LiH (4 qubits), BeH_2 (6 qubits), and H_2O (up to 12 qubits with **tapering** — a technique that exploits molecular symmetries to remove qubits from the Hamiltonian). Google's 2020 Hartree-Fock experiment on 12 qubits was a milestone, though it used a non-variational approach.

The gap to drug discovery. A drug-sized active site might require 50–100 qubits (after encoding and tapering — exploiting symmetries to reduce qubit count) and circuits with thousands or millions of gates. Current NISQ devices can handle at most ~20 qubits with meaningful accuracy. **Error mitigation** techniques (like zero-noise extrapolation, which runs circuits at artificially increased noise levels and extrapolates back to zero) help but don't close the gap for industrially relevant molecules.

The measurement problem. For a molecule with M orbitals, the number of Pauli terms in the Hamiltonian scales as $O(M^4)$. Each term needs many measurement shots for acceptable statistics. For 100 orbitals, you might need 10^{10} measurements. Techniques like **classical shadow tomography** (which extracts many properties from relatively few randomised measurements) and grouping of commuting terms reduce this, but it remains a bottleneck.

What would change the picture. Fault-tolerant quantum computers would enable **quantum phase estimation** (Unit 7) instead of VQE; giving exact energies rather than variational upper bounds, without the measurement overhead. But QPE requires much deeper circuits and is a post-NISQ algorithm.

What's real today: The real near-term strategy is **active-space methods**: use a classical computer for the "easy" electrons (core, weakly correlated) and a quantum computer for the "hard" ones (strongly correlated, in the active space). This hybrid approach works for small molecules today, and we'll develop it fully for industrial-scale systems in Unit 8 (Climate & Energy).

Chef's Notes

- **This chapter introduces the molecular Hamiltonian pattern.** In Unit 1, we encoded an optimisation objective as a cost Hamiltonian. Here we encode the physical Hamiltonian of a molecule. The mathematical structure is the same; a sum of Pauli operators; but the origin is different: physics rather than combinatorics.
- **VQE is QAOA's cousin.** Same variational loop, different cost function, different ansatz. If you understood the QAOA loop in Unit 1, VQE is a natural extension.
- **Fermion-to-qubit encodings are a deep topic.** We've given the 30-second version (Jordan-Wigner, Bravyi-Kitaev). For the full story; including parity encoding, ternary tree encoding, tapering, and the algebraic structure behind all of them; see our companion book: From Molecules to Qubits.
- **H_2 is the "Hello World" of quantum chemistry.** It's the simplest molecule with a non-trivial electronic structure — small enough to verify classically,

rich enough to show the full VQE pipeline, and pedagogically indispensable. Our encodings book uses H_2 throughout for the same reason.

- **The path from here to real drug discovery** runs through Unit 8 (Climate & Energy), where we'll use quantum embedding methods to handle realistic molecular sizes; computing the active-site electrons quantumly and the environment classically.
- **Further reading:**
  - Peruzzo et al. (2014). A variational eigenvalue solver on a photonic quantum processor. Nature Communications 5:4213 (arXiv:1304.3061)
  - McArdle, Endo, Aspuru-Guzik, Benjamin, Yuan (2020). Quantum computational chemistry. Reviews of Modern Physics 92:015003 (arXiv:1808.10402)
  - Google AI Quantum (2020). Hartree-Fock on a superconducting qubit quantum computer. Science 369:1084–1089 (arXiv:2004.04174)
  - Azariah et al. From Molecules to Qubits. github.com/johnazariah/encodings-book
  - Szabo and Ostlund (1996). Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory. The classical reference for computational chemistry foundations.

Deep-Dive 3: The VQE Pipeline

This deep dive pairs with Unit 3 (Drug Discovery), which explained why molecular simulation is hard and how VQE addresses it. Here we build the VQE pipeline from molecule to energy, step by step.

In This Chapter

- **What you'll learn:** How electrons become qubits; second quantisation, fermion-to-qubit encodings, Pauli-basis measurement, and the variational optimisation loop that finds the ground-state energy.
- **What you need:** From Deep-Dive 1, you know qubits, superposition, CNOT, and the variational optimisation loop (QAOA). From Deep-Dive 2, you know phase kickback and the idea that quantum operations encode information in phases. Here we add new physics: fermions.
- **Runnable version:** The companion notebook `03-drug-discovery.ipynb` runs a single-geometry, reduced-Hamiltonian H_2 VQE demo on a cloud Quokka.

Why electrons aren't qubits

In Chapters 2 and 4, qubits were abstract binary variables; node colours in MaxCut, input bits in Shor's algorithm. We could assign any meaning we wanted to $|0\rangle$ and $|1\rangle$.

Electrons are different. They obey rules that qubits don't, and ignoring these rules gives wrong answers. The central rule is:

Electrons are fermions. The wavefunction of a multi-electron system must be antisymmetric — swapping any two electrons flips the sign:

$$\Psi(\dots, \mathbf{r}_i, \dots, \mathbf{r}_j, \dots) = -\Psi(\dots, \mathbf{r}_j, \dots, \mathbf{r}_i, \dots)$$

This minus sign is the Pauli exclusion principle: two electrons can't be in the same state (if they were, swapping them would change nothing, but the minus sign says the wavefunction must flip; the only function equal to its own negative is zero).

Qubits don't have this rule. If you swap two qubits, no minus sign appears — the state is unchanged. If we naively map electrons to qubits — electron in orbital i → qubit i is $|1\rangle$, not in orbital i → qubit i is $|0\rangle$ — we get the right occupations but the wrong phases. The standard encoding produces correct energies for H_2, but for larger molecules the missing signs lead to completely wrong results.

> **Common Mistake #1:** "Qubits are quantum, electrons are quantum, so mapping is trivial." No. Swapping two qubits produces no sign change. Swapping two electrons produces a minus sign. The encoding must bridge this gap. This is the entire subject of the companion book From Molecules to Qubits.

Second quantisation: the language of electrons

Occupation numbers

Instead of tracking where each electron is (first quantisation), we track which orbitals are occupied (second quantisation). For a molecule with M spin-orbitals, the state is described by an occupation vector:

$$|n_0, n_1, \dots, n_{M-1}\rangle$$

where $n_i = 0$ (orbital empty) or $n_i = 1$ (orbital occupied). For H_2 in a minimal basis: 4 spin-orbitals, 2 electrons. A possible state: $|1, 1, 0, 0\rangle$ (the two lowest orbitals occupied).

This looks like qubits! And it is: each orbital maps to one qubit, with $|1\rangle$ meaning "occupied." But the antisymmetry rule means the ordering of orbitals matters. $|1, 1, 0, 0\rangle$ and $|0, 0, 1, 1\rangle$ are not related by a simple swap; the sign depends on how many occupied orbitals you "pass through" to move an electron.

Creation and annihilation operators

The tools that respect antisymmetry are **creation** ($a_i^\dagger$) and **annihilation** (a_i) operators. The symbol † ("dagger") denotes the Hermitian conjugate — roughly, the quantum version of transposing and complex-conjugating a matrix:

- $a_i^\dagger$ creates an electron in orbital i (flips qubit i from $|0\rangle$ to $|1\rangle$, with sign adjustments)
- a_i removes an electron from orbital i (flips $|1\rangle$ to $|0\rangle$, with sign adjustments)

The key property — the thing that encodes the minus sign — is the **anticommutation relation**:

$$\{a_i, a_j^\dagger\} \equiv a_i a_j^\dagger + a_j^\dagger a_i = \delta_{ij}$$

The curly braces $\{A, B\} \equiv AB + BA$ denote the anticommutator — unlike the commutator $[A, B] = AB - BA$ you may know from Pauli algebra. And δ_{ij} is the Kronecker delta: 1 when $i = j$, 0 otherwise.

What does this say? Creating electron j then destroying electron i is not the same as destroying i then creating j — they differ by a sign (unless $i = j$). Concretely: $a_i a_i^\dagger + a_i^\dagger a_i = 1$, so if creating then destroying gives some result, destroying then creating gives 1 minus that result. For different orbitals ($i \neq j$), the anticommutator is zero: $a_i a_j^\dagger = -a_j^\dagger a_i$. Swapping the order flips the sign. That minus sign is the entire difference between fermions and bosons, and it's the mathematical expression of the Pauli exclusion principle.

The molecular Hamiltonian

In second quantisation, the molecular Hamiltonian is:

$$H = \sum_{ij} h_{ij}\, a_i^\dagger a_j + \frac{1}{2} \sum_{ijkl} h_{ijkl}\, a_i^\dagger a_j^\dagger a_k a_l$$

The first sum is the **one-electron terms** (kinetic energy + electron-nucleus attraction). The second sum is the **two-electron terms** (electron-electron repulsion). The coefficients h_{ij} and h_{ijkl} are called **molecular integrals** — they come from classical quantum chemistry computations and depend on the molecule's geometry and its **basis set** (the finite set of mathematical functions used to approximate orbitals, like choosing how many Fourier terms to keep in a series).

For H_2 in STO-3G (a minimal basis set: 3 Gaussian functions per orbital): there are about 15 unique molecular integrals. For a drug-sized molecule: millions. But the Hamiltonian always has this same structure — sums of products of creation and annihilation operators, weighted by integrals.

Fermion-to-qubit encodings

The problem

We have a Hamiltonian written in terms of $a_i^\dagger$ and a_i (which obey anticommutation rules). We need to express it in terms of Pauli operators X, Y, Z, I (which obey commutation rules). The encoding translates between these two algebras.

Jordan-Wigner: the simplest encoding

The **Jordan-Wigner** encoding (1928; older than quantum computing!) maps each orbital to one qubit. The occupation number is stored directly: orbital i occupied → qubit i is $|1\rangle$.

The creation operator becomes:

$$a_i^\dagger \to \frac{1}{2}(X_i - iY_i) \otimes Z_{i-1} \otimes Z_{i-2} \otimes \cdots \otimes Z_0$$

Here $\otimes$ is the **tensor product** — it combines operators on separate qubits into one multi-qubit operator. (The $Z_0 Z_1$ you saw in QAOA is shorthand for $Z_0 \otimes Z_1$.) The factor $\frac{1}{2}(X_i - iY_i)$ is called the **raising operator** — it flips $|0\rangle$ to $|1\rangle$ (creating an electron) and annihilates $|1\rangle$ (gives zero — you can't create an electron in an orbital that's already occupied).

The string of Z operators is the price we pay for antisymmetry. Each Z_k checks whether orbital k is occupied; if it is, it contributes a minus sign. This enforces the rule that creating an electron "past" occupied orbitals picks up the right number of minus signs.

Example: $a_2^\dagger$ in a 4-qubit system is $\frac{1}{2}(X_2 - iY_2) \otimes Z_1 \otimes Z_0$. If orbitals 0 and 1 are both occupied, the two Z operators each contribute -1, and the total sign is $(-1)^2 = +1$. If only orbital 0 is occupied, the sign is $(-1)^1 = -1$.

What the Hamiltonian looks like after encoding

Each fermionic term $a_i^\dagger a_j$ becomes a sum of Pauli strings. For H_2 with Jordan-Wigner, the full Hamiltonian decomposes into about 15 terms:

$$H = g_0 I + g_1 Z_0 + g_2 Z_1 + g_3 Z_0 Z_1 + g_4 X_0 X_1 + g_5 Y_0 Y_1 + \cdots$$

Each g_k is a real number computed from the molecular integrals. The Hamiltonian is now a weighted sum of Pauli operators; exactly the kind of thing we know how to measure on a quantum computer.

The locality problem

The Z string in Jordan-Wigner means a single fermionic operator can touch all qubits. For orbital i, the string has length i. For the last orbital in a 100-qubit system, the string is 100 qubits long.

Bravyi-Kitaev encoding improves this: the longest string is $O(\log n)$ instead of $O(n)$. The tradeoff: the mapping between occupation numbers and qubit states is more complex (it uses a binary tree structure). Other encodings (parity, ternary tree, compact) make different tradeoffs.

For the full story; including the algebraic structure behind all encodings, tapering to reduce qubit count, and benchmarks; see From Molecules to Qubits.

Measuring the energy

The variational principle

VQE uses the **variational principle**: for any quantum state $|\psi\rangle$, the expected energy is an upper bound on the true ground-state energy:

$$E_0 \leq \langle\psi|H|\psi\rangle$$

Equality holds only when $|\psi\rangle$ is the ground state. So: prepare trial states, measure their energy, and minimise. The minimum you find is your best estimate of E_0.

Decomposing the energy measurement

The Hamiltonian is $H = \sum_k g_k P_k$ where each P_k is a Pauli string (like $Z_0 Z_1$ or $X_0 X_1$). The expected energy is:

$$\langle H\rangle = \sum_k g_k \langle P_k\rangle$$

We measure each $\langle P_k\rangle$ separately. The total energy is the weighted sum.

Measuring Pauli strings

How do you measure $\langle Z_0 Z_1 \rangle$? Run the circuit, measure both qubits in the computational basis, compute $z_0 \cdot z_1$ where $z_i = +1$ if you measured $|0\rangle$ and $z_i = -1$ if you measured $|1\rangle$. Average over many shots.

How do you measure $\langle X_0 X_1 \rangle$? You can't measure X directly in the computational basis. But applying a Hadamard before measurement rotates X into Z:

$$HXH = Z$$

So: apply H to both qubits, then measure in the computational basis. The result gives $\langle X_0 X_1 \rangle$.

How do you measure $\langle Y_0 Y_1 \rangle$? Apply $S^\dagger$ then H to each qubit before measuring in the computational basis. This rotates the measurement axis from Z to Y, letting you estimate $\langle Y \rangle$ from computational-basis measurement results.

Each distinct measurement basis requires a separate circuit execution. This is the **measurement overhead**; for T Pauli terms in the Hamiltonian, you need at least T different circuit runs (reducible by grouping commuting terms).

> **Common Mistake #2:** "VQE measures the energy with one circuit." No. The Hamiltonian is a sum of terms. Each term (or group of commuting terms) requires its own measurement circuit. For molecules with thousands of Pauli terms, this is a major cost; potentially requiring billions of total measurement shots for acceptable statistical precision.

The ansatz: parameterised chemistry

What an ansatz is

The **ansatz** is the parameterised quantum circuit that prepares trial wavefunctions. In QAOA (Deep-Dive 1), the ansatz was the alternating problem/mixer layers with parameters γ, β. In VQE, the ansatz is motivated by chemistry.

Starting point: the Hartree-Fock state

The simplest starting point is the **Hartree-Fock state** — the best approximation that treats each electron as independent. For H_2: two electrons in the two lowest spin-orbitals:

$$|\text{HF}\rangle = |1100\rangle$$

In circuit terms: `x q[0]; x q[1];` (flip qubits 0 and 1 from $|0\rangle$ to $|1\rangle$).

Adding correlation: excitations

The Hartree-Fock state ignores electron correlation; it treats each electron as if it moves in the average field of all others. To capture correlation, we apply **excitation operators**: controlled rotations that mix in states where electrons have been moved to different orbitals.

A **single excitation** $a_i^\dagger a_j$ moves one electron from orbital j to orbital i. A **double excitation** $a_i^\dagger a_j^\dagger a_k a_l$ moves two electrons. The UCCSD (Unitary Coupled Cluster with Singles and Doubles) ansatz applies all possible singles and doubles with tunable amplitudes:

$$|\psi(\theta)\rangle = e^{\sum_{ij} \theta_{ij}^{(1)}(a_i^\dagger a_j - \text{h.c.})} \cdot e^{\sum_{ijkl} \theta_{ijkl}^{(2)}(a_i^\dagger a_j^\dagger a_k a_l - \text{h.c.})} |\text{HF}\rangle$$

Here "h.c." stands for **Hermitian conjugate** — the dagger of the preceding term. Writing $A - \text{h.c.}$ means $A - A^\dagger$, which makes the exponent anti-Hermitian and ensures $e^{A-A^\dagger}$ is unitary (a valid quantum gate). The exponential of an operator is defined by its Taylor series: $e^A = I + A + A^2/2 + \cdots$

After encoding, each excitation becomes a sequence of CNOTs and parameterised rotations.

For H_2: one parameter

H_2 has only one relevant double excitation — moving both electrons from the **bonding orbital** (lower energy, electron density concentrated between the two nuclei) to the **antibonding orbital** (higher energy, a node between the nuclei). So the entire ansatz is:

"'{figure} ../figures/h2-vqe-ansatz.png :name: fig-h2-vqe-ansatz :alt: H2 VQE ansatz circuit with an X gate for the Hartree-Fock state, an Ry rotation, and a CNOT.

For H_2, the ansatz is intentionally tiny: one parameter is enough to interpolate between the Hartree-Fock reference and the correlated ground state.

```
One parameter $\theta$, one CNOT. The simplest possible VQE circuit; but it captures the


## The classical optimiser

The same variational loop from Deep-Dive 1 (QAOA):

1. Choose parameters $\theta$
2. Prepare $|\psi(\theta)\rangle$ on the quantum computer
3. Measure each Pauli term → compute $\langle H \rangle$
4. Feed $\langle H \rangle$ to a classical optimiser:
   - **COBYLA** (gradient-free — doesn't need derivatives)
   - **L-BFGS** (gradient-based — faster when gradients are available)
  - **SPSA** (stochastic gradient estimate — popular on noisy hardware because it needs
5. Optimiser suggests new $\theta$
6. Repeat until convergence
```

```
The variational principle guarantees $\langle H \rangle \geq E_0$; every measurement is

For H₂ (one parameter), the landscape is a smooth function of $\theta$ —
a simple 1D minimisation. For larger molecules with hundreds of parameters, the landscap
based optimisers need the **parameter-shift rule** to estimate gradients from circuit me
\pi/2$) and take the difference.


## Putting it all together

The complete VQE pipeline:

```{figure} ../figures/vqe-pipeline.png
:name: fig-vqe-pipeline
:alt: The VQE pipeline from molecular Hamiltonian to ground-state energy through a quant
classical optimisation loop.

VQE is useful because the quantum computer only evaluates expectation values; the chemis
```

Every step is well-defined. The pipeline works for any molecule; only the integrals and the circuit size change. H_2 needs 2–4 qubits. Caffeine needs ~100. A drug-protein complex needs thousands; beyond current hardware but within the scope of fault-tolerant machines.

The companion notebook keeps the runnable example narrower than the full theory in this chapter. It starts from a precomputed reduced H_2 Hamiltonian at one geometry, measures the Z, XX, and YY terms directly on Quokka, and compares the VQE minimum against exact diagonalisation of that same reduced model.

→ **See notebook `03-drug-discovery.ipynb` for the runnable version.**

What you should take away

1. **Electrons are fermions, not qubits.** The antisymmetry of the fermionic wavefunction requires an encoding (Jordan-Wigner, Bravyi-Kitaev, etc.) that maps fermionic operators to Pauli operators with the correct signs.
2. **The Hamiltonian becomes a sum of Pauli strings.** After encoding, the molecular energy is just a weighted sum of measurable quantities; each Pauli string can be estimated from repeated measurements.
3. **Measurement has a cost.** Each distinct Pauli basis requires a separate circuit execution. For large molecules, the measurement overhead can dominate the total computation time.
4. **The ansatz encodes chemical intuition.** Good ansätze (like UCCSD) start from the Hartree-Fock state and apply physically motivated excitations. Bad ansätze miss the ground state or suffer from **barren plateaus** — regions where the energy landscape is exponentially flat and the optimiser has no useful gradient to follow.
5. **VQE is the same variational loop as QAOA.** If you understood Deep-Dive 1, you understand VQE's architecture. The difference is the cost function
```

(molecular energy, not graph cuts) and the ansatz (chemistry-motivated, not graph-motivated).

6. **For the full encoding story,** see From Molecules to Qubits.

Unit 4: Machine Learning

10^8 Features and Counting

The Hook

In 2006, Netflix offered a $1 million prize to anyone who could improve their movie recommendation algorithm by 10%. The winning team needed three years and a blend of over 100 different algorithms to get there. The core challenge wasn't the prize; it was the **dimensionality** of the problem.

Netflix had roughly 480,000 users and 18,000 movies. Each user-movie pair is a potential data point. The "feature space" that recommendation algorithms operate in has dimensions proportional to the number of users times the number of features describing each user's preferences. Modern recommendation engines at companies like Spotify, TikTok, and Amazon operate in feature spaces with 10^8 or more dimensions.

Machine learning at scale is fundamentally a problem of high-dimensional geometry. A classifier draws a boundary in feature space separating "cat" from "dog," "spam" from "not spam," "buy" from "don't buy." The more dimensions, the more expressive the boundary can be; but the harder it is to find.

In a space with 10^8 dimensions, even defining what "similar" means becomes a computational challenge. The classical solution is the **kernel trick**: a kernel function $K(x, x')$ computes the similarity between two data points by implicitly working in a high-dimensional feature space without ever explicitly constructing it. This is what makes **Support Vector Machines** (SVMs) powerful — an SVM finds the boundary that maximises the margin (the gap between the boundary and the nearest data points of each class), and it needs only pairwise similarities, not the raw feature vectors.

But some useful feature spaces are so large that even the kernel trick breaks down. What if the feature space is exponentially large — and the similarity measure is classically intractable?

The Bottleneck

Return to Netflix. You have 480,000 users, each described by their ratings on 18,000 movies. Two users are "similar" if they tend to rate the same movies the same way. A recommendation engine needs to compute this similarity efficiently — and it needs to do so in a space with hundreds of thousands of dimensions.

The kernel trick handles this: instead of explicitly constructing each user's position in the full feature space $\phi(x)$ and computing inner products $\langle \phi(x), \phi(x') \rangle$, you compute the similarity $K(x, x') = \langle \phi(x), \phi(x') \rangle$ directly from the raw data. The classifier never sees the high-dimensional representation; it only needs the kernel matrix $K_{ij} = K(x_i, x_j)$ over the training set (the labelled examples the algorithm learns from).

This works well when K is cheap to compute. But consider what happens as the feature space grows:

- A feature map into a space of dimension 2^n (exponential in the number of input features) — where user preferences interact in combinatorially many ways
- A kernel that requires sampling from a distribution that's hard to sample classically
- A learning task where the relevant similarity structure has no efficient classical description

In these cases, the kernel trick doesn't help; you can't compute $K(x, x')$ efficiently, even though you never explicitly construct $\phi(x)$.

The question becomes: **are there feature maps whose kernels are hard to compute classically but easy to compute quantumly?** And if so, do they correspond to learning tasks anyone actually cares about?

The Quantum Angle

The Hilbert space as a feature space

The key geometric insight. A quantum computer with n qubits operates in a Hilbert space of dimension 2^n. Preparing a quantum state $|\phi(x)\rangle$ from classical data x is a **quantum feature map**; it maps data into an exponentially large space, natively.

The inner product between two quantum states $|\phi(x)\rangle$ and $|\phi(x')\rangle$ is:

$$K(x, x') = |\langle\phi(x')|\phi(x)\rangle|^2$$

This is just the **Born rule** — the principle we've relied on since Unit 1: the probability of a measurement outcome equals the squared magnitude of its amplitude. Here, it gives us the probability of measuring $|\phi(x)\rangle$ and finding it in state $|\phi(x')\rangle$. A quantum computer can estimate this probability by preparing $|\phi(x)\rangle$, applying $U^\dagger_{\phi(x')}$, and measuring. The result is a kernel value.

If the feature map $|\phi(x)\rangle$ has no efficient classical simulation; if there's no way to compute $\langle\phi(x')|\phi(x)\rangle$ efficiently on a classical computer; then this quantum kernel is something genuinely new. The quantum computer is computing a similarity measure that a classical computer cannot.

Quantum kernel estimation

The algorithm is straightforward:

1. **Encode** classical data point x into a quantum state $|\phi(x)\rangle$ using a parameterised circuit $U_\phi(x)|0\rangle^n$
2. **Compute the kernel** $K(x_i, x_j) = |\langle 0^n|U_\phi(x_j)^\dagger U_\phi(x_i)|0^n\rangle|^2$ by preparing $U_\phi(x_i)|0\rangle$, applying $U_\phi(x_j)^\dagger$, and measuring the probability of getting $|0\rangle^n$
3. **Build the kernel matrix** K_{ij} over the training set (the labelled examples the algorithm learns from)

4. **Train a classical SVM** using this kernel matrix; the classical optimisation is the same as for any kernel SVM

The quantum computer is only used for step 2 — computing kernel values that are classically intractable. Everything else is classical. This encode → measure → classically-optimise pattern is the same variational architecture we saw in QAOA (Unit 1) and VQE (Unit 3) — the quantum computer evaluates a function that a classical computer cannot, and classical methods do the rest.

When does this actually help?

This is the most contested question in quantum machine learning. There are three scenarios:

Provable advantage exists for constructed problems. Huang, Broughton, et al. (2022) showed that there exist classification tasks where quantum kernels achieve exponentially better accuracy than any classical learner using the same data; but these tasks are specifically designed around the structure of quantum circuits. They're real proofs of separation, but the tasks are artificial.

Dequantisation results limit the advantage. Tang (2019) showed that several claimed quantum ML speedups (for recommendation systems, principal component analysis — extracting the most important axes of variation in high-dimensional data) can be matched classically under certain conditions. The classical algorithms are "quantum-inspired"; they use similar sampling techniques. This doesn't kill quantum ML, but it narrows the territory where advantage can exist.

The data loading bottleneck. To use a quantum kernel, you need to encode classical data into quantum states. If the data has N features, encoding typically requires $O(N)$ or $O(\text{poly}(N))$ gates. For $N = 10^6$ features (common in practice), this circuit is too deep for near-term hardware. Quantum ML advantage, if it exists for practical problems, likely requires either quantum data (data that's already quantum) or very compact encodings.

What's the honest picture?

Quantum kernel methods are the most rigorous approach to quantum ML; they come with provable guarantees under specific conditions. But the conditions are narrow, and it's an open question whether natural (non-constructed) datasets exhibit the kind of structure that quantum kernels exploit.

The alternative: **variational quantum ML** (parameterised quantum circuits trained end-to-end as classifiers); is more heuristic. It's easier to run on near-term hardware but has weaker theoretical foundations and faces barren plateau problems (exponentially flat optimisation landscapes) at scale.

Worked Example

We build a quantum kernel classifier for a synthetic 2D dataset: two classes arranged in a pattern (two interleaved half-moons) that's separable by a quantum feature map but not by a linear classifier.

“‘{figure} ../figures/kernel-decision-boundary.png :name: fig-kernel-decision-boundary :alt: Two interleaved half-moon classes with a straight boundary on the left and a curved boundary on the right.

The point of a kernel is geometric: the classes are not linearly separable in the raw coordinates, but a richer feature space can make the boundary simple.

```
1. Generate synthetic data (two interleaved half-moons, 40 points)
2. Define a quantum feature map: encode each 2D point $(x_1, x_2)$ into a 2-
qubit state using parameterised rotations and entanglement
3. Compute the quantum kernel matrix by running circuits on Quokka
4. Train a classical SVM with the quantum kernel
5. Compare the decision boundary with a classical RBF (radial basis function) kernel, whi
\gamma\|x - x'\|^2}$

For this toy 2D problem, both kernels classify the data well —
the quantum kernel doesn't outperform the classical RBF kernel. That's the honest result
and whether the encoding remains feasible. That's the open question the Reality Check ado

→ *The next chapter builds the quantum kernel pipeline from scratch, and shows you the co

### Back to the recommender

We started with Netflix's 480,000 users and $10^8$-dimensional preference space. Could a

In principle, yes — a quantum feature map encodes each user's preference vector into an e
tolerant hardware) or data that's already quantum.

The more likely near-term path is smaller, harder problems: classifying outputs of quant
and the feature vectors are naturally compact.


## Reality Check

**The quantum ML landscape is contentious.** Unlike factoring (where Shor's algorithm gi

**What's been demonstrated.** Quantum kernel classifiers have been run on real hardware

**The most promising direction** may not be kernel methods at all, but using quantum comp
models that learn to *produce* new data resembling the training distribution, rather tha
are a natural fit. A **Born machine** treats the measurement probability distribution of
world tasks is an open question.

**What would change the picture:** Discovery of a natural, practically important dataset
tolerant quantum computers that eliminate the data-loading bottleneck.

**What's real today:** Quantum ML is the most speculative application area in this book.
but the path to practical advantage is less clear than for any other unit.


## Chef's Notes
```

- **Hilbert space as feature space** is a genuinely deep idea. It reframes quantum mechan
dimensional vector space, and measurement *is* an inner product. If you've ever wondered

- **Connection to Unit 1 (Logistics).** The quantum kernel circuit —
parameterised gates, entanglement, measurement — is structurally identical to a QAOA lay

- **The Born rule as a kernel evaluator.** In Unit 2, we used measurement to extract peri

- **Connection to Unit 5 (Finance).** Quantum sampling; preparing and measuring quantum

- **Why we include this unit despite the uncertainty.** ML is what most readers will want

- **Further reading:**
  - Havlíček, Córcoles, Temme, et al. (2019). *Supervised learning with quantum-
enhanced feature spaces.* [Nature 567:209–212](https://doi.org/10.1038/s41586-
019-0980-2) ([arXiv:1804.11326](https://arxiv.org/abs/1804.11326))
  - Huang, Broughton, Mohseni, et al. (2022). *Power of data in quantum machine learning
021-22539-9)
  - Tang (2019). *A quantum-inspired classical algorithm for recommendation systems.* [
  - Schuld and Killoran (2019). *Quantum Machine Learning in Feature Hilbert Spaces.* [
  - Mohanty, Behera & Ferrie (2024). *Solving the vehicle routing problem via quantum su
024-00161-4)
  - Nair & Ferrie (2025). *Local surrogates for quantum machine learning.* [arXiv:2506.

Deep-Dive 4: Quantum Kernels in Detail

_This deep dive pairs with Unit 4 (Machine Learning), which explained why quantum feature

In This Chapter

- **What you'll learn:** How to design a quantum feature map, compute kernel values from
- **What you need:** From the earlier units, you know qubits and measurement (Unit 1), pa
Dive 1), and the variational loop (Units 1 and 3). Here we use measurement in a new way: t
- **Runnable version:** The companion notebook [`04-machine-learning.ipynb`](../notebo
machine-learning.ipynb) trains a quantum kernel SVM on a cloud Quokka.

The geometric picture

In Unit 1, we encoded binary decisions as qubit states: red/blue → $|0\rangle$/$|1\rangl
valued feature vectors; into qubit states. The quantum state space becomes a feature spac

A single qubit lives on the **Bloch sphere**; a 2D surface parameterised by two angles $(

Two qubits live in a 4-dimensional complex space ($\mathbb{C}^4$). With entanglement, th

For n qubits: the feature space has 2^n complex dimensions. A classical computer can

Designing a feature map

```
A **quantum feature map** is a parameterised circuit $U_\phi(x)$ that maps a classical da

### The simplest feature map: product encoding

Encode each feature independently:

```{figure} ../figures/product-encoding.png
:name: fig-product-encoding
:alt: Product encoding with independent Ry rotations on each qubit.

Independent single-qubit rotations create a feature map with no entanglement, which is w
```

This produces a **product state** — a tensor product of independent single-qubit states — with no entanglement. The feature space is just two independent Bloch spheres. The kernel $K(x, x')$ is the product of two cosines; equivalent to a classical kernel. No quantum advantage.

Adding entanglement: the ZZ feature map

Entanglement is what makes quantum feature maps non-classical. Add a CNOT and an interaction term:

```
"'{figure} ../figures/zz-feature-map.png :name: fig-zz-feature-map :alt: ZZ feature
map with Ry encoding and a CNOT-Rz-CNOT entangling block.

Entangling the qubits makes the feature map sensitive to nonlinear feature inter-
actions rather than just the features one by one.

The $R_Z(x_1 \cdot x_2)$ gate, sandwiched between CNOTs, is exactly the ZZ interaction fr
Dive 1 (QAOA). But here, instead of encoding a graph edge cost, it encodes a *nonlinear fe

> **Design principle:** The expressiveness of the feature map comes from *entanglement*


## Computing the kernel value

The kernel between two data points $x$ and $x'$ is:

$$K(x, x') = |\langle \phi(x') | \phi(x) \rangle|^2$$

This is the *squared overlap* between the two quantum states; the probability of measurin

### The kernel circuit

To compute $K(x, x')$:

1. Apply $U_\phi(x)$; encode data point $x$
2. Apply $U_\phi(x')^\dagger$; the *adjoint* (reverse order, negate all angles)
3. Measure; the probability of getting $|00\ldots0\rangle$ is $K(x, x')$

```{figure} ../figures/kernel-circuit.png
:name: fig-kernel-circuit
:alt: Kernel circuit with U(x), then the adjoint U(x') dagger, followed by measurement.
```

```
Kernel evaluation is just an overlap test: prepare one encoded state, undo the other, the
```

Run this N_{shots} times. Count how often you get $|00\rangle$. That fraction is $K(x, x')$.

If $x = x'$: the adjoint perfectly undoes the encoding → you always get $|00\rangle$ → $K(x, x) = 1$.

If x and x' are very different: the states don't overlap much → $|00\rangle$ is rare → $K(x, x') \approx 0$.

Building the kernel matrix and training the SVM

For m training points, compute all m^2 pairwise kernel values (using the circuit above for each pair). This gives the **kernel matrix** K_{ij}. Then train a classical SVM with this precomputed kernel; `SVC(kernel='precomputed')` in scikit-learn.

The quantum computer is only used for kernel evaluation. The SVM training, prediction, and cross-validation (testing the trained model on held-out data to estimate real-world accuracy) are entirely classical. This clean separation means quantum ML inherits all the theory and tooling of classical kernel methods.

The companion notebook trains a quantum kernel SVM on a synthetic dataset end-to-end — defining the feature map, computing the kernel matrix, training the SVM, and comparing with a classical baseline.

→ See notebook `04-machine-learning.ipynb` for the runnable version.

When does this actually help?

The honest answer: we don't know for natural datasets.

Provable advantage exists for constructed problems (Huang et al. 2022): classification tasks designed so that the quantum kernel captures structure that no classical kernel can. These proofs are real but the tasks are artificial.

For natural data, the question reduces to: does the data have structure that lives naturally in the entangled quantum feature space? If the data is essentially low-dimensional (as most real datasets are), a classical kernel might capture the same structure with fewer resources.

The data loading bottleneck: encoding d-dimensional classical data requires $O(d)$ gates per data point. For $d = 10^6$ (common in practice), the circuit is too deep for near-term hardware.

The most promising near-term application may be **quantum data**; data that is already quantum (e.g., outputs of quantum experiments or quantum simulations). For such data, classical kernels can't access the relevant feature space at all.

What you should take away

1. **Quantum feature maps encode data as quantum states.** The feature space is the Hilbert space of n qubits; exponentially large and natively available.

2. **The kernel is a measurement probability.** $K(x, x') = \Pr(|0\rangle^n)$ after applying $U_\phi(x)$ then $U_\phi(x')^\dagger$. The Born rule is the kernel evaluator.
3. **Entanglement is the source of power.** Without entanglement, quantum feature maps reduce to classical ones. The CNOT + R_Z interaction (the ZZ gate from Deep-Dive 1) is reused here for a completely different purpose.
4. **The quantum computer computes kernel values; everything else is classical.** SVM training, prediction, and interpretation use standard classical tools.
5. **Advantage is not guaranteed.** Quantum kernels provably help for some problems, but whether natural datasets benefit is an open question.

Unit 5: Finance

Pricing the Unpriceable

The Hook

In 2008, the financial crisis taught the world that mispriced **derivatives** — financial contracts whose value derives from an underlying asset like a stock, a bond, or an interest rate — can destroy economies. At the heart of the crisis were exotic options: contracts that give the holder the right to buy or sell an asset at a fixed price on a future date, but with complex conditions that make them hard to value.

Pricing a derivative means answering one question: **what is the expected payoff** — the profit (if any) the contract delivers at maturity? For a simple **European call option** (the right to buy a stock at a fixed price, exercisable only at maturity), there's an analytical formula (Black-Scholes, 1973). For exotic options — path-dependent, multi-asset, with early exercise features — there is no formula. The standard method is **Monte Carlo simulation**: generate many random market scenarios, compute the payoff for each, and take the average.

Monte Carlo works, but it's slow. The statistical error (the uncertainty in your price estimate) shrinks as $1/\sqrt{N}$ where N is the number of samples. To get one more digit of accuracy, you need **100 times more samples**. For complex derivatives with expensive path simulation, this translates to substantial overnight compute budgets across large portfolios.

The $1/\sqrt{N}$ convergence rate isn't a software limitation. For generic random sampling — estimating the mean of an arbitrary distribution from independent samples — you cannot do better than $1/\sqrt{N}$. (Structured classical techniques like variance reduction and quasi-Monte Carlo can improve performance for specific problems, but they don't change the fundamental scaling.)

Unless you're quantum.

The Bottleneck

The mathematical structure is clean. You have a **random variable** X — a quantity whose value depends on a random outcome; here, the derivative's payoff under a randomly sampled market scenario — with expected value $\mu = \mathbb{E}[X]$ (the probability-weighted average of all possible outcomes). You want to estimate μ.

Classical Monte Carlo: draw N independent samples $X_1, \dots, X_N$ and compute the sample mean $\bar{X} = \frac{1}{N}\sum_i X_i$. The error in this estimate shrinks as:

$$|\bar{X} - \mu| \sim \frac{\sigma}{\sqrt{N}}$$

where σ is the **standard deviation** (a measure of how spread out the payoff values are). To halve the error, quadruple N. To gain a factor of 10 in precision, multiply

N by 100. This $1/\sqrt{N}$ convergence rate is a mathematical fact about generic sampling, not a software limitation — for any estimator based on independent classical samples from an arbitrary distribution, you cannot do better.

For a normalised problem targeting additive accuracy around 10^{-6} with $\sigma \sim 1$: you need $N \sim 10^{12}$ samples. At 1 microsecond per sample, that's about 12 days.

The Quantum Angle

Amplitude amplification: Grover as a subroutine

Before we get to financial pricing, we need the tool that makes it possible: **amplitude amplification**, which is Grover's search algorithm generalised.

Here's the idea. Imagine you have a bag of N balls, M of which are gold. Classical approach: draw balls one at a time, check each one. On average you need N/M draws to find a gold ball.

Grover's algorithm (1996) does this quantumly with only $\sqrt{N/M}$ queries — a quadratic speedup. It works not by checking balls faster, but by rotating the quantum state. Think of it geometrically: the quantum state starts pointing mostly toward "non-gold" and Grover's algorithm rotates it, step by step, toward "gold." Each step rotates by a fixed angle 2θ, where $\sin\theta = \sqrt{M/N}$. After about $\pi/(4\theta)$ rotations, the state points almost entirely at "gold." Measure, and you get a gold ball with near-certainty.

The rotation is implemented by the **Grover iterator** G, which applies two reflections that together produce a rotation. Deep-Dive 5 builds G from gates; here we just need the geometric picture.

From search to estimation

Now the key insight. The rotation angle θ itself encodes the answer to a different question: what fraction of balls are gold? Since $\sin\theta = \sqrt{M/N}$, measuring θ precisely gives you M/N — without ever finding a specific gold ball.

Quantum Amplitude Estimation (QAE) measures θ. It uses the same controlled-powers-plus-inverse-QFT pattern we saw in Unit 2 (Shor's algorithm) — that pattern is called **Quantum Phase Estimation** (QPE), and it works for any unitary operator. Here we apply it to the Grover iterator G, whose eigenvalues are $e^{\pm 2i\theta}$, and QPE extracts θ with precision $O(1/N_{\text{queries}})$ — that's $1/N$, not $1/\sqrt{N}$.

Applying QAE to Monte Carlo

For derivative pricing, the "gold balls" are market scenarios, and the "fraction of gold" is generalised to the expected payoff. Specifically:

1. **Encode** the probability distribution as a quantum state: prepare $\sum_x \sqrt{p(x)}|x\rangle$
2. **Encode the payoff** into an ancilla amplitude: for each price state $|x\rangle$, rotate an ancilla qubit so that its amplitude encodes the (normalised) payoff $f(x)$. The combined state becomes $\sum_x \sqrt{p(x)}|x\rangle(\sqrt{1-f(x)}|0\rangle + \sqrt{f(x)}|1\rangle)$

3. **Run QAE**: estimate the probability of measuring the ancilla in $|1\rangle$. This probability is $\sum_x p(x) \cdot f(x) = \mathbb{E}[f(X)]$ — the normalised expected payoff. Multiply by the payoff bound $P_{\max}$ to recover the actual expected payoff.
4. Precision ϵ requires $O(1/\epsilon)$ queries to the oracle

Classical Monte Carlo: precision ϵ requires $O(1/\epsilon^2)$ samples. Canonical, phase-estimation-based quantum amplitude estimation: precision ϵ requires $O(1/\epsilon)$ oracle uses.

Same target accuracy, quadratically fewer oracle calls. For the normalised back-of-the-envelope above, 10^{12} classical samples becomes about 10^6 quantum oracle uses. That is the asymptotic gain. Whether it becomes a wall-clock advantage depends on the cost of state preparation, payoff encoding, and fault tolerance.

“‘{figure} ../figures/qae-scaling.png :name: fig-qae-scaling :alt: Scaling comparison showing classical Monte Carlo with 1 over square root N convergence and quantum amplitude estimation with 1 over N convergence.

Monte Carlo is already useful, so the point of QAE is not a new problem class but a quadratic reduction in the number of samples needed for the same precision.

```
### When does $\sqrt{}$ matter?

A quadratic speedup sounds modest compared to Shor's exponential speedup. But for Monte C
complexity gain exists; it does. The open question is whether a full fault-
tolerant pricing oracle can realise it before implementation overhead erases the advanta


## Worked Example

Price a European call option classically, then use a compiled **toy amplitude-
estimation phase readout** for a discretised exercise-probability proxy. Compare the cla

Setup:
- Stock price $S_0 = 100$, strike price $K = 105$ (the fixed price at which the option can
how wildly the price swings; this is the same $\sigma$ from the Bottleneck, now applied s
free rate $r = 5\%$ (the return on a riskless investment, like a government bond)
- Payoff: $\max(S_T - K, 0)$
- Black-Scholes analytical price: ~$8.02

We discretise the stock price into $2^n$ bins, encode the **log-
normal distribution** (stock prices are modelled so that their logarithm follows a norma
K, 0) / P_{\max}$ into its amplitude. QAE then estimates the normalised expected payoff;

(The companion notebook keeps the pricing pipeline honest by splitting the problem in two
Scholes and Monte Carlo for the actual option price, then a compiled toy amplitude-
estimation circuit for the **discretised exercise probability**. It does **not** encode
encoding construction described above remains the standard approach for pricing continuo

→ *The next chapter builds the amplitude estimation circuit from Grover's algorithm, and

### Back to the trading floor
```

We priced a European call option — the simplest derivative there is. How does this help p

The pipeline is structurally similar. A more complex derivative has a more complex payof
dependent, multi-asset, with barriers and early exercise), and QAE still estimates the e
complexity level; practical advantage depends on the total cost of that oracle.

Reality Check

What's been demonstrated. Goldman Sachs and IBM published a series of papers (2019—
2021) on quantum amplitude estimation for derivative pricing, including simulator studi
scale: for example, Stamatopoulos et al. run a heavily discretised European-
call experiment on three qubits using amplitude estimation without phase estimation and

The depth problem. QAE requires quantum phase estimation, which requires deep circui
3}$), that's thousands of sequential operations, each involving the full oracle circuit.

Approximate QAE. Variants like iterative QAE and maximum-likelihood / no-
phase-estimation QAE reduce the circuit depth at the cost of more measurements. Suzuki e

What would change the picture. Fault-tolerant quantum computers able to run long coh

The quadratic wall. A quadratic speedup means quantum advantage arrives only above a
construction and error-correction overhead. The option-pricing literature gives evidenc
count crossover on simplified models, but it does **not** yet pin this down to a settled
stack threshold like "a few thousand logical qubits."

What's real today: The algorithm is sound and the quadratic speedup is mathematicall
tolerant overhead remains open.

Chef's Notes

- **Grover's algorithm appears here, not as a standalone topic.** Most books introduce G

- **The quadratic speedup is the most common quantum advantage.** Exponential speedups (

- **Connection to Unit 2.** QAE uses QPE, which uses the QFT; the same machinery from Sho

- **Further reading:**
  - Montanaro (2015). *Quantum speedup of Monte Carlo methods.* [arXiv:1504.06987](htt
  - Stamatopoulos, Egger, Sun, et al. (2020). *Option pricing using quantum computers.*
2020-07-06-291)
  - Suzuki, Uno, Raymond, et al. (2020). *Amplitude estimation without phase estimation
019-2565-2)
  - Grover (1996). *A fast quantum mechanical algorithm for database search.* [arXiv:qu
ph/9605043](https://arxiv.org/abs/quant-ph/9605043)
  - 3Blue1Brown (2023). [*Grover's algorithm visually explained*](https://www.youtube

Deep-Dive 5: Amplitude Estimation from Grover

_This deep dive pairs with Unit 5 (Finance), which explained why Monte Carlo pricing is s

In This Chapter

- **What you'll learn:** How Grover's algorithm works geometrically, how quantum amplitu
- **What you need:** From Deep-Dive 2, you know the controlled-powers-plus-inverse-QFT pattern that extracts eigenvalues from a unitary operator (that pattern is c Dive 1, you know the ZZ gate and the variational loop. Here we apply QPE to a new operator the Grover iterator — and the eigenvalue it extracts encodes a probability rather than a
- **Runnable version:** The companion notebook [`05-finance.ipynb`](../notebooks/05-finance.ipynb) prices the option classically, then runs a compiled toy amplitude-estimation phase readout for a discretised exercise-probability proxy on a cloud Quokka.

Grover's algorithm: the geometric picture

The search problem

You have a function $f:\{0,1\}^n \to \{0,1\}$ with M "marked" inputs (where $f(x) = 1$)

Classically: $O(N/M)$ queries (check inputs one by one). Quantumly: $O(\sqrt{N/M})$ que

Two subspaces

Define two states:

$$|\text{good}\rangle = \frac{1}{\sqrt{M}} \sum_{x: f(x)=1} |x\rangle, \quad |\text{bad
M}} \sum_{x: f(x)=0} |x\rangle$$

The initial uniform superposition lies in the plane spanned by these two states:

$$|s\rangle = \sin\theta\,|\text{good}\rangle + \cos\theta\,|\text{bad}\rangle$$

where $\sin\theta = \sqrt{M/N}$. For $M \ll N$ (a needle in a haystack), θ is smal

The Grover iterator as a rotation

The Grover iterator G consists of two reflections:

1. **Oracle reflection** S_f: flip the phase of marked states. $S_f|x\rangle = (-1)^{f(x)}|x\rangle$. This is phase kickback from Deep-Dive 2; the oracle marks solutions

2. **Diffusion** $S_0 = 2|s\rangle\langle s| - I$: reflect about the mean amplitude. Here it produces an operator (a matrix), not a number. Applied to any state $|\psi\rangle$, it the row-vector partner of the ket $|s\rangle$, obtained by taking the adjoint.) So $2|s\ I$ doubles the component along $|s\rangle$ and subtracts the original — a reflection about $|s\rangle$. In circuit terms: $H^{\otimes n} \cdot (2|0\rangle\langl I) \cdot H^{\otimes n}$.

Two reflections make a rotation. $G = S_0 \cdot S_f$ rotates the state by angle 2θ

```
$$G^k|s\rangle = \sin((2k+1)\theta)\,|\text{good}\rangle + \cos((2k+1)\theta)\,|\text

After $k_\text{opt} = \lfloor \pi / (4\theta) \rfloor$ iterations, the state is nearly $
O(M/N)$.

> **The key insight:** Grover doesn't search. It *rotates*. The search space isn't explo

### The oracle circuit

For the financial application (Unit 5), the full construction encodes the normalised pay
f(x)}|0\rangle + \sqrt{f(x)}|1\rangle$, where $f(x) = \max(\text{price}(x) -
K, 0) / P_{\max}$ is the normalised payoff. The probability of measuring the ancilla in $
the normalised expected payoff — which is rescaled by $P_{\max}$ to give the option price

The companion notebook demonstrates a simplified version using discretised price bins ar
eigenphase toy. It estimates the exercise probability rather than the full expected payo

### The diffusion circuit

The diffusion operator $2|s\rangle\langle s| - I$ is implemented as:

```{figure} ../figures/grover-diffusion-circuit.png
:name: fig-grover-diffusion-circuit
:alt: Grover diffusion circuit for four qubits.

The diffusion block is the reusable heart of Grover-style algorithms: reflect about the r
```

The pattern is: Hadamard all qubits, flip all qubits (X), apply a **multi-controlled** Z (which flips the phase only when all qubits are $|1\rangle$ — decomposed as H + multi-controlled X + H on the last qubit), then undo the X and H layers. This is the most gate-intensive part of Grover's algorithm, but it's a fixed overhead per iteration.

From Grover to amplitude estimation

The connection

Grover's algorithm uses G to find marked items. But the angle θ; which determines how many iterations are needed; also encodes the fraction of marked items: $\sin^2\theta = M/N$.

Quantum Amplitude Estimation (QAE) extracts θ directly, without finding any specific marked item. This is exactly what we need for Monte Carlo pricing: we don't want a specific market scenario, we want the expected payoff across all scenarios.

QPE on the Grover operator

In Deep-Dive 2, we used a pattern — controlled powers of a unitary, followed by the inverse QFT — to extract the period of $a^x \bmod N$. That pattern is **Quantum Phase Estimation** (QPE), and it works for any unitary operator: if $U|v\rangle = e^{i\phi}|v\rangle$, QPE extracts ϕ.
```

Here we apply QPE to the Grover iterator G, which has eigenvalues $e^{\pm 2i\theta}$. The recipe is identical:

1. Prepare ancilla qubits in $|+\rangle$ (via Hadamard)
2. Apply controlled-G^{2^k} operations (controlled by ancilla qubit k)
3. Apply inverse QFT to the ancillas
4. Measure → get an estimate of θ

From θ: compute $\tilde{a} = \sin^2\theta \approx M/N$. This is the amplitude (probability) we want.

The convergence advantage

With m ancilla qubits, QPE estimates θ to precision $O(1/2^m)$. This requires $O(2^m)$ applications of the Grover operator G.

Each application of G is one "query" to the oracle. So:

- **Precision** ϵ requires $O(1/\epsilon)$ queries
- **Classical Monte Carlo** requires $O(1/\epsilon^2)$ samples for the same precision

The quadratic advantage: same target accuracy, quadratically fewer oracle queries.

Target precision	Classical samples	Quantum queries	Speedup
10^{-1}	100	10	10×
10^{-3}	10^6	10^3	1,000×
10^{-6}	10^{12}	10^6	10^6×

These are scaling-law counts, suppressing constant factors and assuming coherent oracle access.

The companion notebook does **not** run full payoff-encoding amplitude estimation end-to-end. Instead, it keeps the real pricing work classical and runs a compiled toy QAE phase-readout circuit for the discretised exercise probability, which is the narrowest honest runnable version on the current setup.

→ **See notebook `05-finance.ipynb` for the runnable version.**

What you should take away

1. **Grover's algorithm is a rotation, not a search.** It rotates the quantum state from "mostly wrong" to "mostly right" in the 2D good/bad plane. The number of rotations is $O(\sqrt{N/M})$.
2. **Amplitude estimation = QPE on Grover.** The angle θ that controls Grover encodes the probability $\sin^2\theta = M/N$. QPE extracts θ with precision $O(1/2^m)$ using $O(2^m)$ queries.
3. **The quadratic speedup is $1/\epsilon$ vs. $1/\epsilon^2$.** This is a query-complexity statement. Whether it becomes a wall-clock gain depends on the cost of building and repeating the full pricing oracle.

4. **Phase kickback appears again.** The oracle uses it (Deep-Dive 2). The Grover iterator uses it. QPE uses it. It's the same mechanism at every level.
5. **The circuit depth is the bottleneck.** QAE requires $O(1/\epsilon)$ sequential applications of the full Grover operator. Each application includes the oracle circuit. For useful precision, this means deep circuits; a fault-tolerant algorithm.

Unit 6: Supply Chains

10,000 Nurses, 50 Hospitals, 1 Schedule

The Hook

Every winter, Britain's National Health Service enters a scheduling crisis. Across 50 hospitals, 10,000 nurses need to be assigned to shifts; day, evening, night; across wards, specialties, and locations. Each nurse has qualifications (ICU-certified? paediatrics-trained?), contractual constraints (maximum hours, minimum rest), preferences (no nights, school pickup at 3 PM), and legal requirements (EU Working Time Directive: no more than 48 hours per week, 11 hours rest between shifts).

The NHS spends an estimated **£3 billion per year** on agency nurses; temporary staff hired at premium rates because the permanent workforce can't be scheduled efficiently. Not because there aren't enough nurses (though there often aren't), but because the scheduling problem is so complex that planners routinely produce suboptimal rosters, leaving gaps that agencies fill at 2–3× the cost.

This isn't a peculiarly British problem. Airlines schedule crews across thousands of flights. Manufacturing plants assign workers to production lines. Call centres staff agents across time zones. In every case, the mathematical structure is the same: assign N resources to M slots subject to constraints, minimising cost (or maximising fairness, or minimising overtime).

These are **constraint satisfaction problems**, and they are NP-hard. The nurse scheduling problem (NSP) was proven NP-hard by Osogami and Imai in 2000. No polynomial-time algorithm exists (unless P = NP). Classical solvers; integer linear programming (ILP), constraint programming (CP), simulated annealing; can handle instances with hundreds of nurses. At thousands, they struggle. At tens of thousands, they break.

The Bottleneck

The nurse scheduling problem has a distinctive mathematical structure. It's not a pure optimisation problem (like MaxCut); it's a **constrained** optimisation problem, with both hard constraints (must satisfy) and soft constraints (prefer to satisfy).

Hard constraints: - Each shift must be staffed by a qualified nurse - No nurse works more than 48 hours/week - Minimum 11 hours between consecutive shifts - No nurse works more than 5 consecutive days

Soft constraints (with penalties): - Honour nurse preferences (nights, weekends) - Distribute unpopular shifts fairly - Minimise travel between hospital sites - Balance workload across the team

The search space is enormous. With N nurses and S shifts, there are N^S possible assignments (ignoring constraints). For $N = 100$ nurses and $S = 500$ shifts across a week, that's $100^{500} \approx 10^{1000}$; absurdly beyond brute force.

The real difficulty is the **interaction structure**. Constraints are non-local: assigning Nurse A to Monday night affects whether Nurse B can take Tuesday morning (if they share qualifications and wards). These interactions create a web of dependencies that classical local-search methods struggle with; improving one part of the schedule can break another.

The Quantum Angle

QUBO: turning constraints into energy

The key idea is **QUBO** — **Q**uadratic **U**nconstrained **B**inary **O**ptimisation. Each word matters: quadratic (the cost function involves at most pairs of variables), unconstrained (constraints are folded into penalty terms), binary (every variable is 0 or 1), optimisation (find the assignment that minimises the total cost). We convert the constrained scheduling problem into an unconstrained minimisation of a quadratic function over binary variables, then map that to the ground state of a quantum Hamiltonian.

Each binary decision gets a variable: $x_{n,s} = 1$ if nurse n is assigned to shift s, 0 otherwise. The total number of binary variables is $N \times S$.

Hard constraints become **penalty terms**: add a large positive cost for any assignment that violates them. Soft constraints become **smaller penalties**, proportional to how undesirable the violation is.

The result is a cost function:

$$C(x) = \sum_i \sum_j Q_{ij} x_i x_j + \sum_i c_i x_i$$

where Q_{ij} encodes the interactions between binary decision variables and c_i encodes the individual preferences. This is the QUBO form. The optimal schedule is the binary string x^* that minimises $C(x)$.

From QUBO to Ising Hamiltonian

The substitution $x_i = \frac{1-Z_i}{2}$ converts each binary variable (0 or 1) to a spin variable ($+1$ or -1). The QUBO cost function becomes an **Ising Hamiltonian**:

$$H = \sum_{i<j} J_{ij} Z_i Z_j + \sum_i h_i Z_i + \text{const}$$

The ground state of this Hamiltonian is the optimal schedule. This is the same pattern as Unit 1 (MaxCut → cost Hamiltonian) and Unit 3 (molecule → molecular Hamiltonian); different problem, same mathematical structure.

Quantum annealing

There are two quantum approaches to finding the ground state of an Ising Hamiltonian:

Gate-based QAOA (Unit 1): apply alternating problem and mixer unitaries, optimise parameters variationally. Works on universal quantum computers.

Quantum annealing: a different paradigm entirely. Start in the ground state of a simple Hamiltonian H_{init} (whose ground state you know; typically all qubits in $|+\rangle$). Slowly transform H_{init} into the problem Hamiltonian H_{problem}:

$$H(t) = \left(1 - \frac{t}{T}\right) H_{\text{init}} + \frac{t}{T} H_{\text{problem}}$$

The **adiabatic theorem** guarantees: if you change the Hamiltonian slowly enough (large T), the system stays in the ground state throughout. The intuition is physical: imagine a marble sitting at the bottom of a bowl. If you tilt the bowl slowly, the marble rolls smoothly to the new lowest point. Tilt it too fast, and the marble sloshes up the sides and doesn't settle. The quantum system behaves the same way — slow changes let it continuously adapt to the shifting energy landscape. At the end ($t = T$), you're in the ground state of H_{problem} — the optimal schedule.

Why quantum annealing might help

Classical **simulated annealing** explores the energy landscape by making random local moves — flip one variable and check if the cost improves. It occasionally accepts worse moves (uphill steps) with a probability that decreases over time, mimicking a cooling process. This randomness helps escape local minima, but the method gets stuck when the optimal solution is separated from the current state by a tall energy barrier — it has to go over the barrier.

Quantum annealing can tunnel through barriers. Quantum tunnelling; the same effect that allows radioactive decay; lets the quantum state traverse energy barriers without climbing them. Barriers that are tall but narrow are easy to tunnel through; barriers that are tall and wide are hard.

This gives quantum annealing a structural advantage for problems with specific landscape properties: many tall, narrow barriers. Whether real-world scheduling problems have this property is an empirical question; and a contested one.

"'{figure} ../figures/qubo-landscape.png :name: fig-qubo-landscape :alt: A rugged scheduling cost landscape with a local minimum, a better global minimum, and a narrow barrier between them.

QUBO penalties create basins and barriers; annealing is only attractive when the barrier blocking the better schedule is tall but narrow enough that tunnelling could matter.

```
## Worked Example

To keep the runnable circuit honest and readable, the companion notebook uses a **2-
nurse, 2-shift micro-example**:

- Nurse A and Nurse B must cover Mon-day and Mon-night
- Hard constraint: Nurse A must take exactly one of the two shifts, so Nurse B takes the o
- Soft constraint: Nurse A prefers the day shift, so the night assignment carries a small
```

This gives a two-variable QUBO with binary variables x_0 and x_1, where $x_0 = 1$ mea
day and $x_1 = 1$ means Nurse A takes Mon-night.

Building the penalties

The hard constraint is "Nurse A takes exactly one of the two shifts." The penalty is:

$$P \cdot (x_0 + x_1 - 1)^2$$

If exactly one shift is assigned to Nurse A, the penalty is zero. If Nurse A takes neither
example cost becomes:

$$C(x_0, x_1) = P \cdot (x_0 + x_1 - 1)^2 + w x_1$$

The notebook writes this cost exactly, converts it to a two-
qubit Ising Hamiltonian, and checks all four assignments classically before running QAOA

What QAOA produces

The classical optimum is the day-shift assignment for Nurse A. A classically tuned depth-
1 QAOA circuit on the same two-qubit Ising Hamiltonian puts almost all of its probability
cost day assignment. The point is not performance — the micro-
example is trivial classically — but transparency: the penalties, Ising map, and circuit

Back to the hospital

We started with the NHS spending £3 billion a year on agency nurses because schedules are
nurse micro-example is tiny on purpose, but the algebraic pattern is the same one used in
state search problem. A real hospital with 200 nurses and 50 shift types would still use

→ *The next chapter shows how to formulate constraints as QUBO penalties and build the an
example scale.*

Reality Check

D-Wave. The company most associated with quantum annealing, D-
Wave has built machines with over 5,000 qubits; far more than any gate-
based machine. Their Advantage system can natively solve QUBO problems with thousands of

The speedup debate. Despite two decades of effort, no definitive quantum speedup for
purpose speedup remains unproven.

Hybrid solvers. D-Wave's most practically useful tool is its **hybrid classical-
quantum solver** (Leap), which uses classical heuristics for most of the work and the qua

Gate-based vs. annealing. QAOA on gate-based hardware offers more flexibility (arbi

What would change the picture. A demonstration of quantum speedup on a real schedulin
not a crafted benchmark instance. This would require either significant improvements in
based machines with $O(10^3)$ logical qubits.

What's real today: Quantum annealing hardware exists at scale (5,000+ qubits) but ha
based QAOA works in principle but is limited to toy sizes. The hybrid approach —
classical solvers augmented by quantum sub-routines — is the most pragmatic near-
term path, but attributing advantage to the quantum component remains difficult.

Chef's Notes

- **QUBO is the universal input format for constrained optimisation on quantum computers
Wave Ocean SDK has a library of pre-built QUBO templates for common problems.

- **The QUBO-to-Ising map is the bridge between classical OR and quantum computing.** If

- **Quantum tunnelling vs. thermal hopping.** Simulated annealing hops over barriers usi
narrow barriers favour tunnelling; short-wide barriers favour hopping.

- **Connection to Unit 1.** QAOA and annealing are two paradigms for the same goal: findi

- **Further reading:**
  - Lucas (2014). *Ising formulations of many NP problems.* [Frontiers in Physics 2:5](h
  - Farhi et al. (2001). *A quantum adiabatic evolution algorithm applied to random inst
complete problem.* [Science 292:472–475](https://doi.org/10.1126/science.1057726) ([ar
ph/0001106](https://arxiv.org/abs/quant-ph/0001106))
  - Rønnow et al. (2014). *Defining and detecting quantum speedup.* [Science 345:420–
424](https://doi.org/10.1126/science.1252319) ([arXiv:1401.2910](https://arxiv.org/ab
  - King et al. (2023). *Quantum critical dynamics in a 5,000-
qubit programmable spin glass.* [Nature 617:61–66](https://doi.org/10.1038/s41586-
023-05867-2) ([arXiv:2207.13800](https://arxiv.org/abs/2207.13800))
  - Katzgraber, Hamze & Andrist (2014). *Glued trees are not hard for quantum annealing:
  - Katzgraber, Trebst, Huse & Troyer (2006). *Feedback-optimized parallel tempering Mo
5468/2006/03/P03018) ([arXiv:cond-mat/0602085](https://arxiv.org/abs/cond-
mat/0602085))
  - Mandrà, Katzgraber & Thomas (2017). *The pitfalls of planar spin-
glass benchmarks.* [Quantum Science and Technology 2:038501](https://doi.org/10.1088/2
9565/aa7877) ([arXiv:1703.00622](https://arxiv.org/abs/1703.00622))

Deep-Dive 6: QUBO Engineering

_This deep dive pairs with Unit 6 (Supply Chains), which introduced nurse scheduling and
world constraints as QUBO problems and solve them on quantum hardware._

In This Chapter

- **What you'll learn:** How to turn constraints into penalty terms, convert QUBO to an I
- **What you need:** From Deep-Dive 1, you know cost Hamiltonians and the ZZ gate. From

- **Runnable version:** The companion notebook [`06-supply-chains.ipynb`](../notebooks
supply-chains.ipynb) solves a 2-nurse, 2-shift scheduling QUBO micro-
example on a cloud Quokka.

From constraints to energy

Real problems have rules

In Deep-Dive 1 (QAOA), our cost function was simple: count the cut edges. Every binary st
there were no constraints, just better and worse solutions. Flip any node's colour, and y

Scheduling isn't like that. "Assign two nurses to the same shift" isn't a bad schedule —
it's an *invalid* schedule. Real problems come with rules, and breaking a rule isn't just

QUBO (Quadratic Unconstrained Binary Optimisation) handles this by a trick you've al
large positive costs added whenever a rule is broken. Preferences become small costs. The
energy state" — exactly the ground-state search from Unit 1. A solution with zero penalty

Encoding equality constraints

"Each shift must have exactly one nurse assigned."

Binary variables: $x_{n,s} = 1$ if nurse n takes shift s. The constraint for shift $s

$$\sum_n x_{n,s} = 1$$

Convert to penalty: $P \cdot \left(\sum_n x_{n,s} - 1\right)^2$ where P is large.

Expand (using $x_i^2 = x_i$ for binary variables):

$$P \cdot \left(\sum_n x_{n,s}^2 + 2\sum_{n < n'} x_{n,s} x_{n',s} -
2\sum_n x_{n,s} + 1\right) = P \cdot \left(-\sum_n x_{n,s} + 2\sum_{n < n'} x_{n,s} x_{n'

This is quadratic in the binary variables; it fits QUBO form. The minimum (zero penalty)

Encoding inequality constraints

"No nurse works more than 3 shifts."

Equality constraints were clean: square the violation, get a quadratic penalty. Inequali
any number from 0 to 3 is fine.

The simplest approach: **penalise every way the constraint can be violated.** If nurse $n

$$P \cdot \sum_{s_1 < s_2 < s_3 < s_4} x_{n,s_1} \, x_{n,s_2} \, x_{n,s_3} \, x_{n,s_4}$$

This penalty is zero when the nurse works 3 or fewer shifts (no group of 4 exists), and po
but there's a problem: it's a product of *four* variables. QUBO only allows products of *

```
There are standard tricks to reduce higher-order terms to quadratic form (introducing ex
Wave's `dwavebinarycsp` handle the reduction automatically.

The honest takeaway: equality constraints map to QUBO naturally. Inequality constraints
choosing the right penalty structure, managing the extra variables —
is where practical QUBO formulation gets interesting. It's part of why operations resear

> **The QUBO art:** simple constraints (equalities, pairwise exclusions) map cleanly to

### Soft constraints as linear costs

**"Nurse A prefers day shifts."**

Add a small cost for night assignments: $w \cdot x_{A,\text{night}}$ where $w \ll P$. The
constraint penalty $P$ is much larger than the preference cost $w$).

This priority structure; hard constraints as large penalties, soft preferences as small


## From QUBO to Ising Hamiltonian

Once you have a QUBO cost function, turning it into a Hamiltonian is mechanical —
and it lands you right back in familiar territory. Substitute $x_i = \frac{1 -
Z_i}{2}$ (so $x_i = 0$ when qubit $i$ is $|0\rangle$ and $x_i = 1$ when it's $|1\rangle$)

$$x_i x_j = \frac{(1 - Z_i)(1 - Z_j)}{4} = \frac{1 - Z_i - Z_j + Z_i Z_j}{4}$$

Every QUBO term becomes a combination of:
- $Z_i Z_j$ terms (the same interactions as MaxCut in Unit 1)
- $Z_i$ terms (local fields)
- Constants (global energy offset)

The resulting Ising Hamiltonian $H = \sum_{ij} J_{ij} Z_i Z_j + \sum_i h_i Z_i + \text{co
Dive 1. Same gates, same structure, just more terms.

### Same gates, bigger circuit

The QAOA circuit for a QUBO problem uses the same building blocks as Deep-
Dive 1:
- $ZZ$ gates (CNOT sandwich) for the $J_{ij}$ terms
- $R_Z$ rotations for the $h_i$ terms
- $R_X$ mixer rotations

The only difference from Deep-Dive 1 is scale: a QUBO with $n$ variables and $O(n^2)$ int


## Quantum annealing: a different paradigm

### The adiabatic approach

QAOA (Deep-Dive 1) uses gate-based quantum computing: discrete operations applied in sec
```

Start in the ground state of a simple Hamiltonian H_0 (typically a **transverse field*
an X operator on every qubit — whose ground state is $|+\rangle^n$, which we already kn
Dive 1). Slowly interpolate to the problem Hamiltonian H_1:

$$H(t) = \left(1 - \frac{t}{T}\right) H_0 + \frac{t}{T} H_1$$

The **adiabatic theorem** says: if the interpolation is slow enough, the system stays in

How slow is slow enough?

"Slow enough" means $T \geq O(1/\Delta_\text{min}^2)$ where Δ_min is the m

For easy problems, Δ_min is large and the anneal is fast. For hard problems

The open question: for which problems is the spectral gap large enough that annealing is

Quantum tunnelling: the potential advantage

Classical simulated annealing gets stuck at energy barriers; to escape a local minimum, i

Quantum annealing can **tunnel through** barriers. Quantum tunnelling; the same physics

For problems with many tall, narrow barriers: quantum annealing may explore the energy la

> **The honest picture:** Despite two decades of effort, definitive quantum speedup for a
Wave's quantum annealers (5,000+ qubits) solve some crafted instances faster than classi
purpose advantage has not been demonstrated. This is an active research frontier.

What you should take away

1. **QUBO is the universal language for constrained binary optimisation.** Any problem w

2. **QUBO → Ising is mechanical.** The substitution $x_i = (1-Z_i)/2$ converts any QUBO to an Ising Hamiltonian that can be solved with QAOA or anneali

3. **The circuit is the same as Deep-Dive 1, just bigger.** More variables → more qubits.

4. **Annealing is a different paradigm.** Continuous time evolution instead of discrete

5. **Penalty weight matters.** P must be large enough that violating a hard constraint

Unit 7: Materials Science — *The Room-Temperature Superconductor Problem*

The Hook

A room-temperature superconductor would be one of the most transformative discoveries in
temperature" superconductors, below −250°C for conventional ones). The cooling infrastr
intensive.

```
If you could superconduct at room temperature, you could:
- **Transmit electricity with zero loss.** The U.S. electrical grid loses about 5% of ger
- **Build MRI machines without liquid helium.** A hospital MRI costs $1–
3M, largely because of the superconducting magnets that need continuous helium cooling.
- **Enable practical fusion reactors.** Tokamak fusion requires superconducting magnets
temperature superconductors would simplify the engineering dramatically.
- **Revolutionise computing.** Superconducting circuits (already the basis of most quan

The problem is that we can't *predict* which materials will superconduct at high tempera
quantum systems where the interactions between electrons are so strong that the **mean-
field** approximation (treating each electron as if it moves in the average field of all

The **Hubbard model** — the simplest model that captures this physics —
has been studied for over 60 years. Its full phase diagram in two dimensions remains unse
accuracy methods disagree in key parameter regimes, and whether the model supports a robu
wave superconducting phase at intermediate coupling is still debated. The issue is not th


## The Bottleneck

### The Hubbard model

The Hubbard model describes electrons on a lattice with just two ingredients.

**Hopping.** An electron can jump from one lattice site to a neighbouring site. The ampli
Dive 3:

$$H_\text{hop} = -t \sum_{\langle i,j \rangle, \sigma} (c_{i\sigma}^\dagger c_{j\sigma}

Here $c_{i\sigma}^\dagger$ creates an electron with spin $\sigma$ (up or down) at site $i
it ensures electrons can hop in both directions.

**Repulsion.** Two electrons on the *same* site pay an energy cost $U$. The number operat
$\sigma$ electron (0 or 1), so:

$$H_\text{int} = U \sum_i n_{i\uparrow} n_{i\downarrow}$$

This term is zero unless both a spin-up and a spin-down electron occupy the same site, in

**The full Hamiltonian** is just these two terms combined:

$$H = H_\text{hop} + H_\text{int} = -t \sum_{\langle i,j \rangle, \sigma} (c_{i\sigma}^\

Two parameters, $t$ and $U$. The ratio $U/t$ controls the physics:

- **Small $U/t$:** electrons hop freely, metal
- **Large $U/t$:** electrons localise to avoid repulsion, insulator (Mott insulator)
- **Intermediate $U/t$:** neither picture works; strongly correlated regime; this is whe

### Why classical methods fail
```

The Hilbert space of N electrons on L sites with spin has dimension $\binom{2L}{N}$.

Density functional theory (DFT) — the workhorse of solid-state physics — approximates the many-electron problem by working with the electron dens field method: it handles weakly correlated materials well but *systematically fails* for

Quantum Monte Carlo (QMC) — the best classical method for many-body quantum systems — works for bosons but suffers the **sign problem** for fermions: th to-qubit encodings) causes Monte Carlo sampling to produce catastrophic cancellations, r hard in general.

Exact diagonalisation — exact but limited to ~20 sites. Physically relevant results

DMRG (Density Matrix Renormalization Group) — an approach that represents the wavefu

The 2D Hubbard model at intermediate U/t is in a computational no-man's land. Classical methods provide important but partial windows into the problem; no

The Quantum Angle

Quantum Phase Estimation (QPE)

In Unit 3 (Drug Discovery), we used VQE; a variational method that gives energy *upper bo

Quantum Phase Estimation (QPE) is the fault-tolerant alternative. It gives the *exac

QPE works like this:

1. **Prepare a trial state** $|\psi\rangle$ with non-zero overlap with the ground state
2. **Apply controlled time evolution** e^{-iHt} to the trial state, controlled by an ar
3. **Apply the inverse QFT** to the ancilla register
4. **Measure** the ancilla → the result is the energy eigenvalue (encoded as a binary fra

The key ingredient is step 2: implementing e^{-iHt}; the time evolution operator under

Hamiltonian simulation via Trotterisation

The Hamiltonian $H = H_\text{hop} + H_\text{int}$ is a sum of non-commuting terms. We can't implement e^{-iHt} directly, but we can approximate it using Suzuki decomposition**:

$$e^{-iHt} \approx \left(e^{-iH_\text{hop}\Delta t} \cdot e^{-iH_\text{int}\Delta t}\r

Each Trotter step breaks the evolution into pieces that are individually easy to compile network layers, so the circuit becomes a structured sequence rather than one opaque giant

Higher-order Trotter formulas reduce the error further. The encodings book (Chapter 15,

Resource estimation: how big does the quantum computer need to be?

```
This is the critical question. The honest answer is that there is no single settled numbe
tolerance stack. A useful summary from McArdle et al. (2020) for a 100-
site ($10 \times 10$) 2D Fermi-Hubbard model is:

| Literature estimate | Fault-tolerant scale |
|---|---|
| Babbush et al. (2018), qubitization with an intensive error target | about $7.1 \times
gates, roughly $2$-$3$ million physical qubits, and tens of hours under the surface-
code assumptions used in that work |
| Kivlichan et al. (2019), Trotterisation with a size-extensive error target | about $10^
$10^8$ T-gates, roughly $4 \times 10^5$-$6 \times 10^5$ physical qubits, and a couple of

(A **T-gate** is a specific rotation gate that, together with the Clifford gates, enables
gates are expensive in fault-tolerant computing because they require resource-
intensive distillation procedures, so the T-gate count is a standard cost metric.)

The takeaway is not that one paper has *the* answer. It is that scientifically interestin
model QPE lives in the hundreds-of-thousands-to-millions of physical qubits, not in toda
dependent order-of-magnitude estimates, not countdown clocks.


## Worked Example

Classically benchmark the **2-site Hubbard model** across $U/t \in [0, 10]$, then feed o
state energy into a compiled **3-bit QPE toy**. The sweep shows the **crossover from more

The 2-site Hubbard model with 2 electrons has a 6-dimensional Hilbert space, representab
Wigner encoding. Small enough to benchmark exactly, and still useful for showing the str

- At $U/t = 0$: the ground state is more delocalised
- At $U/t \gg 1$: the benchmark ground state becomes more localised, one electron per sit
- The notebook's QPE step is a **toy phase-readout demonstration**, not a full controlle
$e^{-iHt}$ Hubbard simulation

```{figure} ../figures/hubbard-crossover.png
:name: fig-hubbard-crossover
:alt: Ground-state energy of the two-site Hubbard model as a function of U over t.

Even the 2-site dimer shows the qualitative story: as $U/t$ grows, localisation wins ove
```

→ The next chapter builds QPE and the Trotter circuit from the components introduced in earlier deep dives, and the companion notebook shows the phase-readout part honestly on a compiled toy.

Reality Check

The 2D Hubbard model is one of the grand challenges of quantum simulation. It has been called the “standard model of condensed matter physics”; the simplest model that might explain high-temperature superconductivity in cuprate
```

materials. It is widely regarded as one of the leading candidates for an early scientifically meaningful quantum advantage in simulation.

What's been demonstrated. Small Hubbard models on the scale of a few sites have been simulated on quantum hardware using VQE-style methods. But these sizes are trivially classical; the value was in demonstrating the pipeline, not in producing new physics.

The VQE stopgap. Until fault-tolerant QPE is available, VQE on the Hubbard model is an active area of research. Problem-specific ansätze (like the Hamiltonian variational ansatz) can capture Hubbard physics better than generic hardware-efficient circuits. But VQE is fundamentally limited by measurement overhead and barren plateaus (exponentially flat optimisation landscapes) at scale.

What would change the picture. A fault-tolerant quantum computer capable of running QPE on a roughly 100-site 2D Hubbard model with controlled error bars. That would let us probe a regime that remains unresolved by today's controlled classical methods. Published estimates for that scale span roughly 4×10^5 physical qubits to a few million, depending on the algorithm, error target, and code assumptions.

What's real today: Small Hubbard models are classically accessible. The interesting target is the intermediate-coupling, doped 2D regime where controlled methods still leave open questions. The algorithmic route is reasonably clear; the fault-tolerant hardware and state-preparation stack are not. Of all the applications in this book, materials simulation is one of the clearest paths to genuinely new scientific information once those machines exist.

Chef's Notes

- **QPE = QFT + controlled time evolution.** QPE was previewed in Unit 2 (Shor uses it for period-finding). Here it appears in its natural habitat: extracting energy eigenvalues from a physical Hamiltonian. The QFT, phase kickback, and controlled unitaries are the same machinery; applied to a different problem.
- **Trotterisation bridges Units 3, 7, and 8.** In Unit 3 (VQE), we prepared trial states and measured energy variationally. Here and in Unit 8, we implement actual time evolution (e^{-iHt}), which requires decomposing the Hamiltonian into a Trotter product. The encodings book (Chapters 14–16) covers this in detail.
- **The sign problem is deep.** The reason QMC fails for fermions is intimately connected to the antisymmetry of the fermionic wavefunction; the same physics that requires fermion-to-qubit encodings. Quantum computers bypass the sign problem by representing the fermionic state directly (via encodings), rather than sampling from it.
- **Further reading:**
  - Hubbard (1963). Electron correlations in narrow energy bands. Proceedings of the Royal Society A 276:238–257
  - Babbush, Wiebe, McClean, et al. (2018). Low-Depth Quantum Simulation of Materials. Physical Review X 8:011044

- McArdle, Endo, Aspuru-Guzik, Benjamin, Yuan (2020). Quantum computational chemistry. Reviews of Modern Physics 92:015003 (arXiv:1808.10402)
- Troyer and Wiese (2005). Computational Complexity and Fundamental Limitations to Fermionic Quantum Monte Carlo Simulations. Physical Review Letters 94:170201 (arXiv:cond-mat/0407066)
- Qin, Schäfer, Andergassen, Corboz, Gull (2022). The Hubbard Model: A Computational Perspective. Annual Review of Condensed Matter Physics 13:275–302 (arXiv:2103.12097)

Deep-Dive 7: QPE and Trotterisation

This deep dive pairs with Unit 7 (Materials Science), which explained why the Hubbard model defeats classical methods and how QPE could solve it. Here we build QPE and the Trotter time-evolution circuit from the components introduced in earlier deep dives.

In This Chapter

- **What you'll learn:** How QPE extracts exact energy eigenvalues, how Trotterisation approximates time evolution as a circuit, and how to estimate the resources needed for real materials simulation.
- **What you need:** From Deep-Dive 2 (Shor), you know the QFT and phase kickback. From Deep-Dive 1 (QAOA), you know the ZZ gate and CNOT sandwich. Here we combine them for Hamiltonian simulation.
- **Runnable version:** The companion notebook `07-materials-science.ipynb` classically benchmarks the 2-site Hubbard model and then runs a compiled toy QPE phase-readout circuit on a cloud Quokka.

QPE: from phases to eigenvalues

Where we are

Let's take stock of what we already have. From Deep-Dive 1, we know how to build cost Hamiltonians from ZZ gates and encode optimisation problems as quantum operators. From Deep-Dive 2, we know how to extract a hidden number from the phase of a quantum state — that's what the controlled-powers-plus-inverse-QFT pattern did for Shor's algorithm. From Deep-Dive 3, we know how to encode a molecular Hamiltonian as a sum of Pauli operators.

QPE puts these together. The question it answers: given a physical system described by a Hamiltonian H, what are its energy levels?

Here's the idea in words before we touch a formula. If you let a quantum system evolve under its Hamiltonian H for time t, each energy eigenstate picks up a phase proportional to its energy — states with higher energy rotate faster. QPE measures that rotation speed. The QFT (from Deep-Dive 2) converts the rotation into a binary number you can read out. Phase kickback (from Deep-Dive 2) is the mechanism that transfers the phase information from the system to the measurement qubits.

That's it. Same tools, new application. Let's see how they fit together.

The setup

QPE involves two registers. The **system register** holds the quantum state whose energy we want to measure. The **ancilla register** — m helper qubits used as a measurement instrument — will hold the answer.

If the system is in an eigenstate $|$
psi

$rangle$ of the Hamiltonian with energy E, then time evolution gives $e^{-iHt}|$
psi
$rangle = e^{-iEt}|$
psi
$rangle$. The phase
$phi = -Et/(2$
$pi)$ encodes the energy. QPE extracts
phi.

The circuit

The circuit has four steps — all of which reuse pieces you've seen before:

1. **Hadamard** all ancillas → superposition
2. **Controlled-U^{2^k}** for each ancilla qubit k → phase accumulation
3. **Inverse QFT** on the ancillas → converts phases to a binary number
4. **Measure** the ancillas → read out ϕ

Steps 1, 3, and 4 we've seen before. The inverse QFT is the QFT from Deep-Dive 2, run backwards (swap the order of gates, negate the rotation angles). Step 2 is the new ingredient.

Controlled-U^{2^k}

Ancilla qubit k controls the application of U^{2^k} to the system register. If the system is in eigenstate $|\psi\rangle$:

$$\text{c-}U^{2^k}\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)|\psi\rangle = \frac{|0\rangle + e^{2\pi i \cdot 2^k \phi}|1\rangle}{\sqrt{2}}|\psi\rangle$$

The ancilla picks up a phase proportional to $2^k\phi$. After all m controlled operations, the ancilla register holds:

$$\frac{1}{\sqrt{2^m}}\sum_{k=0}^{2^m-1} e^{2\pi i k\phi}|k\rangle$$

This is exactly the QFT of $|\phi\rangle$. Applying the inverse QFT yields ϕ (or its best m-bit approximation).

> **Connection to Shor:** In Deep-Dive 2, QPE was implicit — the controlled modular exponentiation + QFT was QPE applied to the modular multiplication operator. Here we make it explicit and apply it to a physically motivated Hamiltonian.

Trotterisation: implementing e^{-iHt}

The problem

QPE requires controlled applications of $U = e^{-iHt}$. But H is a sum of terms that don't commute:

$$H = H_1 + H_2 + \cdots + H_L$$

We can implement $e^{-iH_k t}$ for each individual term (because each term has a simple structure; products of Pauli operators). But $e^{-iHt} \neq e^{-iH_1 t} \cdot e^{-iH_2 t} \cdots e^{-iH_L t}$ because the terms don't commute.

First-order Trotter

The **Trotter formula** says: the product of exponentials approximates the exponential of the sum if the time step is small:

$$e^{-iHt} \approx \left(e^{-iH_1 \Delta t} \cdot e^{-iH_2 \Delta t} \cdots e^{-iH_L \Delta t}\right)^N$$

where $\Delta t = t/N$. The error is $O(L^2 t^2 \Delta t)$; it goes to zero as $\Delta t \to 0$ (more Trotter steps).

Second-order Trotter (Suzuki)

A better approximation symmetrises the product:

$$e^{-iHt} \approx \left(e^{-iH_1 \Delta t/2} \cdot e^{-iH_2 \Delta t/2} \cdots e^{-iH_L \Delta t} \cdots e^{-iH_2 \Delta t/2} \cdot e^{-iH_1 \Delta t/2}\right)^N$$

The error drops to $O(L^3 t^3 \Delta t^2)$; much better for the same circuit depth. Higher-order formulas exist but add circuit complexity.

Implementing each term

For the Hubbard model:

Hopping terms $c^\dagger_{i\sigma} c_{j\sigma} + \text{h.c.}$: after Jordan-Wigner encoding (Deep-Dive 3), these become $\frac{1}{2}(X_i X_j + Y_i Y_j) \cdot Z_{\text{string}}$. The time evolution under this term is:

$$e^{-i\theta(X_i X_j + Y_i Y_j)/2}$$

This requires 2 CNOTs mixed with single-qubit rotations — a known decomposition that generalises the ZZ sandwich from Deep-Dive 1.

Interaction terms $n_{i\uparrow}n_{i\downarrow}$: after encoding, each becomes $\frac{(1-Z_{i\uparrow})(1-Z_{i\downarrow})}{4}$. This is diagonal; the time evolution is a ZZ phase gate, exactly the CNOT sandwich from Deep-Dive 1:

"'{figure} ../figures/trotter-zz-interaction.png :name: fig-trotter-zz-interaction :alt: Trotter ZZ interaction term implemented as a CNOT-Rz-CNOT sequence.

The interaction term is diagonal, so the Hubbard Trotter step reuses the same ZZ phase gadget that already appeared in QAOA and VQE.

Every piece of the Trotter circuit is built from gates we already know.

Resource estimation

Gate count

For an $L \times L$ Hubbard lattice with $2L^2$ spin-orbitals:

Component	Gate count
Hopping + interaction terms per Trotter step	$O(L^2)$ two-qubit gates
One Trotterized call to e^{-iHt}	$O(N_\text{Trotter} \cdot L^2)$ gates
Standard QPE to phase precision ϵ_PE	$O(N_\text{Trotter} \cdot L^2 /
Inverse QFT overhead	$O(\log^2(1/\epsilon_\text{PE}))$ controlled rotations

The key point is that QPE is **not** linear in the number of ancilla bits. If you use m a
1}}$, so the total coherent evolution scales like $2^m = O(1/\epsilon_\text{PE})$, not $
to-end resource estimates than to multiply a few back-of-the-
envelope factors.

McArdle et al. (2020) summarise two representative results for a 100-
site 2D Fermi-Hubbard problem. A qubitization-based estimate with an intensive error tar
gates and roughly 2-3 million physical qubits. A Trotter-
based estimate with a size-extensive error target lands at about 10^6 Toffoli gates / $
10^8 T-gates and roughly 4×10^5-6×10^5 physical qubits. Those figure
and-play comparable because the error conventions differ, but they put the problem firml
tolerant regime.

Physical qubits

There is no single, architecture-independent conversion from logical to physical qubits
qubit costs, from hundreds of thousands into the low millions, rather than one fixed numb
to-physical multiplier.

What you should take away

1. **QPE = controlled time evolution + inverse QFT.** It extracts energy eigenvalues to a
Dive 2) and phase kickback (from Deep-Dive 2) do the heavy lifting.

2. **Trotterisation is the bridge between Hamiltonians and circuits.** It breaks e^{-iHt} into a product of simple operations — each implemented with the gates from Deep-Dive 1 (ZZ sandwich for diagonal terms) and generalisations for off-diagonal terms.

3. **The circuit is deep but structured.** Every gate in a Trotter circuit has a physical

4. **Resource estimates are concrete, but assumption-dependent.** For roughly 100-site 2D Hubbard problems, published fault-tolerant studies land in the 10^7-10^8 to 10^9 T-gate band and the hundreds-of-thousands to low-millions physical-qubit band, depending on the algorithm and error model.

5. **Everything connects.** QPE reuses the QFT from Deep-Dive 2 (Shor). Trotterisation r
Dive 1 (QAOA). The fermionic encoding comes from Deep-Dive 3 (VQE). This is where the thr

Beyond Trotter: qubitization

Trotterisation approximates time evolution by breaking it into small steps — like drawing a curve with many short straight-line segments. More segments give a better

Qubitization (Low and Chuang, 2019) takes a fundamentally different approach: instea
quantum signal processing (QSP) and **quantum singular value transformation** (QSVT
that achieve the theoretically optimal gate count for Hamiltonian simulation.

The tradeoff: qubitization requires more ancilla qubits and a more complex circuit struc
term devices, Trotterisation's simpler circuits may still win. But for the long-term fault-tolerant regime, qubitization is the theoretically optimal approach.

- Low and Chuang (2019). *Hamiltonian Simulation by Qubitization.* [Quantum 3:163](https
2019-07-12-163) ([arXiv:1610.06546](https://arxiv.org/abs/1610.06546))
- Gilyén, Su, Low, Wiebe (2019). *Quantum singular value transformation and beyond.* [ST

Unit 8: Climate & Energy — *Designing a Better Catalyst for Carbon Capture*

The Hook

In 2024, the concentration of CO_2 in Earth's atmosphere passed 427 parts per million; the

The bottleneck isn't engineering; it's chemistry. Every DAC process depends on a **catal

This is a quantum chemistry problem. The catalyst's active site; the few atoms where the
level accuracy for the active site and classical efficiency for everything else.

No single classical method gives uniformly reliable, systematically improvable treatmen
to-moderate correlation regimes, but it becomes expensive and less reliable for strongly

This is where the quantum computing story comes together.

```
## The Bottleneck

### Surface chemistry: the accuracy-size tradeoff

A realistic catalyst simulation involves three scales:

1. **The active site** (~10–50 atoms, involving transition metals with partially filled
orbitals** — the outermost electron orbitals of metals like iron and copper, which give t
2. **The local environment** (~50–200 atoms of the support material and nearby solvent):
3. **The bulk** (the rest of the material): can be described by a mean-
field or continuum model

The challenge is that the active site's electronic structure depends on the environment,

Classical methods pick one scale and approximate the others:
- **DFT**: handles everything at the same (insufficient) level of accuracy
- **Multiscale QM/MM**: use quantum mechanics (QM) for the active site and classical mole
and-spring model) for the environment — but the QM part is still classical DFT, which fai
- **Embedded CCSD(T)**: classical high-accuracy method for the active site, DFT for the e
better, but CCSD(T) still scales as $O(N^7)$ and breaks down for the strongly correlated

What we need: a method that gives quantum-accurate results for the active site while scal


## The Quantum Angle

### Quantum embedding: the best of both worlds

A plausible route is **quantum embedding**: use a quantum computer for the hard correlate

The mathematical framework:

1. **Define the active space**: identify the orbitals (typically 20–
50) where strong correlation lives; the partially filled d-orbitals of the transition me
orbitals of the reacting molecule
2. **Embed**: use a classical method (DFT or Hartree-Fock) to compute the effect of the e
3. **Solve the active space quantumly**: run VQE (Unit 3) or QPE (Unit 7) on the effective
4. **Self-consistently update**: the active-space solution modifies the environment, wh

This approach — embedding a quantum-solved active space inside a classically-
solved environment — goes by several names. **DMET** (Density Matrix Embedding Theory) i
developed framework; **active-space VQE/QPE** is the broader idea.

### The full pipeline

This unit is the culmination of the book. The complete quantum simulation pipeline draws

```{figure} ../figures/climate-pipeline.png
:name: fig-climate-pipeline
:alt: Quantum simulation pipeline for catalyst modelling with cross-
references to earlier units.
```
```

`The capstone workflow is mostly classical plumbing around one hard quantum subproblem: s`

Every step in this pipeline has been introduced in an earlier unit. This chapter shows how they connect into a single, end-to-end workflow.

Why the active space matters

The active space is where the computational savings come from; and where the physical insight enters.

As an order-of-magnitude counting example, a CO_2-capture active site on a metal oxide surface might involve an iron or copper centre and its nearest-neighbour ligands — say 16 spatial orbitals (32 spin-orbitals after accounting for spin). After Jordan-Wigner encoding: 32 qubits. After symmetry reduction in a favourable encoding: perhaps 20–24 qubits. That does not make the problem automatically easy, but it shows why embedding is attractive: the quantum register tracks the fragment, not the whole surface.

The key point: **you don't need to simulate the whole surface.** The quantum computer handles the orbitals where strong correlation matters. The classical computer handles the rest of the surface and solvent at a cheaper approximate level. This division of labour is what makes quantum-classical embedding practical.

Worked Example

To keep the capstone notebook runnable and honest, the companion notebook uses a **precomputed 2-qubit embedded Hamiltonian** for a toy catalyst active site rather than a full Fe_2O_3 cluster calculation.

What the notebook actually does: 1. starts from a classical embedding baseline and a precomputed reduced Hamiltonian; 2. runs one active-space VQE solve step on that reduced model; 3. compares the VQE result against exact diagonalisation of the same embedded toy Hamiltonian; and 4. shows where that quantum solve step sits inside the larger catalyst-screening pipeline.

What it does **not** do: build the active space from molecular integrals, construct the bath, or iterate the embedding self-consistently. Those are part of the real workflow, but they are represented structurally rather than executed in the notebook.

That still teaches the central idea. The point of quantum embedding is not that the quantum computer simulates the whole catalyst; it solves the **hard correlated fragment** after the classical front end has compressed the environment into an effective Hamiltonian.

→ The next chapter builds the embedding workflow conceptually — select, embed, solve, iterate — while the notebook keeps the runnable part focused on the quantum solve step.

Reality Check

Catalysis-scale resource estimates are still large. Published fault-tolerant chemistry estimates for strongly correlated benchmarks such as FeMo-co remain far beyond current hardware. McArdle et al. (2020) summarise about 200 million physical qubits for the 2017 Trotter/QPE FeMo-co estimate under the surface-code assumptions used there, and about 1 million physical qubits' worth of Toffoli-distillation cost in a later qubitization-based estimate, though a full end-to-end fault-tolerant analysis for that later algorithm was not yet completed. These are nitrogen-fixation benchmarks rather than DAC calculations, but they show the right scale.

What's been demonstrated. Active-space VQE has been demonstrated for small molecules (H_2, LiH, BeH_2) on real hardware. No catalyst system has been simulated quantumly at a scale that produces new chemistry; the gap between "toy demonstration" and "useful catalyst screening" remains large.

The classical competition. DMRG and related tensor-network methods have made real progress on strongly correlated active spaces, including metalloenzyme complexes with active spaces above 70 spin-orbitals. But the review literature still points to a hard sweet spot around 100–200 spin-orbitals where no classical method is uniformly reliable.

What remains open. Embedding can shrink the quantum part dramatically, but the literature does not yet support a settled catalyst-screening threshold in qubits or years. Resource needs depend on the active-space choice, the solver (VQE versus QPE), and the fault-tolerance stack. The safe claim is that useful catalyst screening remains a fault-tolerant problem, not a NISQ one.

What's real today: The pipeline is real in pieces: active-space selection, integral generation, encoding, and classical embedding loops are mature workflows, and toy quantum subroutines can be inserted into that stack. What's missing is an end-to-end quantum solve at scientifically useful catalyst scale.

Chef's Notes

- **This chapter is the capstone.** Every concept introduced in the book; qubits as binary variables (Unit 1), superposition and interference (Unit 2), fermions and encodings (Unit 3), variational methods (Units 1, 3), amplitude estimation (Unit 5), QPE and Trotterisation (Unit 7); comes together here in a single end-to-end pipeline.
- **Active-space and embedding methods are the main scaling strategy for quantum chemistry on limited hardware.** They can reduce the quantum resource requirement by an order of magnitude or more on realistic benchmark systems. The classical insight (where does strong correlation live?) combines with the quantum capability (treat the correlated subsystem more accurately).
- **The encodings book covers the encoding step in depth.** This unit's pipeline includes "encode the active-space Hamiltonian as a qubit operator"; the step where From Molecules to Qubits begins. Readers who want to un-

derstand why Jordan-Wigner produces long operator strings, or how tapering reduces qubit count, should start there.

- **Catalyst design is a compelling long-term target for quantum computing.** It's economically important (carbon capture, fertiliser production, hydrogen fuel cells), chemically concrete (compute reaction energetics for an active site), and still difficult for the classical toolbox in the strongly correlated regime. If quantum computers prove their worth anywhere in chemistry, embedded catalytic active sites are one plausible place it could happen.
- **Further reading:**
  - Reiher, Wiebe, Svore, Wecker, Troyer (2017). Elucidating Reaction Mechanisms on Quantum Computers. PNAS 114:7555–7560
  - Lee, Berry, Gidney, et al. (2021). Even More Efficient Quantum Computations of Chemistry Through Tensor Hypercontraction. PRX Quantum 2:030305 (arXiv:2012.09265)
  - Li, Otten, Isaev, et al. (2022). Toward Practical Quantum Embedding Simulation of Realistic Chemical Systems on Near-Term Quantum Computers. Chemical Science 13:8953–8962
  - Azariah et al. From Molecules to Qubits. github.com/johnazariah/encodings-book

Deep-Dive 8: Quantum Embedding Methods

This deep dive pairs with Unit 8 (Climate & Energy), which explained why catalyst design requires quantum accuracy for the active site and classical efficiency for the environment. Here we show how to combine both.

In This Chapter

- **What you'll learn:** How to select an active space, embed it in a classical environment, solve the active space with VQE or QPE, and iterate to self-consistency.
- **What you need:** This is the capstone deep dive. It draws on VQE (Deep-Dive 3), QPE (Deep-Dive 7), and fermion-to-qubit encodings (Deep-Dive 3). If you've followed the deep dive path, you have everything you need.
- **Runnable version:** The companion notebook `08-climate-energy.ipynb` illustrates the embedded VQE solve step on a cloud Quokka using a precomputed toy Hamiltonian.

The active space idea

This chapter moves faster than the earlier deep dives. That's by design: it's the capstone, and every tool it uses has been built in a previous chapter. Where earlier deep dives taught one mechanism at a time (the ZZ gate, the QFT, the VQE loop), this one assembles a pipeline from pieces you already know. If a step feels unfamiliar, the cross-references will take you back to where it was introduced.

The pipeline has three stages: **select** the orbitals that need quantum treatment, **embed** them in the classical environment, and **solve** the embedded problem with VQE or QPE. Let's take them one at a time.

Why you don't simulate the whole molecule

A catalyst system has hundreds of orbitals; carbon scaffolding, solvent molecules, bulk metal. But the chemistry happens at just a few: the metal d-orbitals where CO binds, the π* orbitals where electrons transfer.

Strong correlation; the kind that defeats classical methods; lives in these few orbitals. The rest are either doubly occupied (core electrons: inert, boring) or empty (high-energy virtual orbitals: irrelevant at normal temperatures). Classical methods handle these "boring" orbitals perfectly well.

The strategy: **use a quantum computer only for the hard part.** Select the 10–50 orbitals where classical methods are most stressed (the "active space"), and treat those with VQE (Deep-Dive 3) or, in a fault-tolerant setting, QPE (Deep-Dive 7). Let a classical computer handle the other 450 orbitals with standard DFT or Hartree-Fock.

How to choose the active space

This is part science, part art:

- **The metal d-orbitals.** Transition metals (Fe, Cu, Mn, Co) have partially filled d-shells. These orbitals are close in energy and strongly coupled; the definition of strong correlation.
- **The adsorbate frontier orbitals.** For CO_2 capture: the CO_2 π^* orbital that accepts electrons from the metal.
- **Natural orbital analysis.** Compute approximate natural orbital occupation numbers from a cheap classical calculation (CASSCF — Complete Active Space Self-Consistent Field, a classical method that optimises both orbitals and electron configurations within a small active space — or MP2, the perturbative correction from Unit 3's hierarchy). Orbitals with occupations significantly different from 0 or 2 are strongly correlated; include them.
- **Entropy-based selection.** Compute the single-orbital entropy from an approximate wavefunction. High-entropy orbitals are strongly correlated.

As an order-of-magnitude example, a catalyst active site with 6 metal d-orbitals and 4–8 ligand orbitals gives 10–14 active orbitals. After Jordan-Wigner encoding (Deep-Dive 3): 20–28 qubits. After tapering (using point-group symmetry and particle-number conservation to reduce qubits): possibly 12–20 qubits in a favourable case. That is small enough to motivate embedding, but not automatically practical on current hardware once circuit depth and measurement cost are included.

DMET: Density Matrix Embedding Theory

The mathematical framework

DMET (Knizia and Chan, 2012) is a clean embedding approach. It works by recognising that the environment's effect on the fragment can be captured by a small **bath**; a set of orbitals that encode the entanglement between the fragment and the rest of the system.

The procedure:

1. **Solve the full system cheaply.** Run Hartree-Fock on all 500 orbitals. This gives a mean-field **density matrix** ρ_{MF} — a mathematical object that describes the quantum state of a system in terms of probabilities and correlations (a generalisation of the outer product $|\psi\rangle\langle\psi|$ from Deep-Dive 5, extended to handle statistical mixtures of states).

2. **Build the bath.** From ρ_{MF}, extract the entanglement between the fragment (active site) and the environment. The bath has the same number of orbitals as the fragment; so a 10-orbital fragment gets a 10-orbital bath. The 20-orbital embedded problem captures the fragment-environment entanglement exactly (at the mean-field level).

3. **Build the embedded Hamiltonian.** Project the full Hamiltonian onto the fragment + bath space. The result is a small Hamiltonian; 20 orbitals instead of 500; that includes the environment's effect as an effective potential.

4. **Solve the embedded problem.** Run VQE (Deep-Dive 3) or QPE (Deep-Dive 7) on the 20-orbital embedded Hamiltonian. This yields a fragment energy and density matrix in the presence of the environment: variational and approximate with VQE, systematically improvable and in principle arbitrarily precise with QPE.

5. **Self-consistency.** Update the mean-field solution with the quantum result. The new mean-field changes the bath, which changes the embedded Hamiltonian, which changes the quantum solution. Iterate until convergence (typically 5–10 cycles).

What the quantum computer does

The quantum computer appears exactly once per self-consistency cycle: step 4. Everything else is classical linear algebra. The quantum computer tackles the small, strongly correlated fragment + bath problem; the part classical methods handle least systematically.

The pipeline is solver-agnostic. Today, use VQE on NISQ hardware. Tomorrow, use QPE on fault-tolerant hardware. The embedding framework doesn't change; only the quantum subroutine.

The complete pipeline

This chapter ties together every deep dive in the book:

"'{figure} ../figures/embedding-pipeline.png :name: fig-embedding-pipeline :alt: Quantum-classical embedding pipeline showing NISQ and fault-tolerant paths with chapter cross-references.

Embedding keeps the quantum computer on the small fragment where strong correlation lives and leaves the environment to the classical solver. "'

Every box references a chapter where the concept was introduced. A reader who followed the deep dive path (Chapters 2 → 4 → 6 → 8 → 10 → 12 → 14 → 16) has the tools to understand every step. A reader who skipped the deep dives can see the pipeline and know where to look for details.

The companion notebook is narrower than the full theory in this chapter. It starts from a precomputed embedded Hamiltonian for a 2-qubit toy active space, runs one VQE solve step on Quokka, and uses that runnable fragment to show where the quantum subroutine lives in the larger select → embed → solve → iterate pipeline.

→ **See notebook `08-climate-energy.ipynb` for the runnable version.**

What you should take away

1. **The active space is the key insight.** You don't need a quantum computer for the whole molecule; just for the 10–50 orbitals where classical methods are most stressed. This can reduce the quantum resource requirement by orders of magnitude.
2. **DMET provides the framework.** It tells you how to build the bath, how to embed the fragment, and how to iterate to self-consistency. The quantum computer plugs in as the solver for the embedded problem.
3. **Everything connects.** The qubit Hamiltonian comes from Deep-Dive 3 (VQE pipeline). The solver is either VQE (Deep-Dive 3) or QPE (Deep-Dive 7). The ZZ gates are from Deep-Dive 1 (QAOA). The QFT is from Deep-Dive 2 (Shor).

Phase kickback drives QPE. The cost Hamiltonian pattern from Unit 1 reappears as the molecular Hamiltonian.

4. **This is one plausible pipeline for quantum utility in chemistry.** The target is not another H_2 demo but chemically meaningful embedded active spaces on realistic systems. Published chemistry resource estimates still span a wide band, from roughly a million physical qubits into the hundreds of millions depending on algorithm and assumptions. Embedding matters because it is one of the few routes to pushing those costs down by shrinking the quantum fragment instead of attacking the full system head-on.
5. **For the encoding step,** see From Molecules to Qubits. That book covers exactly the transformation at the heart of this pipeline.

The Elephant in the Room: Quantum Error Correction

Every Reality Check in this book has ended at the same wall: the hardware isn't ready yet. The algorithms work. The mathematics is settled. But running them on real quantum computers — at the scale where they'd actually beat classical methods — requires something we don't yet have: **fault-tolerant quantum computing**.

This chapter explains why, and what it will take to get there.

Why quantum computers make mistakes

A classical computer stores a bit as a voltage: high or low. The voltage can drift, but as long as it stays on the right side of the threshold, the bit is correct. Digital error correction is essentially free — every logic gate refreshes the signal.

Qubits are different. A qubit's state is a point on the Bloch sphere — a continuous value, not a discrete one. Any interaction with the environment (stray electromagnetic fields, thermal vibrations, cosmic rays) nudges the state slightly. This is **decoherence**: the qubit's quantum information leaks into the environment, turning a precise superposition into a noisy mess.

Gate operations aren't perfect either. Every gate applied to a qubit introduces a small rotation error. On current hardware, the error rate per gate is roughly 10^{-3} — one mistake per thousand operations. That sounds small, but Shor's algorithm for RSA-2048 requires $\sim 10^{10}$ gates. With a 10^{-3} error rate, the computation would drown in noise after a few thousand gates.

The core problem: **quantum information is fragile, and the computations we want to run are deep.**

The basic idea: redundancy without looking

Classical error correction is straightforward: make copies. Store three copies of each bit; if one flips, take the majority vote. Simple, effective, and used everywhere from hard drives to deep-space communication.

Quantum error correction can't copy. The **no-cloning theorem** says you cannot make an exact copy of an unknown quantum state. And you can't just check a qubit's value to see if it's correct — measuring it destroys the superposition.

So quantum error correction uses a different kind of redundancy. Instead of copying the state, you **spread it across many physical qubits** in a way that lets you detect and correct errors without ever learning what the state is.

A **logical qubit** — one error-protected unit of quantum information — is encoded in many **physical qubits**. The encoding is designed so that small errors (a single physical qubit flipping or drifting) produce detectable syndromes: patterns in the correlations between physical qubits that reveal which error occurred, without revealing the encoded state itself.

Measure the syndromes. Identify the error. Apply a correction. The logical qubit is restored — and you never learned what state it was in. This is the trick that makes quantum error correction possible despite the no-cloning theorem and the fragility of measurement.

Surface codes: the leading approach

The most developed error-correction scheme is the **surface code**. Physical qubits are arranged on a 2D grid. Each logical qubit is encoded in a patch of $d \times d$ physical qubits, where d is the **code distance** — roughly, the number of physical errors that must occur simultaneously to corrupt the logical qubit.

Key properties:

- **Threshold**: if the physical error rate is below ~1%, the surface code can correct errors faster than they accumulate. Current hardware is near or just below this threshold.
- **Overhead**: each logical qubit requires $\sim 2d^2$ physical qubits. For $d = 20$ (enough for many algorithms): ~ 800 physical qubits per logical qubit.
- **Locality**: syndrome measurements involve only nearest-neighbour interactions on the 2D grid — a good match for superconducting and ion-trap hardware.

The cost: a computation that needs 200 logical qubits (like QPE on the Hubbard model, Unit 7) requires $\sim 160{,}000$ physical qubits with surface codes.

LDPC codes: doing more with less

The surface code's main weakness is overhead: $\sim 2d^2$ physical qubits per logical qubit, with d in the range 15–25 for practical computations. That means 500–1,000 physical qubits for every logical one.

Quantum LDPC codes (Low-Density Parity-Check codes) are a newer family that achieve a better ratio — in principle, a constant number of physical qubits per logical qubit, independent of the code distance. The Pinnacle architecture (Unit 2's Reality Check) uses LDPC codes to reduce the cost of breaking RSA-2048 from 20 million physical qubits to under 100,000 — a 200× improvement.

The tradeoff: LDPC codes require longer-range connectivity between qubits (not just nearest neighbours), which is harder to build in hardware. Surface codes work with a simple 2D grid; LDPC codes need a more complex wiring pattern. Whether current architectures can support LDPC codes at scale is an active engineering challenge — but the potential payoff is enormous, and several hardware groups are actively pursuing it.

Error mitigation: the NISQ bridge

Fault-tolerant quantum computing with full error correction is a future technology. Today's machines — the **NISQ** (Noisy Intermediate-Scale Quantum) era — have too few qubits and too much noise for surface codes.

Error mitigation is the pragmatic alternative: classical post-processing techniques that reduce the impact of noise without the full overhead of error correction. Key approaches:

- **Zero-noise extrapolation**: run the circuit at several artificially increased noise levels, then extrapolate back to zero noise.
- **Probabilistic error cancellation**: decompose noisy gates into combinations of ideal gates, using randomness and classical post-processing to cancel errors statistically.
- **Symmetry verification**: discard measurement results that violate known symmetries of the problem (like electron number conservation in chemistry).

Error mitigation extends the useful depth of NISQ circuits by a modest factor — enough for VQE on small molecules (Unit 3) and small QAOA instances (Unit 1), but not enough for Shor's algorithm or large-scale QPE.

Where we are and where we're going

Milestone	Qubits	Status
NISQ demonstrations	50–1,000 physical	**Now** (2024–2026)
First logical qubit (below break-even)	~1,000 physical → 1 logical	**Demonstrated** (Google, 2024)
Early fault-tolerant (useful QEC)	10^4–10^5 physical → 10^2 logical	**5–10 years**
Cryptographically relevant	10^5–10^6 physical → 10^3 logical	**10–15 years**
Full-scale quantum chemistry	10^5–10^6 physical → 10^3 logical	**10–15 years**

Google's 2024 result — demonstrating a logical qubit with a lower error rate than any of its constituent physical qubits — was the first time a quantum error-correcting code operated below break-even. It used a surface code with distance $d = 5$ (49 physical qubits per logical qubit). Scaling to $d = 20$ and hundreds of logical qubits is an engineering challenge, not a physics one. The path is clear; the question is how fast we walk it.

Every resource estimate in this book — 100,000 qubits for RSA-2048, 160,000 for the Hubbard model, 100,000 for quantum chemistry — assumes that this engineering succeeds. The algorithms are waiting.

Deep-Dive 9: Inside Quantum Error Correction

This deep dive pairs with the preceding chapter (The Elephant in the Room), which explained why quantum computers make mistakes and what error correction buys you. Here we open the box and show how it actually works — from the simplest three-qubit code to the stabiliser formalism that underlies every modern error-correcting scheme.

In This Chapter

- **What you'll learn:** How to detect and correct quantum errors without measuring the encoded state, how the Pauli operators you already know form the mathematical backbone of error correction, and why fault-tolerant gates are hard.
- **What you need:** Pauli operators X, Y, Z (Unit 1 and Deep-Dive 1), CNOT (Deep-Dive 1), measurement (Unit 1), and the idea that quantum operations must be reversible (Deep-Dive 2). Everything else is built here.

The problem, concretely

A single qubit in state $\alpha|0\rangle + \beta|1\rangle$ can suffer three kinds of error:

- **Bit flip** (X error): $|0\rangle \leftrightarrow |1\rangle$. Like a classical bit flip.
- **Phase flip** (Z error): $|0\rangle \to |0\rangle$, $|1\rangle \to -|1\rangle$. No classical analogue — it changes the phase, not the bit value.
- **Both** ($Y = iXZ$ error): a bit flip and a phase flip together.

Any single-qubit error can be decomposed as a combination of I, X, Y, Z. So if we can correct X and Z errors separately, we can correct any single-qubit error. This is the key insight that makes quantum error correction possible despite the continuous nature of quantum states.

But we can't just check the qubit to see if it's wrong — measurement destroys the superposition. We need to detect errors without learning the state.

The bit-flip code: your first quantum error-correcting code

Encoding

Encode one logical qubit into three physical qubits:

$$|0\rangle_L = |000\rangle, \qquad |1\rangle_L = |111\rangle$$

A general state $\alpha|0\rangle + \beta|1\rangle$ becomes $\alpha|000\rangle + \beta|111\rangle$. This is not three copies of the qubit (that would violate the no-cloning theorem). It's one logical qubit

spread across three physical qubits — the superposition coefficients α and β are preserved in the correlations between the three qubits, not in any individual one.

The encoding circuit uses two CNOTs, but let's see the entire correction cycle at once — encoding, error, syndrome extraction, and correction — in a single circuit:

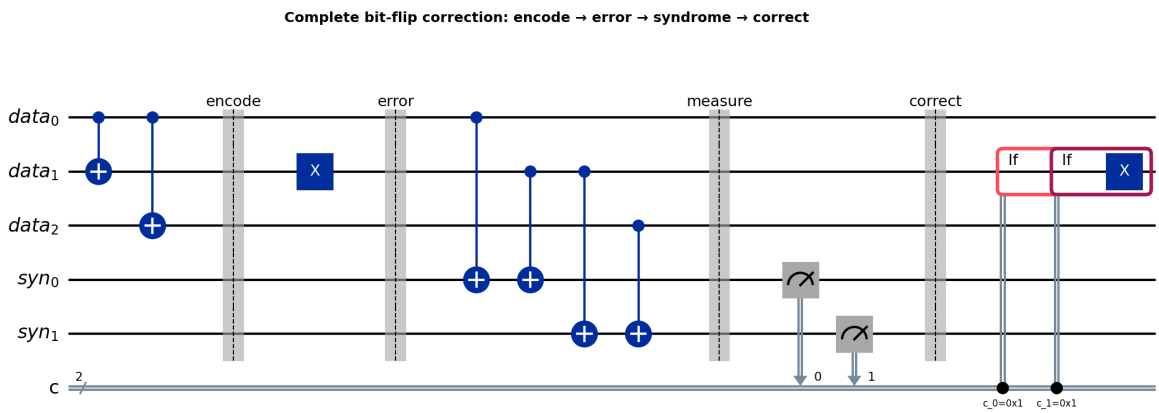

Figure 1. Complete bit-flip correction cycle: encode with CNOTs, an X error strikes qubit 2, syndrome ancillas detect it via parity checks, and a classically-conditioned X gate corrects it

Read left to right through four stages:

Encode: Two CNOTs spread the data qubit across three physical qubits: $\alpha|0\rangle + \beta|1\rangle \to \alpha|000\rangle + \beta|111\rangle$. This is not three copies (that would violate no-cloning). It's one logical qubit whose information lives in the correlations between the three qubits.

Error: A bit flip (X) hits qubit 2. The state becomes $\alpha|010\rangle + \beta|101\rangle$. We don't know this has happened — that's the point.

Syndrome: Two ancilla qubits measure the parity of qubit pairs via CNOTs. Z_1Z_2 checks whether qubits 1 and 2 agree; Z_2Z_3 checks qubits 2 and 3. Crucially, these parity checks reveal which qubit disagrees without revealing whether the logical state is $|0\rangle_L$ or $|1\rangle_L$:

Measurement	What it checks	No error	Qubit 1 flipped	Qubit 2 flipped	Qubit 3 flipped
Z_1Z_2	Do qubits 1 and 2 agree?	$+1$	-1	-1	$+1$
Z_2Z_3	Do qubits 2 and 3 agree?	$+1$	$+1$	-1	-1

The pair of outcomes — the **syndrome** — tells you exactly which qubit flipped:

- $(+1, +1)$: no error — do nothing
- $(-1, +1)$: qubit 1 flipped → apply X_1 to flip it back
- $(-1, -1)$: qubit 2 flipped → apply X_2 to flip it back
- $(+1, -1)$: qubit 3 flipped → apply X_3 to flip it back

The correction is just an X gate — the same Pauli-X (bit flip) from Unit 1 — applied to the qubit the syndrome identified. The error was an unwanted X; the correction is a deliberate X; and $X \cdot X = I$ (applying a bit flip twice returns to the original state). The entire cycle is: **encode → (error happens) → measure syndrome → apply corrective X → state restored.**

Notice that at no point did we learn α or β. The syndrome told us where the error was, not what the state is. This is the deep trick: error correction extracts information about the error while remaining ignorant of the encoded data.

This is the same $Z_i Z_j$ operator from the MaxCut Hamiltonian in Unit 1 — but now it's playing a completely different role. In QAOA, it encoded a cost function. Here, it detects errors. Same operator, new purpose.

What this code can't do

The bit-flip code corrects X errors but is blind to Z (phase) errors. If qubit 2 suffers a phase flip — $|0\rangle \to |0\rangle$, $|1\rangle \to -|1\rangle$ — the syndrome measurements both return $+1$ (the parities haven't changed), and the error goes undetected.

We need a code that handles both.

The phase-flip code

The phase-flip code is the bit-flip code in a different basis. Instead of encoding in the $|0\rangle/|1\rangle$ basis, encode in the $|+\rangle/|-\rangle$ basis:

$$|0\rangle_L = |{+}{+}{+}\rangle, \qquad |1\rangle_L = |{-}{-}{-}\rangle$$

Now a phase flip on one qubit (Z error) flips that qubit from $|+\rangle$ to $|-\rangle$ (or vice versa) — which is a bit flip in the Hadamard basis. The syndrome measurements become X_1X_2 and X_2X_3 instead of Z_1Z_2 and Z_2Z_3, and the correction is a Z gate on the identified qubit (since $Z \cdot Z = I$, just like $X \cdot X = I$).

Here's the full phase-flip correction cycle — compare it with the bit-flip circuit above:

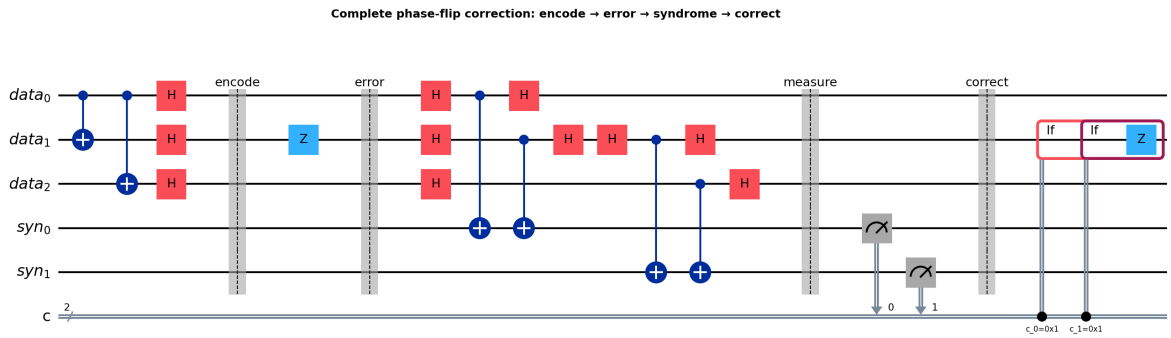

Figure 2. Complete phase-flip correction cycle: encode in Hadamard basis, a Z error strikes qubit 2, X-basis parity checks detect it, and a classically-conditioned Z gate corrects it

The structure is identical. The only differences: H gates wrap the encoding, the syndrome checks use X-basis parity instead of Z-basis, and the correction is Z instead of X. Everything else — the CNOTs, the ancillas, the classical conditioning — is the same circuit in a different basis.

The X/Z/Hadamard duality

This is not a coincidence. There is a deep symmetry at work:

Bit-flip code	Phase-flip code	Connected by
Encodes in $\|0\rangle$, $\|1\rangle$	Encodes in $\|+\rangle$, $\|-\rangle$	H
Detects X errors	Detects Z errors	H
Syndrome uses Z_iZ_j	Syndrome uses X_iX_j	H
Corrects with X	Corrects with Z	H

The Hadamard gate H swaps the X and Z operators: $HXH = Z$ and $HZH = X$. So everything about bit-flip correction transforms into phase-flip correction under H, and vice versa. Apply H to every qubit in the bit-flip code, and you get the phase-flip code. Apply H to the syndrome measurements, and Z_iZ_j becomes X_iX_j.

This duality runs deeper than error correction. It's the same symmetry that relates the computational basis to the Hadamard basis throughout quantum computing — the QFT generalises it, and phase kickback exploits it. Here, it tells us that bit errors and phase errors are the same problem in different bases. Any code that handles one can handle the other by conjugating with H.

The Shor code exploits this: it concatenates one code inside the other, catching both error types simultaneously.

The Shor code: correcting everything

Peter Shor's 9-qubit code (1995) concatenates both ideas: encode against phase flips first (3 physical qubits per block), then encode each block against bit flips (3 blocks of 3 = 9 qubits total).

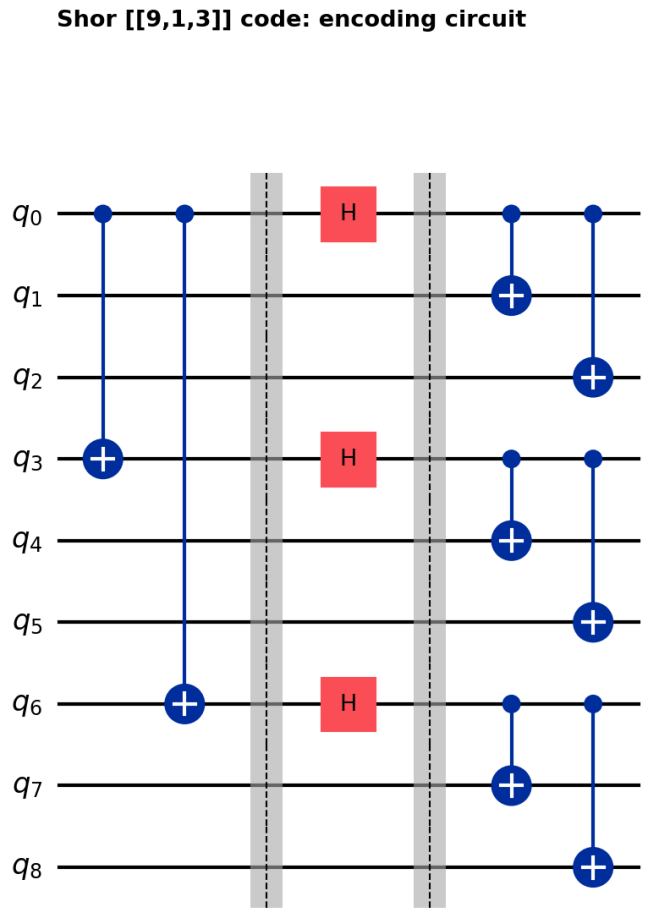

Figure 3. Shor [[9,1,3]] encoding circuit: phase-flip encoding (CNOTs + Hadamards) followed by bit-flip encoding within each block

$$|0\rangle_L = \frac{1}{2\sqrt{2}}(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)$$

$$|1\rangle_L = \frac{1}{2\sqrt{2}}(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)$$

Within each block of 3, the Z_iZ_j parity checks detect bit flips. Between blocks, X-basis parity checks detect phase flips. Together, they detect and correct any single-qubit error on any of the 9 physical qubits.

9 physical qubits for 1 logical qubit is expensive — an overhead of 9×. Modern codes do better. But the Shor code proved that quantum error correction is possible, which was not obvious before 1995. Many physicists believed that the continuous nature of quantum errors (rotations by arbitrary angles, not just discrete flips) would make correction impossible. The Shor code showed that discretising errors into X, Z, and Y components — and correcting each — handles everything.

The stabiliser formalism: the algebra behind it all

The bit-flip code, phase-flip code, and Shor code are all examples of **stabiliser codes**. The stabiliser formalism, developed by Gottesman (1997), provides a unified mathematical framework for constructing and analysing quantum error-correcting codes.

The idea

A stabiliser code is defined by a set of Pauli operators — products of I, X, Y, Z on the physical qubits — called **stabiliser generators**. The code space is the set of states that are $+1$ eigenstates of every generator.

For the 3-qubit bit-flip code: - Generator 1: $Z_1 Z_2$ (qubits 1 and 2 have the same parity) - Generator 2: $Z_2 Z_3$ (qubits 2 and 3 have the same parity)

The states satisfying both conditions are exactly $\alpha|000\rangle + \beta|111\rangle$ — the code space. An error moves the state out of the code space, and the generators' eigenvalues (the syndrome) tell you which error occurred.

Why it works

The Pauli operators $\{I, X, Y, Z\}^{\otimes n}$ form a group under multiplication (up to phases). Stabiliser generators are elements of this group that commute with each other. The key properties:

1. **Syndrome extraction doesn't disturb the code space.** Measuring a stabiliser generator on a codeword always gives $+1$ — it's an eigenvalue measurement, so the state is unchanged. Only errors change the eigenvalue to -1.
2. **Different errors give different syndromes.** If two errors E_1 and E_2 anti-commute with different stabilisers, they produce different syndrome patterns and can be distinguished.
3. **The code corrects t errors if no undetectable error has weight $\leq 2t$.** An undetectable error commutes with all stabilisers. The minimum weight of such an error is the code **distance** d, and the code corrects $\lfloor (d-1)/2 \rfloor$ errors.

The 3-qubit bit-flip code has distance 3 (the smallest undetectable bit-flip error is $X_1 X_2 X_3$, which flips all three qubits and maps $|000\rangle \to |111\rangle$). So it corrects 1 error.

A notation for codes

An $[[n, k, d]]$ code encodes k logical qubits into n physical qubits with distance d:

- 3-qubit bit-flip code: $[[3, 1, 3]]$ (corrects 1 X error, but only X errors)
- Shor code: $[[9, 1, 3]]$ (corrects any 1 error)
- Surface code (distance d): $[[2d^2, 1, d]]$ (corrects $\lfloor (d-1)/2 \rfloor$ errors of any type)

The overhead — the ratio n/k — is the price of protection.

The surface code: why it wins

The surface code arranges physical qubits on a 2D grid. **Data qubits** sit on the edges; **syndrome qubits** sit on the vertices and faces. Each syndrome qubit measures the parity of its neighbouring data qubits — either Z-parity (detecting bit flips) or X-parity (detecting phase flips).

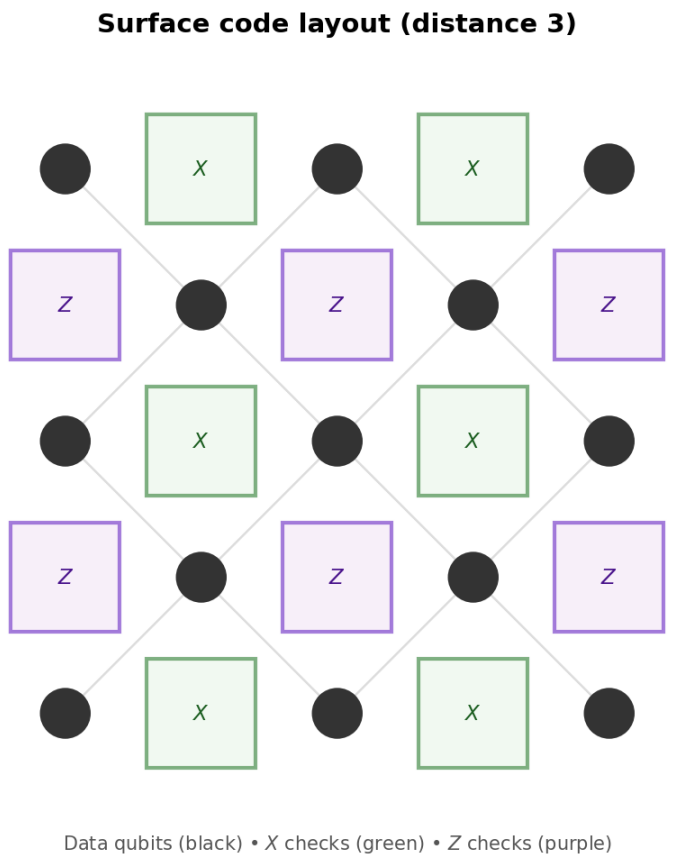

Figure 4. Surface code layout (distance 3): data qubits (black dots) interleaved with X-check plaquettes (green) and Z-check plaquettes (purple), all connected by nearest-neighbour edges

The surface code's advantages:

- **Locality.** Every syndrome measurement involves only nearest-neighbour interactions. No long-range connections needed. This matches the connectivity of superconducting and ion-trap hardware.
- **High threshold.** If the physical error rate is below ~1%, errors can be corrected faster than they accumulate. Current hardware is near this threshold.
- **Simple decoding.** The syndrome pattern can be decoded efficiently (in polynomial time) using minimum-weight matching algorithms.

The cost: $2d^2$ physical qubits per logical qubit for distance d. At $d = 20$ (enough for many useful computations): ~800 physical qubits per logical qubit. That's why the resource estimates in this book are so large — every logical qubit in QAOA, Shor's

algorithm, VQE, or QPE becomes hundreds of physical qubits once you demand reliability.

Why T-gates are expensive

In Deep-Dive 7, T-gate counts appeared as the standard cost metric for fault-tolerant algorithms. Here's why.

Most quantum gates — H, S, CNOT, and all Pauli gates — can be implemented **transversally** in the surface code: apply the gate to each physical qubit independently, and the logical gate is automatically correct. Errors don't spread.

The T gate ($\pi/8$ rotation) cannot be done transversally in any code that detects all single-qubit errors. This is the **Eastin-Knill theorem**: no code can implement a universal gate set entirely transversally.

The workaround: **magic state distillation**. Prepare a special auxiliary state ($T|+\rangle$), purify it through a resource-intensive distillation protocol, and use it to implement T via gate teleportation. Each logical T gate requires ~15× more physical qubits and operations than a Clifford gate.

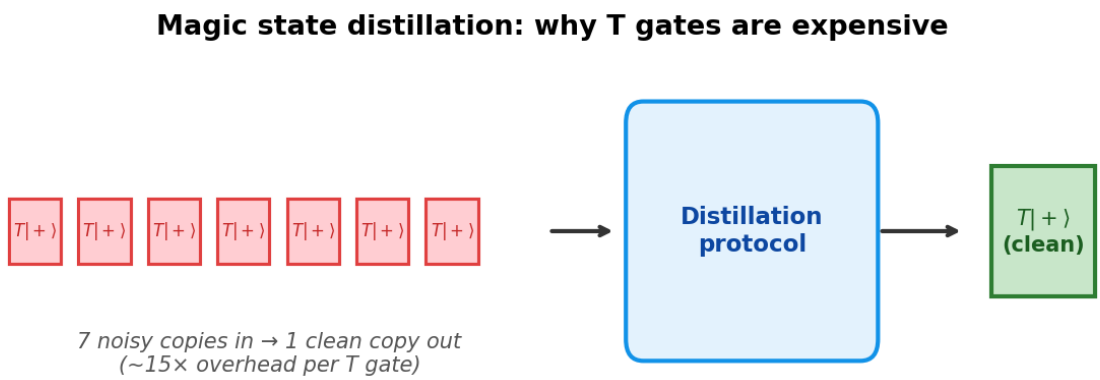

Figure 5. Magic state distillation: 7 noisy copies of the T-state are consumed to produce 1 clean copy, at ~15× overhead per logical T gate

This is why algorithms are optimised to minimise T-gate count. And it's why the resource tables in Units 2 and 7 report T-gates rather than total gates — the T-gates dominate the physical cost.

What you should take away

1. **Quantum errors decompose into Pauli errors.** Correcting X and Z individually handles everything. This discretisation is what makes quantum error correction possible.
2. **Syndrome measurements detect errors without revealing the state.** The Z_iZ_j parity checks from QAOA's cost Hamiltonian reappear here as error detectors — same operator, different purpose.

3. **The stabiliser formalism is the algebra of Pauli groups applied to coding theory.** A code is defined by commuting Pauli operators; the syndrome is a set of eigenvalues; correction is a Pauli operation conditioned on the syndrome.
4. **The surface code wins on locality and threshold.** Its 2D nearest-neighbour structure matches real hardware, and its ~1% error threshold is within reach.
5. **T-gates are expensive because universality conflicts with transversality.** Magic state distillation is the workaround, and it dominates the physical cost of fault-tolerant computation.
6. **Every resource estimate in this book is ultimately a statement about error correction.** The number of logical qubits times the per-qubit overhead of the code times the number of T-gates times the distillation cost — that's the real hardware bill.

Beyond stabiliser codes: the frontier

Everything in this chapter — the bit-flip code, the Shor code, the surface code — belongs to the family of **stabiliser codes**, the best-understood and most widely studied class of quantum error-correcting codes. But stabiliser codes are not the end of the story. Quantum error correction is one of the few areas of quantum computing where the mathematics is still generating fundamentally new approaches, not just optimising existing ones.

A few directions worth knowing about:

Quantum LDPC codes push the overhead down. The surface code uses $O(d^2)$ physical qubits per logical qubit, which means the overhead grows with the protection level. Quantum LDPC codes — the family behind the Pinnacle architecture from Unit 2 — can in principle achieve constant overhead: a fixed number of physical qubits per logical qubit, independent of code distance. The tradeoff is connectivity: they require non-local qubit interactions that are harder to build in hardware.

Colour codes offer an alternative to surface codes with different transversal gate sets and potentially easier implementation of certain logical operations. They use a three-colourable lattice instead of the surface code's checkerboard.

Floquet codes (Hastings and Haah, 2021) use time-varying stabiliser measurements — the code itself changes from one measurement round to the next, but the logical information is preserved. This is a genuinely new idea with no classical analogue.

Quantum error correction with constant space overhead (Gottesman, 2014; Fawzi, Grospellier, Leverrier, 2018) establishes that fault-tolerant quantum computation is possible with only a constant factor more physical qubits than logical ones, asymptotically. Realising this in practice is an open engineering challenge.

Bosonic codes (cat codes, GKP codes, binomial codes) encode a logical qubit into a single oscillator mode rather than into many physical qubits. These are particularly natural for superconducting hardware, where quantum information is stored in microwave cavities.

The field is active and accelerating. New code families are being discovered, new decoders are being designed, and the interplay between codes and hardware architectures is becoming one of the central engineering challenges of quantum computing. The mathematics of quantum error correction — drawing on algebra, topology, combinatorics, and information theory — is richer than any single chapter can convey. What we've covered here is the foundation; the frontier is wide open.

The Quantum Bottleneck

We started with a UPS driver and 20 stops. We end with a catalyst and 50 orbitals. Between them, eight problems — eight bottlenecks — and one recurring pattern.

Every quantum algorithm in this book does the same thing: it encodes a hard problem as a quantum operator, manipulates phases through interference, and extracts the answer through measurement. The operator changes — a cost Hamiltonian in Unit 1, a modular exponentiation oracle in Unit 2, a molecular Hamiltonian in Unit 3, a Grover iterator in Unit 5, a lattice Hamiltonian in Unit 7. The interference mechanism changes — the QAOA mixer, the QFT, the VQE ansatz, Grover's diffusion operator. But the architecture is always the same three acts: encode, interfere, measure.

This is not a coincidence. It's the structure of quantum mechanics itself. Quantum systems carry information in phases. Phases interfere. Measurement collapses the interference pattern into a classical answer. Every quantum algorithm is an interference machine designed to make the right answer loud and the wrong answers quiet.

What's real, what's next, what's unknown

Real today. The algorithms work. Shor's algorithm factors numbers. VQE computes molecular energies. QAOA optimises combinatorial problems. Quantum kernels classify data. These are mathematical facts, verified on simulators and demonstrated on small quantum hardware. The question has never been whether the algorithms work — it's whether the hardware can run them at useful scale.

Next. The hardware gap is closing, but the target moves with the architecture. Resource estimates for breaking RSA-2048 have dropped from the 20-million-physical-qubit surface-code estimate of 2021 to sub-100,000-qubit QLDPC architecture studies in 2026, under different hardware and runtime assumptions. That does not mean practical cryptanalytic machines are imminent. It means the engineering target depends strongly on error correction, architecture, and parallelisation.

Unknown. Whether QAOA will beat classical optimisation at scale. Whether quantum kernels will find natural datasets where they outperform classical methods. Whether the quadratic speedup of amplitude estimation justifies the overhead of error correction. These are open research questions, not marketing talking points. The honest answer is: we don't know yet. This book has tried to give you the tools to follow the answers as they emerge.

The one thing to remember

If you take away a single idea from this book, let it be this: quantum computing is not about parallelism. It's not about "trying all answers at once." It's about **interference** — the ability to make wrong answers cancel and right answers reinforce, using the phase structure of quantum mechanics. Every algorithm, every speedup, every advantage traces back to this one mechanism.

The problems are real. The algorithms are sound. The hardware is coming. What remains is engineering — and the quiet, persistent work of turning mathematical possibility into practical capability.

Where to go from here

If you want to build circuits. Our companion book, the Quokka Cookbook, contains runnable recipes for every algorithm in this book — QAOA, Shor's period-finding, VQE, quantum kernels, amplitude estimation, and more — designed for the Quokka cloud platform. This book told you why; the Cookbook shows you how.

If you want to understand encodings. The transformation from molecules to qubits — fermion-to-qubit encodings — is the subject of From Molecules to Qubits. It covers Jordan-Wigner, Bravyi-Kitaev, parity, ternary tree, and compact encodings, with the algebraic structure behind all of them.

If you want the full mathematical treatment. Nielsen and Chuang's Quantum Computation and Quantum Information (2000) remains the standard reference. It covers everything in this book and more, but starts from qubits rather than problems. Szabo and Ostlund's Modern Quantum Chemistry (1996) is the classical reference for the computational chemistry foundations behind Units 3, 7, and 8.

If you want to follow the research. The arXiv preprint server (arxiv.org) is where quantum computing research appears first. The annotated bibliography below is a starting point; every paper links to its arXiv version where available.

If you want to follow the project structure. The authoritative manuscript and build source stays in the private working repo for this book while it is being prepared for publication. Runnable workbook exports are intended to live in `quantum-workbooks`, so the public notebook surface can stay separate from the book-writing source of truth.

Annotated Bibliography

The references below are the most important papers cited in this book, grouped by topic. Each annotation explains what the paper contributes and why it matters. The full citation list for each unit appears in that unit's Chef's Notes.

Quantum optimisation (Units 1, 6)

- Farhi, Goldstone, Gutmann (2014). A Quantum Approximate Optimization Algorithm. arXiv:1411.4028. The paper that introduced QAOA — the starting point for quantum combinatorial optimisation. Short, readable, and still the best introduction to the algorithm.
- Basso, Farhi, Marwaha, Villalonga, Zhou (2021). The Quantum Approximate Optimization Algorithm at High Depth for MaxCut on Large-Girth Regular Graphs and the Sherrington-Kirkpatrick Model. arXiv:2110.14206. Showed how to compute exact QAOA performance at high depth via tree tensor networks — the framework that enabled precise comparison with other algorithms.

- Lucas (2014). Ising formulations of many NP problems. Frontiers in Physics 2:5. The reference for encoding combinatorial problems as QUBO/Ising Hamiltonians. Covers graph problems, scheduling, satisfiability, and more.
- Farhi et al. (2001). A quantum adiabatic evolution algorithm applied to random instances of an NP-complete problem. Science 292:472–475 (arXiv:quant-ph/0001106). The foundational paper on quantum annealing as an algorithmic paradigm.

Cryptography and period-finding (Unit 2)

- Shor (1994). Algorithms for Quantum Computation: Discrete Logarithms and Factoring. arXiv:quant-ph/9508027. The paper that launched quantum computing as a practical field. Still worth reading for its clarity.
- Gidney and Ekerå (2021). How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. arXiv:1905.09749. The definitive resource estimate for breaking RSA with a quantum computer — the benchmark that every subsequent estimate improves on.
- Webster, Berent, Chandra, Hockings et al. (2026). The Pinnacle Architecture. arXiv:2602.11457. Reduced the quoted RSA-2048 footprint to under 100,000 physical qubits under its stated hardware and runtime assumptions, using quantum LDPC codes — a dramatic improvement over surface-code estimates.

Quantum chemistry and simulation (Units 3, 7, 8)

- Peruzzo et al. (2014). A variational eigenvalue solver on a photonic quantum processor. Nature Communications 5:4213 (arXiv:1304.3061). The first experimental demonstration of VQE — the algorithm that makes quantum chemistry possible on near-term hardware.
- McArdle, Endo, Aspuru-Guzik, Benjamin, Yuan (2020). Quantum computational chemistry. Reviews of Modern Physics 92:015003 (arXiv:1808.10402). The most comprehensive review of quantum algorithms for chemistry. Start here for the full landscape.
- Low and Chuang (2019). Hamiltonian Simulation by Qubitization. Quantum 3:163 (arXiv:1610.06546). Introduced qubitization — a fundamentally different approach to Hamiltonian simulation that achieves optimal query complexity. The theoretical foundation for next-generation quantum simulation algorithms.
- Reiher, Wiebe, Svore, Wecker, Troyer (2017). Elucidating Reaction Mechanisms on Quantum Computers. PNAS 114:7555–7560. A landmark early FeMoCo resource estimate for fault-tolerant quantum chemistry; useful as a benchmark, though later algorithms materially change the cost picture.
- Babbush, Wiebe, McClean, et al. (2018). Low-Depth Quantum Simulation of Materials. Physical Review X 8:011044. A landmark resource-estimate paper for fault-tolerant Hubbard-model simulation; best read alongside later reviews when comparing assumptions across algorithms.

Quantum machine learning (Unit 4)

- Havlíček, Córcoles, Temme, et al. (2019). Supervised learning with quantum-enhanced feature spaces. Nature 567:209–212 (arXiv:1804.11326). The experimental demonstration of quantum kernel methods — the algorithm from Unit 4 running on real hardware.
- Tang (2019). A quantum-inspired classical algorithm for recommendation systems. STOC 2019 (arXiv:1807.04271). The dequantisation result that narrowed the territory for quantum ML advantage. Essential reading for anyone claiming quantum speedups in ML.

Quantum finance (Unit 5)

- Montanaro (2015). Quantum speedup of Monte Carlo methods. arXiv:1504.06987. The theoretical foundation for quantum amplitude estimation applied to Monte Carlo — the quadratic speedup that Unit 5 is built on.
- Grover (1996). A fast quantum mechanical algorithm for database search. arXiv:quant-ph/9605043. Grover's search algorithm, which underlies amplitude estimation. The original quadratic speedup.

Textbooks

- Nielsen and Chuang (2000). Quantum Computation and Quantum Information. Cambridge University Press. The standard reference for the field. Covers algorithms, error correction, and the mathematical foundations. Dense but comprehensive.
- Szabo and Ostlund (1996). Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory. Dover. The classical reference for computational chemistry. Essential background for Units 3, 7, and 8.